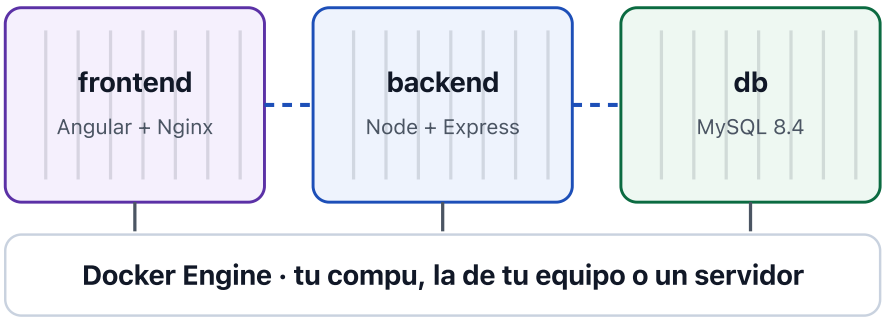
TÉCNICO UNIVERSITARIO EN PROGRAMACIÓN

Docker desde cero

Guía práctica para programadores



Guía de estudio

Del problema de “en mi computadora funciona” al deploy: imágenes, contenedores, Dockerfile, Compose, MySQL, Angular + Express y diagnóstico de errores.

Versión 1.0 · octubre de 2026

Presentación interactiva: https://pedroibanez97.github.io/docker/
Repositorio con ejemplos y práctica: https://github.com/pedroibanez97/docker

Índice

Introducción

- Cómo usar este material
- Mapa de la clase
- Antes de empezar

Módulo 1. El problema que resuelve Docker

- “En mi computadora funciona”
- ¿Qué cambia de una computadora a otra?
- Una analogía: el contenedor de transporte
- ¿Qué es Docker?
- Antes y después de Docker
- Qué resuelve Docker… y qué no
- Casos de uso reales
- Qué NO es Docker
- Recapitulación del módulo 1

Módulo 2. Conceptos fundamentales

- El mapa general
- Docker Engine, CLI y Docker Desktop
- Imagen
- Contenedor
- Imagen vs contenedor
- Contenedor vs máquina virtual
- Dockerfile
- Registro de imágenes y Docker Hub
- Docker Compose
- Volumen
- Red
- Puerto publicado
- Variables de entorno
- Resumen: las piezas de Docker
- Comprobemos

Módulo 3. Instalación desde cero

- ¿Qué hay que instalar?
- Instalación en macOS
- Instalación en Windows (con WSL 2)
- Instalación en Linux (Ubuntu)
- Abrir una terminal
- Verificar la instalación
- Tu primer contenedor: hello-world
- Ver, detener y eliminar ese contenedor
- Problemas frecuentes de instalación (1 de 2)
- Problemas frecuentes de instalación (2 de 2)
- Plan B: si la instalación no sale hoy
- Recapitulación del módulo 3

Módulo 4. Primeros comandos

- Cómo se lee un comando de Docker
- docker pull y docker images
- docker run: en primer plano y en segundo plano
- Ver el contenedor en el navegador
- docker ps y docker ps -a
- docker logs: lo que imprime el contenedor
- Ciclo de vida: stop, start, restart, rm
- docker exec: ejecutar algo dentro de un contenedor
- docker inspect: toda la ficha técnica
- docker rm y docker rmi: borrar
- Detener, eliminar y recrear no son lo mismo
- Limpieza segura
- Actividad: tu propio servidor web
- Solución: tu propio servidor web
- Resumen de comandos del módulo 4

Módulo 5. Crear una imagen propia con Dockerfile

- ¿Por qué construir nuestra propia imagen?
- La imagen base: FROM
- El proyecto de ejemplo
- El Dockerfile completo
- Instrucción por instrucción
- npm ci y el lockfile
- EXPOSE documenta; -p publica
- CMD y ENTRYPOINT
- El contexto de construcción y .dockerignore
- Capas y caché: por qué copiamos en dos pasos
- Construir y ejecutar
- Misma imagen, otra configuración
- Cambié el código… ¿y no se ve el cambio?
- Buenas prácticas (a tu medida)
- Errores habituales al dockerizar una API
- Actividad: tu versión 1.1
- Solución: tu versión 1.1
- Recapitulación del módulo 5

Módulo 6. Docker Compose

¿Por qué Compose?
Lo que vamos a construir
El compose.yaml completo
name, services, image y build
ports y environment
volumes y networks
depends_on y healthcheck: iniciado no es lo mismo que listo
El backend lee todo de variables de entorno
Los comandos de Compose
⚠ docker compose down -v borra los datos
Demo: levantar, probar y bajar
Errores habituales con Compose
Actividad: Compose en tus manos
Solución: Compose en tus manos
Recapitulación del módulo 6

Módulo 7. MySQL: datos, conexión y persistencia

MySQL en un contenedor, sin Compose
Las variables y los scripts se aplican solo la primera vez
Scripts de inicialización
¿Quién es localhost?
Host y puerto según desde dónde te conectás
Conectarse para administrar la base
¿Qué pasa con los datos cuando…?
Demo: los datos sobreviven al contenedor
Copia de seguridad y restauración
Leer los registros de MySQL
Diagnóstico de errores de conexión
Puertos y credenciales en desarrollo
Actividad: persistencia y diagnóstico
Solución: persistencia y diagnóstico
Recapitulación del módulo 7

Módulo 8. Dockerizar Angular + Express + MySQL

La arquitectura
Dos formas de ejecutar Angular (no las mezcles)
Estructura del ejemplo
Dockerfile del frontend: dos etapas
nginx.conf: archivos estáticos y proxy de /api
El código de Angular corre en el navegador
URL de la API y CORS
El compose.yaml de la app completa
Demo A: todo en Docker
Demo B: Angular con ng serve y la API en Docker
Adaptar tu proyecto existente (sin reescribirlo)
Versiones de Node para compilar Angular
Problemas habituales al dockerizar la app completa
Actividad: la app completa
Solución: la app completa
Recapitulación del módulo 8

Módulo 9. Variables de entorno y buenas prácticas

¿Qué configuración conviene separar del código?
.env y .env.example
Interpolación, environment y env_file
ENV en el Dockerfile vs environment en Compose
.gitignore vs .dockerignore
Nunca subas secretos reales a GitHub
Un .env no es un sistema de gestión de secretos
Desarrollo vs producción
Evitar credenciales inseguras
Checklist de buenas prácticas

Módulo 10. Deploy: llevar la aplicación a un servidor

¿Qué significa desplegar?
El flujo, de tu compu al servidor
¿Construir en el servidor o descargar del registro?
Publicar imágenes en un registro
compose.prod.yaml
Llevar la configuración sin subir secretos
Levantar los servicios en un servidor Linux
Puertos, firewall, DNS, HTTPS y proxy inverso (introducción)
Datos y copias de seguridad en producción
Registros, reinicios, actualizaciones y vuelta atrás
¿Qué se reutiliza y qué cambia?
Lo que Docker no garantiza por sí solo
Actividad: simular un deploy en tu compu
Solución: simular un deploy
Recapitulación del módulo 10

Módulo 11. Errores frecuentes y diagnóstico

- Un método de diagnóstico en 6 preguntas
- 1 · Docker no está iniciado
- 2 · Puerto ocupado
- 3 · El contenedor está detenido
- 4 · Imagen no encontrada
- 5 · La aplicación termina inmediatamente
- 6 · Error de conexión con MySQL
- 7 · El backend usa localhost
- 8 · Variables de entorno faltantes
- 9 · Se perdieron los datos
- 10 · Cambié el código y no se ve el cambio
- 11 · Archivos excluidos por .dockerignore
- 12 · Permisos o arquitectura incompatible
- Actividad: rompé y arreglá
- Solución: rompé y arreglá
- Recapitulación del módulo 11

Módulo 12. Repaso y cierre

- Resumen visual
- Comandos esenciales
- Preguntas de comprensión
- Ejercicios cortos
- Cuestionario (1 de 2)
- Cuestionario (2 de 2)
- Guía de decisión: ¿qué uso para…?
- ¿Mi proyecto está bien dockerizado?
- Para seguir aprendiendo
- Cierre

Práctica integradora

- Objetivos mínimos
- Paso a paso, con puntos de control
- Camino B: el proyecto base
- Entregables
- Rúbrica de evaluación (100 puntos)
- Criterios de aprobación
- Lista de verificación final (antes de entregar)

EXPLICACIÓN Introducción

Cómo usar este material

Sirve para dar la clase y para estudiar solo. Cada diapositiva tiene una etiqueta que indica qué tipo de momento es.

Tipos de diapositiva

EXPLICACIÓN conceptos con analogías, ejemplos y diagramas.

DEMO DEL DOCENTE se ejecuta en vivo; vos mirás y anotás qué pasa.

ACTIVIDAD lo hacés vos, en tu compu.

SOLUCIÓN para comparar después de intentarlo.

REPASO recapitulación y preguntas para comprobar.

Cajas que se repiten

Analogía y **dónde deja de servir** · **Resultado esperado** · **Error habitual** · **Pregunta para comprobar** (con botón para ver la respuesta).

Material complementario

- **Guía en PDF** para leer o imprimir.
- **Ejemplos completos** y probados en la carpeta `ejemplos/` del repositorio.
- **Práctica integradora** con rúbrica en `practica/`.

EXPLICACIÓN Introducción

Mapa de la clase

Doce módulos y una práctica integradora. Los tiempos son orientativos: adaptalos a tu horario.

#	Módulo	Al terminar vas a poder…	Tiempo
1	El problema que resuelve Docker	Explicar para qué sirve Docker y para qué no	20 min
2	Conceptos fundamentales	Distinguir imagen, contenedor, Dockerfile, Compose, volumen, red y puerto	40 min
3	Instalación desde cero	Instalar Docker y ejecutar tu primer contenedor	30 min + instalación
4	Primeros comandos	Crear, ver, detener, inspeccionar y borrar contenedores	45 min
5	Imagen propia con Dockerfile	Dockerizar una API de Express	50 min
6	Docker Compose	Levantar API + MySQL con un comando	50 min
7	MySQL: datos y persistencia	Conectarte, inicializar, persistir y respaldar datos	35 min
8	Angular + Express + MySQL	Dockerizar una aplicación completa	55 min
9	Variables de entorno y buenas prácticas	Configurar sin exponer secretos	25 min
10	Deploy	Entender cómo llega la app a un servidor	40 min
11	Errores frecuentes y diagnóstico	Investigar y resolver problemas con método	30 min
12	Repaso y cierre	Elegir la herramienta adecuada para cada tarea	30 min
P	Práctica integradora	Dockerizar tu propio proyecto	2–4 h

EXPLICACIÓN Introducción

Antes de empezar

Lo que conviene tener listo para que la clase sea práctica desde el primer minuto.

Alumnos

- ✓ Docker instalado y probado con `docker run hello-world` (módulo 3).
- ✓ Unos 10 GB libres en disco y, si es posible, 8 GB de RAM o más.
- ✓ Un editor (VS Code) y una terminal.
- ✓ Git para clonar los ejemplos: `git clone https://github.com/pedroibanez97/docker.git`
- ✓ Un proyecto propio Express + MySQL para la práctica final (opcional).

Docente

- ✓ Repositorio clonado y ejemplos probados en la compu que se proyecta.
- ✓ Imágenes descargadas de antemano (la red del aula puede ser lenta):

Terminal

```
docker pull hello-world
docker pull nginx:1.30-alpine
docker pull httpd:2.4-alpine
docker pull node:24-alpine
docker pull mysql:8.4
```

- ✓ Plan B para quien no pueda instalar (ver módulo 3).

MÓDULO 1

El problema que resuelve Docker

Antes de aprender comandos, entendamos qué dolor de cabeza viene a resolver.

Objetivo

Explicar con tus palabras qué es Docker, qué problemas resuelve y cuáles no.

Vamos a ver

- “En mi computadora funciona”
- Qué cambia entre computadoras
- Qué es (y qué no es) Docker
- Casos de uso reales

Tiempo estimado

20 minutos · sin computadoras todavía

EXPLICACIÓN Módulo 1 · El problema que resuelve Docker

“En mi computadora funciona”

Terminaste tu API de Express con MySQL. Funciona perfecto. Se la pasás a otra persona del equipo y…

Tu compu

- macOS · Node 24 · MySQL 8.4
- Base `tareas_db` creada a mano hace meses
- Archivo `.env` con tus credenciales

TERMINAL

```
$ npm start
API escuchando en el puerto 3000
```

La otra compu

- Windows · Node 18 · MySQL instalado en otra versión (o ninguno)
- No existe la base ni el usuario
- Sin `.env` (no se sube a GitHub, ¡bien!)

TERMINAL

```
$ npm start
Error: connect ECONNREFUSED
127.0.0.1:3306
```

IDEA CLAVE

El código es el mismo. Lo que cambia es **el entorno**: todo lo que el código necesita alrededor para funcionar.

EXPLICACIÓN Módulo 1 · El problema que resuelve Docker

¿Qué cambia de una computadora a otra?

Fuente de diferencias	Ejemplo en un proyecto Express + MySQL	Síntoma típico
Sistema operativo	Rutas `C:\` vs `/home`, mayúsculas en nombres de archivo, saltos de línea	“Funciona en Windows pero no en el servidor Linux”
Versión del runtime	Node 18 vs Node 24	Sintaxis o funciones que no existen en la versión vieja
Dependencias	`npm install` sin lockfile instala versiones distintas	Una librería se comporta distinto
Configuración	Variables de entorno, puertos, credenciales	`undefined` donde esperabas un valor
Servicios externos	MySQL instalado o no, otra versión, otra base	`ECONNREFUSED`, `Access denied`, tabla inexistente

PREGUNTA PARA COMPROBAR

De los problemas que contaron al principio de la clase, ¿en qué fila de la tabla cae cada uno?

EXPLICACIÓN Módulo 1 · El problema que resuelve Docker

Una analogía: el contenedor de transporte

ANALOGÍA

Antes de los contenedores de carga, cada barco, tren o camión se cargaba distinto: bolsas, cajones, barriles. Cada traslado era lento y algo se rompía.

Con una caja estándar, cualquier grúa, barco o camión puede mover la carga **sin saber qué hay adentro**.

EN PROGRAMACIÓN

Docker empaqueta tu aplicación junto con lo que necesita (Node, dependencias, configuración por defecto) en un formato estándar. Cualquier computadora con Docker la puede ejecutar igual.

DÓNDE DEJA DE SERVIR LA ANALOGÍA

- Un contenedor de Docker no es una caja cerrada e inerte: **ejecuta** procesos, abre puertos y se comunica con otros contenedores.
- Lo que se guarda adentro de un contenedor **se pierde** si lo borrás (salvo que uses volúmenes: módulo 7).
- No es “portable a cualquier cosa”: los contenedores de Linux necesitan un kernel Linux. En macOS y Windows, Docker Desktop lo provee con una máquina virtual liviana.

EXPLICACIÓN Módulo 1 · El problema que resuelve Docker

¿Qué es Docker?

Docker es una plataforma para empaquetar una aplicación con todo lo que necesita y ejecutarla de forma aislada y repetible en cualquier máquina que tenga Docker.

Empaquetar

Tu código + Node + dependencias + configuración por defecto quedan en una **imagen**: un paquete de solo lectura.

Ejecutar aislado

Cada imagen se ejecuta como un **contenedor**: un proceso con su propio sistema de archivos, red y variables.

Repetir igual

La misma imagen produce el mismo entorno en tu compu, en la de tu equipo y en el servidor.

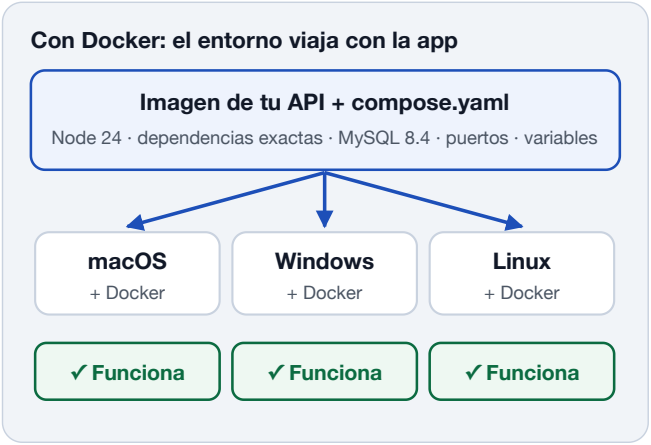
PARA SABER

Imagen y **contenedor** son los dos conceptos centrales. Los vemos en detalle en el módulo 2; por ahora: la imagen es el paquete, el contenedor es ese paquete funcionando.

EXPLICACIÓN Módulo 1 · El problema que resuelve Docker

Antes y después de Docker





README sin Docker: instalá Node 24, instalá MySQL 8.4, creá la base, creá el usuario, ejecutá el script SQL, copiá el `.env`, `npm install`, `npm start`… y rezá.

RESULTADO ESPERADO

README con Docker:

```
Terminal
git clone
https://github.com/equipo/proyecto.git
cp .env.example .env
docker compose up -d
```

EXPLICACIÓN Módulo 1 · El problema que resuelve Docker

Qué resuelve Docker… y qué no

Resuelve

- ✓ Reproducir el **mismo entorno** en cualquier compu.
- ✓ Levantar servicios (MySQL, Nginx) **sin instalarlos** en tu sistema.
- ✓ Tener **varias versiones** a la vez: MySQL 8.4 para un proyecto y 9 para otro.
- ✓ Que alguien nuevo arranque el proyecto en minutos.
- ✓ Llevar a producción **lo mismo** que probaste.

No resuelve

- ✗ Errores de tu código o un mal diseño de la base de datos.
- ✗ La seguridad por sí solo: una contraseña débil sigue siendo débil.
- ✗ La performance: no hace más rápida una consulta lenta.
- ✗ Las copias de seguridad: si no las hacés, no existen.
- ✗ Todas las diferencias: arquitectura del procesador (Intel vs ARM) y kernel siguen importando.

PREGUNTA PARA COMPROBAR

Tu API tiene un bug que devuelve tareas duplicadas. ¿Lo arregla dockerizarla?

No. Docker reproduce el entorno; el bug viaja con el código y va a aparecer igual en todas las computadoras. Eso sí: al ser reproducible, es más fácil que otra persona lo vea y lo corrija.

EXPLICACIÓN Módulo 1 · El problema que resuelve Docker

Casos de uso reales

Desarrollo

Levantar MySQL para un proyecto sin instalarlo. Borrar la base y empezar de cero en segundos.

Trabajo en equipo

Todo el equipo usa las mismas versiones. El entorno queda escrito en archivos versionados en Git.

Pruebas

Una base limpia para cada corrida de tests. Probar la app con otra versión de Node o MySQL sin romper tu compu.

Despliegue

La imagen que probaste es la que corre en el servidor. Actualizar o volver atrás es cambiar de versión de imagen.

EN PROGRAMACIÓN

Ejemplo concreto: en la empresa A usan MySQL 8.0 y en tu proyecto de la facultad MySQL 8.4. Con Docker cada proyecto tiene su propio contenedor de MySQL, con su versión, y no se pisan.

EXPLICACIÓN Módulo 1 · El problema que resuelve Docker

Qué NO es Docker

No es un lenguaje

Seguís programando en JavaScript, TypeScript y SQL. Docker ejecuta lo que ya hacés.

No es un framework

No reemplaza a Express ni a Angular, ni cambia cómo organizás tu código.

No es una máquina virtual tradicional

No emula una computadora completa con su propio sistema operativo. Los contenedores comparten el kernel del sistema (módulo 2).

No reemplaza a Git

Git versiona tu código. Docker usa ese código para construir imágenes. Se usan juntos.

No es “la nube”

Funciona en tu compu, sin Internet una vez descargadas las imágenes. Muchos servicios en la nube lo usan, pero son cosas distintas.

No es magia

Si la app necesita una variable y no se la pasás, falla igual que sin Docker.

REPASO Módulo 1 · El problema que resuelve Docker

Recapitulación del módulo 1

IDEA CLAVE

- El problema no suele ser el código sino **el entorno**.
- Docker empaqueta la app con su entorno en una **imagen** y la ejecuta como **contenedor**.
- Resuelve reproducibilidad; no resuelve bugs, seguridad ni backups por sí solo.

PREGUNTA PARA COMPROBAR

Explicale a alguien que no programa, en una oración, para qué sirve Docker.

Por ejemplo: “Sirve para que un programa funcione igual en cualquier computadora, porque viaja con todo lo que necesita”.

PREGUNTA PARA COMPROBAR

Nombrá dos cosas que cambian entre computadoras y pueden romper un proyecto Express + MySQL.

La versión de Node, la versión o existencia de MySQL, las variables de entorno, el sistema operativo, las versiones de las dependencias.

MÓDULO 2

Conceptos fundamentales

Las piezas de Docker, una por una, y cómo encajan entre sí.

Objetivo

Distinguir imagen, contenedor, Dockerfile, Compose, registro, volumen, red, puerto y variables de entorno.

Vamos a ver

- El mapa general
- Engine, CLI y Desktop
- Imagen vs contenedor
- Contenedor vs máquina virtual
- Dockerfile, registro y Compose
- Volúmenes, redes, puertos y variables

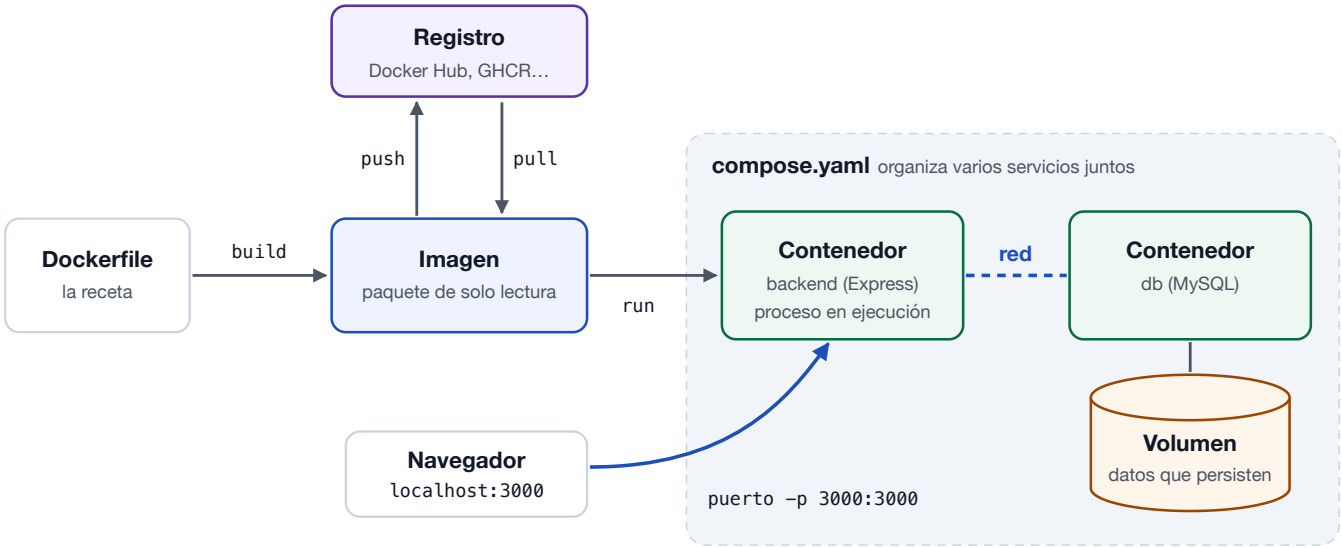
Tiempo estimado

40 minutos · conceptual, con diagramas

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

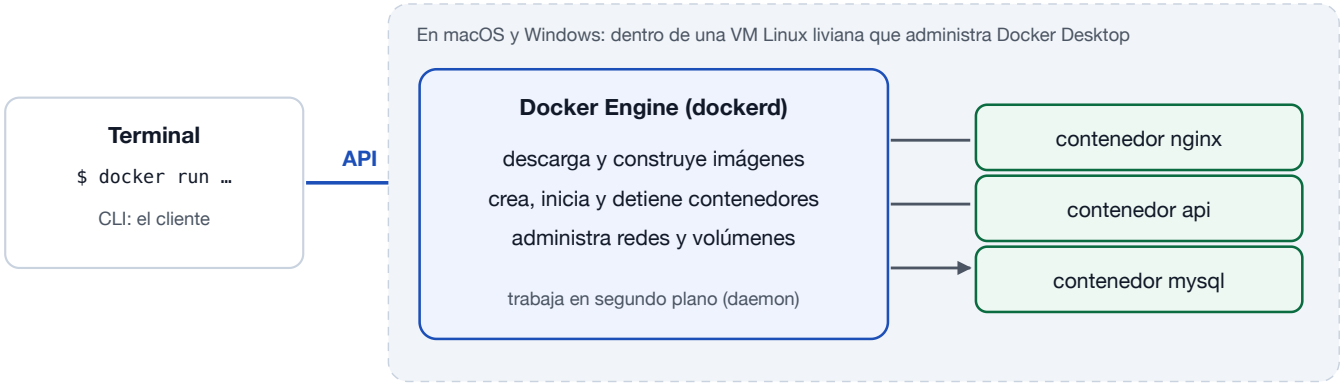
El mapa general

Todo lo que vamos a aprender cabe en este dibujo. Lo vamos a ir recorriendo.



EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Docker Engine, CLI y Docker Desktop



Docker Engine

El "motor": un servicio en segundo plano que hace el trabajo real. Necesita un kernel Linux.

CLI `docker`

El programa que usás en la terminal. Solo envía pedidos al Engine. **CLI** = interfaz de línea de comandos.

Docker Desktop

La aplicación para macOS y Windows (también existe para Linux): trae Engine, CLI, Compose, una interfaz gráfica y la VM Linux.

ANALOGÍA

La CLI es el mozo que toma el pedido; el Engine es la cocina. Si la cocina está cerrada (Docker Desktop apagado), el mozo no puede hacer nada: por eso aparece `Cannot connect to the Docker daemon`.

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Imagen

Una **imagen** es un paquete de **solo lectura** con todo lo necesario para ejecutar algo: un sistema de archivos mínimo, el runtime (Node, MySQL…), las dependencias, tu código y la indicación de qué comando ejecutar al iniciar.

- Se identifica por **nombre:etiqueta**: `mysql:8.4`, `node:24-alpine`, `api-express:1.0`.
- Está formada por **capas** que se reutilizan entre imágenes (módulo 5).
- **No cambia**: para modificarla se construye una nueva.

EN PROGRAMACIÓN

La imagen `mysql:8.4` trae MySQL 8.4 instalado y configurado para arrancar. No tenés que instalar MySQL en tu sistema.

ANALOGÍA

Una imagen es como una **receta impresa** o un **molde**: con el mismo molde podés hacer muchas tortas iguales, y el molde no se gasta.

EN PROGRAMACIÓN

Si pensás en clases y objetos: la imagen es la **clase** y cada contenedor es un **objeto** creado con `new`.

DÓNDE DEJA DE SERVIR LA ANALOGÍA

Una imagen no es código que se "instancia" en memoria como un objeto: son archivos y metadatos guardados en disco. Y la receta no es la imagen: la receta es el Dockerfile; la imagen es el resultado ya preparado.

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Contenedor

Un **contenedor** es una imagen **en ejecución** (o detenida): un proceso aislado con su propio sistema de archivos, su red, su nombre y sus variables de entorno.

- Se crea a partir de una imagen con `docker run`.
- Tiene una **capa de escritura** propia: lo que se modifica adentro queda ahí… hasta que el contenedor se borra.
- Puede estar **corriendo**, **detenido** o **eliminado**.
- Vive mientras vive su proceso principal: si la app termina, el contenedor se detiene.

ANALOGÍA

Un edificio de departamentos: cada departamento (contenedor) tiene su puerta, sus muebles y su llave, pero todos comparten los cimientos, el agua y la luz del edificio (el kernel del sistema).

DÓNDE DEJA DE SERVIR LA ANALOGÍA

Si “demolés” un departamento (borrás el contenedor), sus muebles desaparecen. En Docker, los datos importantes se guardan fuera, en un **volumen**, como una baulera del edificio.

ERROR HABITUAL

Guardar la base de datos “adentro” del contenedor y perderla al recrearlo. Lo resolvemos en el módulo 7.

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Imagen vs contenedor

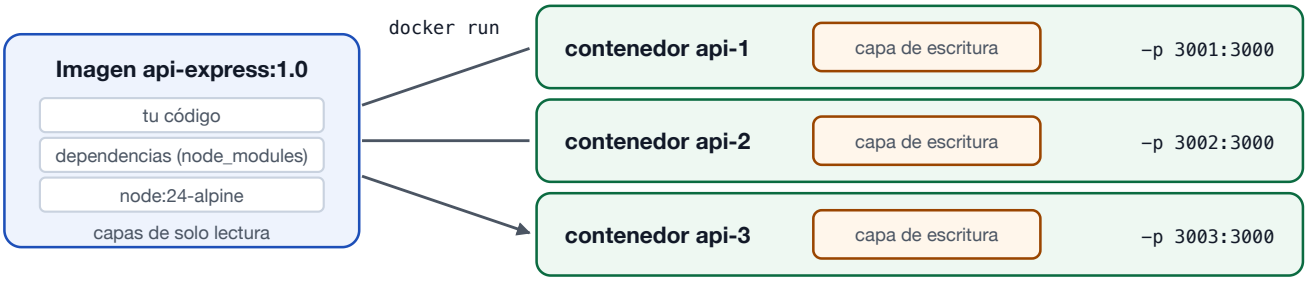


	Imagen	Contenedor
Qué es	Paquete de solo lectura	Proceso en ejecución creado desde una imagen
Analogía	Molde · clase	Torta · objeto
¿Cambia?	No; se construye otra	Sí, en su capa de escritura (se pierde al borrarlo)
Cómo se crea	`docker build` o `docker pull`	`docker run`
Cómo se lista	`docker images`	`docker ps -a`

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Contenedor vs máquina virtual



	Máquina virtual	Contenedor
Incluye	Un SO completo con su kernel	App y librerías; usa el kernel del anfitrión
Arranque	Minutos	Segundos o menos
Tamaño	Gigabytes	Decenas o cientos de MB
Aislamiento	Más fuerte	Bueno, pero comparten kernel

ANALOGÍA

Las VM son **casas**: cada una con sus cimientos y su instalación eléctrica. Los contenedores son **departamentos**: comparten la estructura del edificio.

PARA SABER

El **kernel** es el núcleo del sistema operativo: administra procesos, memoria, archivos y red. macOS y Windows no tienen kernel Linux, por eso Docker Desktop usa una VM Linux liviana y tus contenedores comparten **ese** kernel.

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Dockerfile

Un **Dockerfile** es un archivo de texto con las **instrucciones para construir una imagen**: de qué imagen partir, qué archivos copiar, qué instalar y qué comando ejecutar al iniciar.

Dockerfile (versión resumida)

```
FROM node:24-alpine
...
WORKDIR /app
...
COPY package.json package-lock.json ./
...
RUN npm ci --omit=dev
...
COPY . .
...
CMD ["node", "src/index.js"]
```

ANALOGÍA

El Dockerfile es la **receta**; la imagen es la **comida ya preparada y envasada**; el contenedor es esa comida **servida en la mesa**.

DÓNDE DEJA DE SERVIR LA ANALOGÍA

Una receta de cocina puede salir distinta cada vez. Un Dockerfile bien escrito (versiones explícitas, lockfile) produce siempre el mismo resultado.

PARA SABER

Lo escribimos completo, instrucción por instrucción, en el módulo 5.

RESULTADO ESPERADO

`docker build -t api-express:1.0 .` lee el Dockerfile y produce la imagen `api-express:1.0`.

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Registro de imágenes y Docker Hub

Un **registro** es un servidor que guarda y distribuye imágenes. **Docker Hub** (hub.docker.com (https://hub.docker.com)) es el registro público que Docker usa por defecto. Hay otros: GitHub Container Registry (GHCR), los de cada nube o uno propio de la empresa.

- `docker pull` descarga una imagen del registro.
- `docker push` sube una imagen tuya.
- Las **imágenes oficiales** (node, mysql, nginx) están mantenidas y revisadas: preferilas.

ANALOGÍA

Docker Hub es a las imágenes lo que **npm** es a los paquetes de JavaScript.

DÓNDE DEJA DE SERVIR LA ANALOGÍA

Un paquete de npm es una librería que importás en tu código. Una imagen es un entorno completo que se ejecuta: no se "importa".

Anatomía del nombre de una imagen



Si omitís partes, Docker completa: `mysql` = `docker.io/library/mysql:latest`. Una imagen tuya en Docker Hub sería `tu-usuario/tareas-backend:1.0.0`.

ATENCIÓN

`latest` no significa "la más nueva y estable": es solo la etiqueta por defecto. Usá siempre una versión explícita, como `mysql:8.4`.

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Docker Compose

Una app real tiene varias piezas: API, base de datos, frontend. **Docker Compose** describe todas en un archivo `compose.yaml` y las levanta juntas con **un comando**.

compose.yaml (resumido)

```yaml
services:
  db:
    image: mysql:8.4
...
  backend:
    build: ./backend
...
    ports:
      - "3000:3000"
```

	Dockerfile	`compose.yaml`
Describe	Cómo **construir una** imagen	Cómo **ejecutar varios** contenedores juntos
Contiene	FROM, COPY, RUN, CMD…	Servicios, puertos, variables, volúmenes, redes
Comando	`docker build`	`docker compose up`
Relación	Compose puede usar un Dockerfile con `build:` o una imagen existente con `image:`	

Terminal

```
docker compose up -d
```

ANALOGÍA

Si el Dockerfile es la receta de un plato, `compose.yaml` es el **menú del evento**: qué platos van, en qué orden y en qué mesa.

PARA SABER

Se escribe `docker compose` (con espacio). El viejo `docker-compose` con guion ya no se usa.

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Volumen



Un **volumen** es un espacio de almacenamiento que administra Docker, **fuera** del ciclo de vida del contenedor. Se “monta” en una carpeta del contenedor.

- **Volumen con nombre** (`datos-mysql:/var/lib/mysql`): lo administra Docker. Ideal para bases de datos.
- **Bind mount** (`./db/init:/docker-entrypoint-initdb.d`): una carpeta de tu compu se ve dentro del contenedor. Ideal para archivos que editás vos.

ANALOGÍA

La **baulera** del edificio: aunque vacíes o cambies de departamento, lo que guardaste en la baulera sigue ahí.

DÓNDE DEJA DE SERVIR LA ANALOGÍA

Un volumen **no es una copia de seguridad**: si lo borrás (`docker compose down -v`) o se rompe el disco, se pierde. Los backups se hacen aparte (módulo 7).

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Red

Una **red** de Docker conecta contenedores entre sí. Dentro de una misma red, cada contenedor encuentra a los otros **por su nombre de servicio**, como si fuera un dominio.

Configuración de la conexión en el backend

```
// Dentro de la red de Compose, "db" es el
nombre del servicio MySQL
host: 'db',
port: 3306,
```

ANALOGÍA

Los **internos telefónicos** de una oficina: desde adentro marcás “contabilidad” y te comunica; desde la calle no podés marcar un interno, tenés que llamar al número público (el puerto publicado).

ATENCIÓN

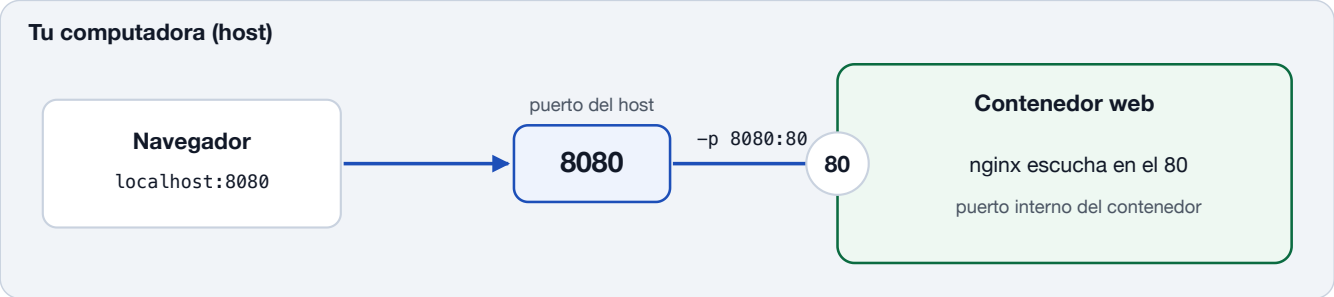
Dentro de un contenedor, `localhost` es **el propio contenedor**, no tu compu ni otro contenedor. Es el error más común del curso: lo vemos a fondo en el módulo 7.

PARA SABER

Compose crea una red automáticamente para cada proyecto. En los ejemplos la declaramos con nombre (`red-app`) para que se vea explícitamente.

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Puerto publicado



Los contenedores están aislados: desde tu compu no podés entrar a sus puertos… salvo que los **publiques**.

`-p 8080:80` se lee **"puerto de tu compu : puerto del contenedor"**. En Compose: `ports: - "8080:80"`.

Un **puerto** es un número que identifica a qué programa va una conexión en una máquina: 3306 suele ser MySQL, 80 HTTP, 3000 tu API.

ANALOGÍA

La **recepción** de un edificio: el visitante llega a la puerta 8080 de la calle y la recepción lo acompaña al departamento, puerta 80.

ERROR HABITUAL

Invertir los números (`-p 80:8080`) o publicar un puerto en el que la app no escucha. Si no estás seguro, el de la **derecha** es el de la app.

EXPLICACIÓN Módulo 2 · Conceptos fundamentales

Variables de entorno

Las **variables de entorno** son pares `NOMBRE=valor` que el sistema le pasa a un proceso. Ya las usás en Node con `process.env`.

Con Docker se usan para que **la misma imagen** funcione con **distinta configuración**: otro puerto, otra base de datos, otro mensaje, desarrollo o producción.

Terminal

```
docker run -e MENSAJE="Hola desde otra configuración" api-express:1.0
```

ANALOGÍA

Las perillas de un electrodoméstico: el aparato (imagen) es el mismo; la temperatura que elegís (variables) cambia el resultado.

ATENCIÓN

Las contraseñas también se pasan como variables, pero **nunca** se escriben dentro del Dockerfile ni se suben a GitHub. Lo vemos en el módulo 9.

src/index.js

```
const MENSAJE = process.env.MENSAJE ?? 'Hola desde un contenedor';
```

REPASO Módulo 2 · Conceptos fundamentales

Resumen: las piezas de Docker

Concepto	Qué es	Analogía	Lo vas a usar con…
Docker Engine	Servicio que construye y ejecuta	La cocina	(trabaja solo, en segundo plano)
Docker Desktop	App que trae Engine, CLI, Compose y la VM	El local completo	Abrir la app antes de trabajar
Imagen	Paquete de solo lectura	Molde · clase	`docker pull`, `docker images`
Contenedor	Imagen en ejecución	Torta · objeto · departamento	`docker run`, `docker ps`
Dockerfile	Instrucciones para construir una imagen	Receta	`docker build`
Compose	Varios servicios descriptos en un archivo	Menú del evento	`docker compose up`
Registro	Servidor que guarda imágenes	npm para imágenes	`docker push` / `pull`
Volumen	Datos fuera del contenedor	Baulera	`volumes:`
Red	Conexión entre contenedores por nombre	Internos telefónicos	`networks:`
Puerto publicado	Entrada desde tu compu al contenedor	Recepción del edificio	`-p 8080:80`
Variable de entorno	Configuración que recibe el proceso	Perillas	`-e`, `environment:`

REPASO Módulo 2 · Conceptos fundamentales

Comprobemos

PREGUNTA PARA COMPROBAR

¿Puedo tener tres contenedores corriendo a partir de la misma imagen?

Sí. Cada uno tiene su propia capa de escritura, nombre y puertos. La imagen no se modifica.

PREGUNTA PARA COMPROBAR

¿Cuál es la diferencia entre un Dockerfile y un `compose.yaml`?

El Dockerfile describe cómo construir *una* imagen. El `compose.yaml` describe cómo ejecutar *varios* contenedores juntos (con sus puertos, variables, redes y volúmenes), y puede usar Dockerfiles para construir.

PREGUNTA PARA COMPROBAR

Borré un contenedor de MySQL que no tenía volumen. ¿Qué pasó con los datos?

Un contenedor nuevo arranca con una base vacía: los datos estaban ligados a ese contenedor. Con un volumen con nombre se habrían conservado.

PREGUNTA PARA COMPROBAR

Un contenedor, ¿tiene su propio kernel como una máquina virtual?

No. Comparte el kernel del sistema anfitrión (en macOS/Windows, el de la VM Linux de Docker Desktop). Por eso arranca tan rápido y pesa menos.

MÓDULO 3

Instalación desde cero

Instrucciones para macOS, Windows y Linux, basadas en la documentación oficial de Docker (revisada en octubre de 2026).

Objetivo

Tener Docker funcionando y ejecutar, detener y borrar tu primer contenedor.

Vamos a ver

- Qué instalar según tu sistema
- Paso a paso por sistema operativo
- Verificar la instalación
- Primer contenedor: `hello-world`
- Problemas frecuentes y plan B

Tiempo estimado

30 minutos en clase, **más** la descarga e instalación (ideal: hacerla antes).

EXPLICACIÓN Módulo 3 · Instalación desde cero

¿Qué hay que instalar?

Sistema	Qué instalar (recomendado para aprender)	Requisitos principales
macOS	Docker Desktop (versión para *Apple silicon* o para *Intel*)	macOS actual o una de las dos versiones anteriores · 4 GB de RAM o más
Windows	Docker Desktop con **WSL 2**	Windows 10 22H2 (build 19045) o Windows 11 23H2 (build 22631) o posterior, 64 bits · WSL 2.1.5 o posterior · virtualización activada · 8 GB de RAM
Linux	Docker Engine + plugin Compose desde el repositorio oficial (o Docker Desktop for Linux)	Distribución de 64 bits soportada (por ejemplo Ubuntu 22.04, 24.04 o 26.04 LTS)

PARA SABER

WSL 2 (Windows Subsystem for Linux) permite ejecutar un kernel Linux real dentro de Windows. Docker Desktop lo usa para correr los contenedores.

ATENCIÓN

Licencia: Docker Desktop es gratuito para uso personal, educativo, proyectos open source no comerciales y empresas pequeñas (menos de 250 empleados *y* menos de USD 10 millones de facturación anual). Empresas más grandes necesitan una suscripción paga. Docker Engine en Linux es software libre.

Documentación oficial: macOS (https://docs.docker.com/desktop/setup/install/mac-install/) · Windows (https://docs.docker.com/desktop/setup/install/windows-install/) · Linux (Engine) (https://docs.docker.com/engine/install/) · Linux (Desktop) (https://docs.docker.com/desktop/setup/install/linux/)

EXPLICACIÓN Módulo 3 · Instalación desde cero

Instalación en macOS

1. **Averiguá tu procesador:** menú → *Acerca de esta Mac*. Si dice *Chip Apple M1/M2/M3…* es **Apple silicon**; si dice *Procesador Intel*, es **Intel**. También: `uname -m` en la Terminal muestra `arm64` o `x86_64`.
2. **Descargá** `Docker.dmg` desde la página oficial (https://docs.docker.com/desktop/setup/install/mac-install/), eligiendo la versión de tu procesador.
3. **Instalá:** abrí `Docker.dmg` y arrastrá el ícono de Docker a la carpeta *Aplicaciones*.
4. **Iniciá** *Docker* desde Aplicaciones. Aceptá el acuerdo de servicio (sin aceptarlo no arranca). Iniciar sesión es opcional.
5. **Esperá** a que el ícono de la ballena en la barra de menú indique que Docker Desktop está en ejecución (*Engine running*).
6. **Opcional en Apple silicon:** instalar Rosetta 2, que Docker recomienda para algunas herramientas.

Terminal (opcional, Apple silicon)

```
softwareupdate --install-rosetta
```

ERROR HABITUAL

Descargar la versión del procesador equivocado: la app no abre o indica que no es compatible. Borrala de Aplicaciones y descargá la otra.

PARA SABER

Docker Desktop no inicia solo al prender la compu salvo que lo configures en *Settings* → *General*. Si un comando falla con “Cannot connect to the Docker daemon”, abrí la app.

EXPLICACIÓN Módulo 3 · Instalación desde cero

Instalación en Windows (con WSL 2)

1. **Verificá tu versión:** `Win` + `R`, escribí `winver`. Necesitás Windows 10 22H2 o Windows 11 23H2 o posterior, de 64 bits.
2. **Verificá la virtualización:** Administrador de tareas → *Rendimiento* → *CPU* → “Virtualización: Habilitado”. Si dice deshabilitado, activala en la BIOS/UEFI (Intel VT-x o AMD-V/SVM).
3. **Instalá o actualizá WSL 2:** abrí **PowerShell como administrador** y ejecutá los comandos de la derecha. Reiniciá si te lo pide.
4. **Descargá** `Docker Desktop Installer.exe` desde la página oficial (https://docs.docker.com/desktop/setup/install/windows-install/).
5. **Instalá:** en la pantalla de configuración dejá marcada *Use WSL 2 instead of Hyper-V*. Al terminar, *Close* (y reiniciá si lo pide).

PowerShell (como administrador)

```
wsl --install
wsl --update
wsl --version
```

RESULTADO ESPERADO

`wsl --version` muestra la versión de WSL: tiene que ser **2.1.5 o posterior**.

PARA SABER

Los comandos de Docker se usan desde **PowerShell** o **Windows Terminal** normales; no hace falta entrar a Linux.

6 **Iniciá** Docker Desktop desde el menú Inicio (no arranca solo después de instalar). Aceptá el acuerdo y esperá a que indique *Engine running*.

EXPLICACIÓN Módulo 3 · Instalación desde cero

Instalación en Linux (Ubuntu)

Docker Engine desde el repositorio oficial. Para otras distribuciones, seguí la guía oficial de tu distribución (https://docs.docker.com/engine/install/).

1. Quitar paquetes que generan conflicto

```
sudo apt remove $(dpkg --get-selections docker.io
docker-compose docker-compose-v2 docker-doc
docker-buildx podman-docker containerd runc | cut
-f1)
```

2. Configurar el repositorio oficial

```
# Agregar la clave GPG oficial de Docker:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL
https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Agregar el repositorio a las fuentes de Apt:
sudo tee /etc/apt/sources.list.d/docker.sources
<<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo
"${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

3. Instalar y verificar

```
sudo apt install docker-ce docker-ce-cli
containerd.io docker-buildx-plugin docker-
compose-plugin
sudo systemctl status docker
sudo docker run hello-world
```

4. (Opcional) Usar docker sin sudo

```
sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world
```

> **ATENCIÓN**
> Pertenecer al grupo `docker` equivale a tener privilegios de **root** en esa máquina. Está bien en tu compu de estudio; pensalo dos veces en un servidor compartido.

5. Iniciar Docker al encender la compu

```
sudo systemctl enable docker.service
sudo systemctl enable containerd.service
```

EXPLICACIÓN Módulo 3 · Instalación desde cero

Abrir una terminal

Todos los comandos `docker` se escriben en una terminal de tu sistema. No importa la carpeta en la que estés, salvo cuando trabajemos con archivos de un proyecto.

macOS

- `⌘` + `Espacio`, escribí **Terminal** y Enter.
- O en VS Code: menú *Terminal* → *New Terminal* (`Ctrl` + `` ` ``).

Windows

- Menú Inicio, escribí **PowerShell** o **Terminal** y Enter (no hace falta como administrador para usar Docker).

- O en VS Code: *Terminal → New Terminal*.
- Si un comando de esta guía usa sintaxis de bash (por ejemplo, redirecciones con `<`), vas a ver una alternativa para PowerShell. También podés usar **Git Bash**.

Linux

- `Ctrl` + `Alt` + `T` en la mayoría de los escritorios.
- O en VS Code: *Terminal → New Terminal*.

> **DÓNDE SE EJECUTA**
>
> Cuando un comando dependa de la carpeta (por ejemplo `docker build .` o `docker compose up`), te lo vamos a indicar: hay que estar **en la carpeta del proyecto**. Usá `cd` para moverte y `ls` (o `dir` en PowerShell) para ver qué hay.

DEMO DEL DOCENTE · Módulo 3 · Instalación desde cero

Verificar la instalación

Terminal

```
docker --version
docker compose version
```

SALIDA ESPERADA (LOS NÚMEROS PUEDEN VARIAR)

```
Docker version 28.5.1, build e180ab8
Docker Compose version v2.40.3-desktop.1
```

- `docker --version`: confirma que la **CLI** está instalada.
- `docker compose version`: confirma que el **plugin de Compose** está disponible.

Terminal

```
docker info
```

FRAGMENTO DE LA SALIDA ESPERADA

```
Server:
 Containers: 0
 Images: 0
 Server Version: 28.5.1
 Operating System: Docker Desktop
 OSType: linux
 Architecture: aarch64
```

`docker info` consulta al **Engine**. Si muestra la sección *Server*, Docker está funcionando. `Architecture` dice `aarch64` en procesadores ARM (Apple silicon) y `x86_64` en Intel/AMD.

> **ERROR HABITUAL**
>
> `docker --version` funciona pero `docker info` falla con *Cannot connect to the Docker daemon*: la CLI está instalada pero el Engine no está iniciado. Abrí Docker Desktop (o `sudo systemctl start docker` en Linux).

DEMO DEL DOCENTE · Módulo 3 · Instalación desde cero

Tu primer contenedor: `hello-world`

Terminal

```
docker run hello-world
```

SALIDA (RESUMIDA)

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
...
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.
```

¿Qué pasó? (los 4 pasos, traducidos)

1. La **CLI** le pidió al **Engine** ejecutar `hello-world`.
2. El Engine no tenía la imagen y la **descargó de Docker Hub** (*Unable to find image… locally* no es un error).
3. Creó un **contenedor** a partir de la imagen, que ejecutó un programa que imprime el mensaje.
4. El Engine envió esa salida a tu terminal. El programa terminó, así que **el contenedor se detuvo**.

> **PREGUNTA PARA COMPROBAR**
>
> Si lo ejecutás de nuevo, ¿vuelve a descargar la imagen?
>
> ---
>
> No: la imagen ya está en tu compu. Se crea otro contenedor nuevo a partir de ella (y se ve solo el mensaje).

DEMO DEL DOCENTE · Módulo 3 · Instalación desde cero

Ver, detener y eliminar ese contenedor

Terminal

```
docker ps -a
```

SALIDA ESPERADA

```
CONTAINER ID   IMAGE         COMMAND    CREATED          STATUS                      PORTS   NAMES
b4b4c010a030   hello-world   "/hello"   10 seconds ago   Exited (0) 9 seconds ago            
eager_hopper
```

- `docker ps -a` lista **todos** los contenedores, también los detenidos.
- `Exited (0)`: terminó solo, sin error (código 0). No hace falta detenerlo.
- `NAMES`: si no le das nombre, Docker inventa uno (`eager_hopper`).

Terminal

```
# borrar el contenedor (usá tu ID o tu nombre)
docker rm eager_hopper

# borrar la imagen (ya no la usa ningún contenedor)
docker rmi hello-world
```

RESULTADO ESPERADO

`docker rm` imprime el nombre o ID borrado. `docker rmi` muestra `Untagged` y `Deleted`.

ERROR HABITUAL

`conflict: unable to delete hello-world:latest (must be forced) - container … is using its referenced image`: todavía existe un contenedor (aunque esté detenido) creado con esa imagen. Borrá primero los contenedores con `docker rm`.

PARA SABER

Para detener un contenedor que **sigue corriendo** se usa `docker stop`. Lo practicamos en el módulo 4.

EXPLICACIÓN Módulo 3 · Instalación desde cero

Problemas frecuentes de instalación (1 de 2)

Síntoma	Causa probable	Solución
`docker: command not found` / "no se reconoce como nombre de un cmdlet"	Terminal abierta antes de instalar, o Docker no quedó en el PATH	Cerrá y abrí la terminal. Verificá que Docker Desktop esté instalado y que haya iniciado al menos una vez.
`Cannot connect to the Docker daemon at unix:///…/docker.sock. Is the docker daemon running?`	Docker Desktop cerrado o el servicio detenido	Abrí Docker Desktop y esperá *Engine running*. Linux: `sudo systemctl start docker`.
Windows: `error during connect: … open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.`	Docker Desktop no está en ejecución	Iniciá Docker Desktop desde el menú Inicio y esperá a que termine de arrancar.
Windows: aviso de que WSL falta o está desactualizado	WSL no instalado o anterior a 2.1.5	PowerShell como administrador: `wsl --update` (o `wsl --install`) y reiniciar.
Windows: "Virtualization support not detected"	Virtualización desactivada en BIOS/UEFI	Activar Intel VT-x o AMD-V (SVM) en la BIOS/UEFI.

EXPLICACIÓN Módulo 3 · Instalación desde cero

Problemas frecuentes de instalación (2 de 2)

Síntoma	Causa probable	Solución
Linux: `permission denied while trying to connect to the Docker daemon socket`	Tu usuario no tiene permiso	Usá `sudo` o agregate al grupo `docker` y volvé a iniciar sesión.
`docker-compose: command not found`	Comando viejo (v1)	Usá `docker compose` con espacio. En Linux, instalá `docker-compose-plugin`.
Docker Desktop queda “Starting…” para siempre	Poca RAM o disco, VM trabada	Reiniciá Docker Desktop (menú de la ballena → *Restart*) o la compu. Liberá espacio en disco.
`TLS handshake timeout` o descargas que no avanzan	Red lenta, proxy o firewall	Reintentá, probá otra red o configurá el proxy en *Settings → Resources → Proxies*.
`no space left on device`	Disco o disco virtual de Docker lleno	`docker system df` para ver el uso; borrá contenedores e imágenes que no uses (módulo 4).

EXPLICACIÓN Módulo 3 · Instalación desde cero

Plan B: si la instalación no sale hoy

Trabajar en pareja

Con alguien que tenga Docker funcionando. Alternen quién escribe los comandos.

GitHub Codespaces

Un entorno de desarrollo en el navegador. La imagen por defecto incluye Docker; verificá con `docker --version`. Tiene horas gratuitas mensuales (más con el GitHub Student Developer Pack). Probalo antes de la clase.

Seguir con el material

Toda la clase tiene la salida esperada de cada comando. Leé las demos, anotá dudas y resolvé la instalación con la tabla anterior.

ATENCIÓN

En compus de laboratorio sin permisos de administrador no se puede instalar Docker Desktop. Consultá con quien administra el laboratorio o usá el plan B.

REPASO Módulo 3 · Instalación desde cero

Recapitulación del módulo 3

IDEA CLAVE

- macOS y Windows: **Docker Desktop** (en Windows, con WSL 2). Linux: **Docker Engine** + plugin Compose.
- `docker --version` prueba la CLI; `docker info` prueba el Engine.
- `docker run hello-world` descarga una imagen, crea un contenedor, lo ejecuta y termina.
- Los contenedores detenidos siguen existiendo hasta que los borrás con `docker rm`.

PREGUNTA PARA COMPROBAR

Abrís una terminal, escribís `docker run hello-world` y aparece *Cannot connect to the Docker daemon*. ¿Qué revisás primero?

Que Docker Desktop esté abierto y diga *Engine running* (o que el servicio `docker` esté activo en Linux). La CLI funciona, pero no hay Engine que la atienda.

MÓDULO 4

Primeros comandos

Con un servidor web (nginx) vamos a descargar, ejecutar, mirar por dentro, detener, borrar y recrear contenedores.

Objetivo

Manejar el ciclo de vida de un contenedor y publicar un puerto para verlo en el navegador.

Comandos

`pull` · `images` · `run` · `ps` · `logs` · `stop` · `start` · `restart` · `exec` · `inspect` · `rm` · `rmi`

Tiempo estimado

45 minutos · cada uno en su compu

EXPLICACIÓN Módulo 4 · Primeros comandos

Cómo se lee un comando de Docker

```
docker run -d --name web -p 8080:80 nginx:1.30-alpine
```

programa (CLI)	acción	opciones (cómo)	argumento: qué imagen

- Las opciones empiezan con `-` (cortas: `-d`) o `--` (largas: `--name`).
- A un contenedor lo podés nombrar por su **nombre** (`web`) o por su **ID**; alcanza con los primeros caracteres del ID (`4084`).
- Muchos comandos tienen dos formas: `docker ps` = `docker container ls`; `docker images` = `docker image ls`.

La ayuda siempre está a mano

```
docker --help
docker run --help
```

DÓNDE SE EJECUTA

Estos comandos funcionan desde cualquier carpeta, en la terminal de tu sistema (Terminal, PowerShell o la de VS Code).

DEMO DEL DOCENTE Módulo 4 · Primeros comandos

`docker pull` y `docker images`

Para qué: descargar una imagen de un registro sin ejecutarla todavía.

Terminal

```
docker pull nginx:1.30-alpine
```

SALIDA ESPERADA (PRIMERA VEZ)

```
1.30-alpine: Pulling from library/nginx
...
Digest: sha256:970364228b1e4f93...
Status: Downloaded newer image for nginx:1.30-alpine
docker.io/library/nginx:1.30-alpine
```

Si ya la tenías: `Status: Image is up to date`. La etiqueta `1.30-alpine` es nginx 1.30 sobre Alpine Linux, una distribución mínima.

Para qué: ver qué imágenes tenés en tu compu.

Terminal

```
docker images
```

SALIDA ESPERADA

```
REPOSITORY   TAG           IMAGE ID       CREATED      SIZE
nginx        1.30-alpine   970364228b1e   4 days ago   101MB
```

- **REPOSITORY** y **TAG**: nombre y versión.
- **IMAGE ID**: identificador único (abreviado).
- **CREATED**: cuándo se *construyó* la imagen, no cuándo la descargaste.

> PARA SABER
>
> Según la versión de Docker, las columnas pueden cambiar un poco de nombre o de orden. La información es la misma.

DEMO DEL DOCENTE · Módulo 4 · Primeros comandos

`docker run`: en primer plano y en segundo plano

En primer plano

Terminal

```
docker run nginx:1.30-alpine
```

SALIDA (RESUMIDA)

```
/docker-entrypoint.sh: Configuration complete;
ready for start up
... [notice] 1#1: nginx/1.30.6
... [notice] 1#1: start worker processes
```

La terminal queda "tomada" mostrando los registros. `Ctrl` + `C` detiene el contenedor. Útil para ver errores al instante; incómodo para trabajar.

En segundo plano, con nombre y puerto

Terminal

```
docker run -d --name web -p 8080:80
nginx:1.30-alpine
```

SALIDA ESPERADA: EL ID DEL CONTENEDOR

```
408427da62fed8ab2eccdfc1e7034244d3eeb5fac60cad
8fa924f0e1bf5a6625
```

- `-d` (*detached*): corre en segundo plano y te devuelve la terminal.
- `--name web`: un nombre fácil para referirte a él.
- `-p 8080:80`: publica el puerto 80 del contenedor en el 8080 de tu compu.

> IDEA CLAVE
>
> `docker run` hace tres cosas: descarga la imagen si no está (*pull*), **crea** un contenedor nuevo y lo **inicia**. Cada `run` crea un contenedor **distinto**.

DEMO DEL DOCENTE Módulo 4 · Primeros comandos

Ver el contenedor en el navegador

Con el contenedor `web` corriendo, abrí **http://localhost:8080** en tu navegador.

RESULTADO ESPERADO

La página "**Welcome to nginx!**". Tu navegador habló con el puerto 8080 de tu compu y Docker reenvió la conexión al puerto 80 del contenedor.

También desde la terminal:

Terminal

```
curl http://localhost:8080
```

ERROR HABITUAL

`Bind for 0.0.0.0:8080 failed: port is already allocated` o `address already in use`: otro programa (u otro contenedor) ya usa el 8080 de tu compu. Elegí otro número a la izquierda: `-p 8081:80`.

PREGUNTA PARA COMPROBAR

¿Qué pasa si ejecutás el contenedor **sin** `-p`?

El contenedor funciona, nginx escucha en su puerto 80, pero desde tu compu no hay forma de llegar: ningún puerto de tu compu está conectado a él.

PREGUNTA PARA COMPROBAR

¿Podés tener dos contenedores nginx a la vez, uno en 8080 y otro en 8081?

Sí: cada uno escucha en su propio puerto 80 interno, y en tu compu se publican en puertos distintos. Lo que no se puede es usar el mismo puerto de tu compu dos veces.

DEMO DEL DOCENTE Módulo 4 · Primeros comandos

`docker ps` y `docker ps -a`

Terminal

```
docker ps
```

SALIDA ESPERADA (CONTENEDORES EN EJECUCIÓN)

```
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS
NAMES
408427da62fe   nginx:1.30-alpine   "/docker-entrypoint.…"   2 minutes ago   Up 2 minutes
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   web
```

- **STATUS** `Up 2 minutes`: corriendo desde hace 2 minutos.
- **PORTS** `0.0.0.0:8080->80/tcp`: el 8080 de tu compu (en todas sus interfaces) va al 80 del contenedor. `[::]` es lo mismo para IPv6.
- **COMMAND**: el proceso principal del contenedor.

Terminal

```
docker ps -a
```

Muestra **también los detenidos**, con `Exited (código)`. El código `0` significa que terminó bien; cualquier otro, que terminó con error.

ERROR HABITUAL

"Mi contenedor desapareció": no aparece en `docker ps` porque está detenido. Buscalo con `docker ps -a` y mirá sus registros.

DEMO DEL DOCENTE Módulo 4 · Primeros comandos

`docker logs`: lo que imprime el contenedor

Terminal

```
docker logs web
docker logs --tail 20 web    # solo las últimas 20 líneas
docker logs -f web           # seguir en vivo (Ctrl+C para salir)
```

DESPUÉS DE VISITAR LA PÁGINA

```
... [notice] 1#1: start worker processes
192.168.65.1 - - [19/Oct/2026:18:43:25 +0000] "GET / HTTP/1.1" 200 896 "-" "Mozilla/5.0 ..."
```

Para qué: ver todo lo que el proceso principal escribe en la salida estándar y de error. Es lo mismo que ves en la consola cuando hacés `npm start`.

IDEA CLAVE

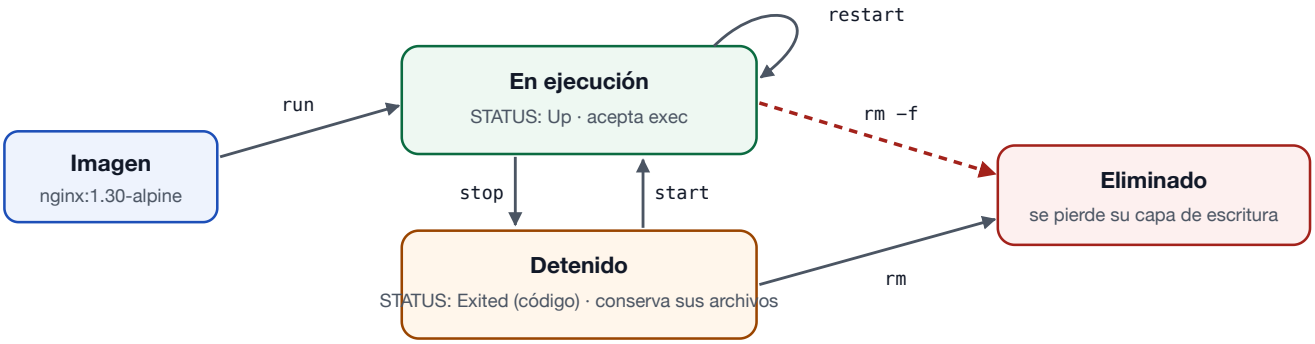
Ante cualquier problema, lo primero es `docker logs`. Ahí aparecen los errores de tu app, de MySQL o de nginx.

ATENCIÓN

`Ctrl` + `C` en `docker logs -f` deja de **mostrar** los registros; **no** detiene el contenedor.

EXPLICACIÓN Módulo 4 · Primeros comandos

Ciclo de vida: `stop`, `start`, `restart`, `rm`



Terminal

```
docker stop web      # detiene (envía SIGTERM; si no termina en 10 s, lo fuerza)
docker start web     # vuelve a iniciar EL MISMO contenedor, con la misma configuración
docker restart web   # stop + start
```

PARA SABER

SIGTERM es la señal que le pide a un proceso que termine de forma ordenada. Por eso en nuestra API de Express vamos a escuchar `process.on('SIGTERM', …)`.

DEMO DEL DOCENTE Módulo 4 · Primeros comandos

`docker exec`: ejecutar algo dentro de un contenedor

Abrir una terminal adentro

```
docker exec -it web sh
```

- `-it`: modo interactivo con terminal (para escribir comandos adentro).
- `sh`: el intérprete de comandos. Las imágenes Alpine traen `sh`, no `bash`.

Ya adentro del contenedor

```
ls /usr/share/nginx/html
cat /etc/os-release
exit
```

SALIDA ESPERADA

```
50x.html    index.html
NAME="Alpine Linux"
ID=alpine
...
```

- `exit` sale del contenedor **sin** detenerlo.

Para un solo comando, sin entrar:

Terminal

```
docker exec web nginx -v
```

ERROR HABITUAL

`Error response from daemon: container … is not running`: `exec` solo funciona con contenedores en ejecución. Inicialo con `docker start`.

ERROR HABITUAL

`exec: "bash": executable file not found`: esa imagen no trae bash. Usá `sh`.

DEMO DEL DOCENTE · Módulo 4 · Primeros comandos

`docker inspect`: toda la ficha técnica

Terminal

```
docker inspect web
```

SALIDA (COMIENZO DE UN JSON LARGO)

```
[
    {
        "Id":
"408427da62fed8ab2eccdfc1e70342...",
        "Created": "2026-10-
19T18:43:24.495114508Z",
        "Path": "/docker-entrypoint.sh",
        "State": { "Status": "running", ... },
        ...
```

Para qué: ver la configuración real de un contenedor (o imagen, red, volumen): estado, variables de entorno, puertos, volúmenes montados, IP.

Para extraer un solo dato:

Terminal

```
docker inspect --format '{{.State.Status}}'
web
```

SALIDA ESPERADA

```
running
```

ATENCIÓN

`inspect` muestra las variables de entorno, **incluidas las contraseñas**. No compartas su salida completa en capturas o foros.

En `cmd.exe` de Windows, cambiá las comillas simples por dobles. En PowerShell funcionan tal cual.

DEMO DEL DOCENTE Módulo 4 · Primeros comandos

`docker rm` y `docker rmi`: borrar

Contenedores

Terminal

```
docker rm web
```

SI TODAVÍA ESTÁ CORRIENDO

```
Error response from daemon: cannot remove container "web": container is running: stop the container before removing or force remove
```

Terminal

```
docker stop web
docker rm web
# o en un paso: detener y borrar
docker rm -f web
```

Imágenes

Terminal

```
docker rmi nginx:1.30-alpine
```

SI ALGÚN CONTENEDOR LA USA

```
Error response from daemon: conflict: unable to delete nginx:1.30-alpine (must be forced) - container 91378e8b9e8e is using its referenced image 970364228b1e
```

Borrá primero los contenedores que la usan (aunque estén detenidos) y después la imagen.

> **ATENCIÓN**
> Antes de borrar, mirá qué vas a borrar: `docker ps -a` y `docker images`. Borrar un contenedor es **definitivo**.

DEMO DEL DOCENTE Módulo 4 · Primeros comandos

Detener, eliminar y recrear no son lo mismo

1. Modificamos un archivo **dentro** del contenedor:

   Terminal

   ```
   docker exec web sh -c 'echo Hola desde adentro del contenedor > /usr/share/nginx/html/index.html'
   ```

   Recargá `localhost:8080`: se ve el texto nuevo.

2. **Detener e iniciar:** `docker stop web` y `docker start web`. Recargá: **el cambio sigue**.

3. **Eliminar y recrear:**

   Terminal

   ```
   docker rm -f web
   docker run -d --name web -p 8080:80 nginx:1.30-alpine
   ```

   Recargá: volvió “Welcome to nginx!”. **El cambio se perdió.**

Acción	Proceso	Contenedor	Archivos modificados adentro
Detener (`stop`)	Termina	Sigue existiendo	**Se conservan**
Iniciar (`start`)	Arranca de nuevo	El mismo	**Se conservan**
Eliminar (`rm`)	—	Desaparece	**Se pierden**
Recrear (`rm` + `run`)	Nuevo	Uno nuevo desde la imagen	**Empieza “de fábrica”**

> **IDEA CLAVE**
> Nunca guardes datos importantes solo dentro de un contenedor. Recrear contenedores es normal (para actualizar, cambiar configuración…): los datos van en **volúmenes**.

PARA SABER

Las opciones de `run` (puertos, variables, nombre) quedan fijas al crear el contenedor. Para cambiarlas hay que **recrearlo**.

EXPLICACIÓN Módulo 4 · Primeros comandos

Limpieza segura

Terminal

```
# ¿Cuánto espacio usa Docker?
docker system df

# Borrar los contenedores DETENIDOS (pide confirmación)
docker container prune

# Borrar imágenes "colgadas" (sin nombre, restos de builds)
docker image prune
```

DOCKER SYSTEM DF

```
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          5         1         1.2GB     900MB (75%)
Containers      3         1         2.1MB     1.9MB (90%)
Local Volumes   2         1         210MB     105MB (50%)
Build Cache     40        0         350MB     350MB
```

ATENCIÓN

No uses como limpieza habitual:

Terminal

```
docker system prune -a --volumes
```

Borra **todas** las imágenes que no estén en uso, todos los contenedores detenidos, redes y **volúmenes** sin usar. Puede eliminar bases de datos de otros proyectos y obligarte a descargar todo de nuevo.

IDEA CLAVE

Regla: primero **mirá** (`docker ps -a`, `docker images`, `docker volume ls`), después borrá lo que sabés qué es, con el comando más específico posible.

ACTIVIDAD Módulo 4 · Primeros comandos

Actividad: tu propio servidor web

Usá la imagen oficial de Apache httpd. Tiempo: 10 minutos.

1. Descargá la imagen `httpd:2.4-alpine` y verificá que aparece en tu lista de imágenes.

2. Ejecutá un contenedor llamado `apache`, en segundo plano, publicado en el puerto **8081** de tu compu. Apache escucha en el **80** del contenedor.
3. Abrí la página en el navegador. ¿Qué texto aparece?
4. Mirá los registros y encontrá la línea de tu visita.
5. Listá los archivos de `/usr/local/apache2/htdocs` dentro del contenedor.
6. Averiguá con `inspect` el estado del contenedor.
7. Detené y borrá el contenedor. Comprobá que ya no aparece en `docker ps -a`.

SOLUCIÓN Módulo 4 · Primeros comandos

Solución: tu propio servidor web

Terminal

```
docker pull httpd:2.4-alpine
docker images httpd

docker run -d --name apache -p 8081:80
httpd:2.4-alpine
# navegador: http://localhost:8081

docker logs apache
docker exec apache ls
/usr/local/apache2/htdocs
docker inspect --format '{{.State.Status}}'
apache

docker stop apache
docker rm apache
docker ps -a
```

RESULTADO ESPERADO

- El navegador muestra **It works!**
- En los logs, una línea con `"GET / HTTP/1.1" 200` (o `304` si recargaste)
- `ls` muestra `index.html`
- `inspect` responde `running`
- Después de `rm`, `apache` ya no figura en `docker ps -a`.

ERROR HABITUAL

Si usaste `-p 80:8081`, el navegador no carga: los números están invertidos. Recreá el contenedor con `-p 8081:80`.

REPASO Módulo 4 · Primeros comandos

Resumen de comandos del módulo 4

Comando	Para qué	Dato clave
`docker pull imagen:tag`	Descargar una imagen	`run` la descarga sola si falta
`docker images`	Listar imágenes locales	= `docker image ls`
`docker run -d --name x -p H:C imagen`	Crear e iniciar un contenedor	Cada `run` crea uno nuevo
`docker ps` / `docker ps -a`	Listar contenedores	`-a` incluye los detenidos
`docker logs [-f] x`	Ver lo que imprime	Primer paso ante cualquier problema
`docker stop` / `start` / `restart x`	Detener / iniciar / reiniciar	Conserva el contenedor y sus archivos
`docker exec -it x sh`	Entrar a un contenedor	Solo si está corriendo
`docker inspect x`	Ver toda su configuración	Muestra secretos: cuidado al compartir
`docker rm [-f] x`	Eliminar un contenedor	Se pierde lo que tenía adentro
`docker rmi imagen`	Eliminar una imagen	Antes, borrar sus contenedores

MÓDULO 5

Crear una imagen propia con Dockerfile

Dockerizamos una API mínima de Node.js + Express, instrucción por instrucción.

Objetivo

Escribir un Dockerfile, construir la imagen, ejecutarla, ver sus registros y reconstruirla cuando cambia el código.

Material

Carpeta `ejemplos/01-api-express` del repositorio.

Tiempo estimado

50 minutos · demo + actividad

EXPLICACIÓN Módulo 5 · Crear una imagen propia con Dockerfile

¿Por qué construir nuestra propia imagen?

¿QUÉ PROBLEMA RESUELVE?

Para nginx o MySQL ya existen imágenes oficiales. Pero **tu API no está en Docker Hub**: nadie publicó una imagen con tu código y tus dependencias.

ANALOGÍA

Es como escribir la receta de tu plato: partís de ingredientes comprados (la imagen de Node) y le agregás los tuyos (dependencias y código), en un orden preciso.

Necesitamos una imagen que contenga:

1. Un sistema mínimo con **Node.js** instalado.
2. Las **dependencias** exactas de tu `package-lock.json`.
3. **Tu código**.
4. La indicación de **qué comando** ejecutar al iniciar.

IDEA CLAVE

Una vez construida, cualquiera la ejecuta con `docker run`, **sin** tener Node instalado.

EXPLICACIÓN Módulo 5 · Crear una imagen propia con Dockerfile

La imagen base: `FROM`

La **imagen base** es el punto de partida: un sistema mínimo con el runtime ya instalado. Para Node usamos la imagen **oficial** `node`.

PARA SABER

Node **24** es la versión **LTS** (soporte extendido) activa en octubre de 2026. Usá la misma versión mayor que usás en desarrollo, o la que pida tu framework.

ATENCIÓN

Alpine usa `musl` en lugar de `glibc`. Casi todo funciona igual; si una dependencia con código nativo falla al instalar, probá con `-slim`.

Etiqueta	Qué trae	Uso sugerido
`node:24-alpine`	Node 24 sobre Alpine Linux (muy chica)	APIs como las nuestras ✔
`node:24-slim`	Node 24 sobre Debian reducido	Si alguna dependencia nativa falla en Alpine
`node:24`	Node 24 sobre Debian completo (grande)	Si necesitás muchas herramientas del sistema
`node` / `node:latest`	La última versión que se publique	**Evitar: cambia sin aviso**

PREGUNTA PARA COMPROBAR

¿Por qué no usar `FROM node` sin etiqueta?

Porque equivale a `node:latest`: el día que salga una versión mayor nueva, tu imagen cambia sin que toques nada y puede romperse. Una versión explícita hace la construcción reproducible.

El proyecto de ejemplo

```
ejemplos/01-api-express/
├── Dockerfile          # receta de la imagen
├── .dockerignore       # qué NO enviar al
build
├── package.json
├── package-lock.json   # versiones exactas
└── src/
    └── index.js        # la API
```

ejemplos/01-api-express/package.json

```
{
  "name": "api-express",
  "version": "1.0.0",
  "description": "API mínima de Express para
practicar Dockerfile",
  "type": "module",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js"
  },
  "dependencies": {
    "express": "^5.3.0"
  }
}
```

ejemplos/01-api-express/src/index.js

```
import express from 'express';
import os from 'node:os';

const app = express();

// La configuración llega por variables de
entorno (con valores por defecto).
const PORT = Number(process.env.PORT ?? 3000);
const MENSAJE = process.env.MENSAJE ?? 'Hola
desde un contenedor';

app.get('/', (req, res) => {
  res.send(`<h1>${MENSAJE}</h1><p>Probá también
<a href="/api/saludo">/api/saludo</a></p>`);
});

app.get('/api/saludo', (req, res) => {
  res.json({
    mensaje: MENSAJE,
    // Dentro de un contenedor, el hostname es el
ID corto del contenedor.
    servidor: os.hostname(),
    hora: new Date().toISOString(),
  });
});

const server = app.listen(PORT, () => {
  console.log(`API escuchando en el puerto
${PORT}`);
});

// `docker stop` envía la señal SIGTERM: cerramos
el servidor de forma ordenada.
process.on('SIGTERM', () => {
  console.log('SIGTERM recibido: cerrando el
servidor...');
  server.close(() => process.exit(0));
});
```

El Dockerfile completo

ejemplos/01-api-express/Dockerfile

```
# 1. Imagen base oficial, con versión explícita (Node 24 LTS sobre Alpine Linux)
FROM node:24-alpine

# 2. Carpeta de trabajo dentro de la imagen: los comandos siguientes se ejecutan acá
WORKDIR /app

# 3. Primero copiamos SOLO los archivos de dependencias (aprovecha la caché)
COPY package.json package-lock.json ./

# 4. Instalamos las versiones exactas del lockfile, sin dependencias de desarrollo
RUN npm ci --omit=dev

# 5. Recién ahora copiamos el resto del código (.dockerignore filtra lo que no va)
COPY . .

# 6. Valores por defecto de configuración (se pueden cambiar al ejecutar)
ENV NODE_ENV=production
ENV PORT=3000
ENV MENSAJE="Hola desde un contenedor"

# 7. Documentamos el puerto en el que escucha la app (NO lo publica)
EXPOSE 3000

# 8. No ejecutamos como root: la imagen oficial trae el usuario "node"
USER node

# 9. Comando que se ejecuta al iniciar cada contenedor
CMD ["node", "src/index.js"]
```

DÓNDE SE EJECUTA

El archivo se llama exactamente `Dockerfile` (sin extensión) y va en la **raíz del proyecto**, junto a `package.json`.

IDEA CLAVE

Cada instrucción se ejecuta en orden, de arriba hacia abajo, y genera una **capa** de la imagen.

PARA SABER

Las líneas que empiezan con `#` son comentarios.

EXPLICACIÓN Módulo 5 · Crear una imagen propia con Dockerfile

Instrucción por instrucción

Instrucción	Qué hace	En nuestro ejemplo
`FROM`	Elige la imagen base. Siempre es la primera.	`node:24-alpine`: Node 24 listo para usar
`WORKDIR`	Crea (si no existe) y se ubica en una carpeta dentro de la imagen. Como un `cd` permanente.	Todo pasa en `/app`
`COPY origen destino`	Copia archivos de tu proyecto (el *contexto*) a la imagen.	Primero los `package*.json`, después el resto
`RUN`	Ejecuta un comando **al construir** la imagen. El resultado queda guardado en ella.	`npm ci --omit=dev` instala dependencias
`ENV`	Define variables de entorno **por defecto**, que se pueden cambiar al ejecutar.	`NODE_ENV`, `PORT`, `MENSAJE`
`EXPOSE`	**Documenta** en qué puerto escucha la app. No publica nada.	3000
`USER`	Usuario con el que se ejecuta el proceso.	`node`, sin privilegios de administrador
`CMD`	Comando que se ejecuta **al iniciar cada contenedor**.	`node src/index.js`

IDEA CLAVE

`RUN` se ejecuta **una vez, al construir**. `CMD` se ejecuta **cada vez que arranca** un contenedor. Confundirlos es un error típico: `RUN node src/index.js` intentaría iniciar el servidor durante el build y el build no terminaría nunca.

EXPLICACIÓN Módulo 5 · Crear una imagen propia con Dockerfile

`npm ci` y el lockfile

- `npm ci` instala **exactamente** lo que dice `package-lock.json`. Si el lockfile no coincide con `package.json`, falla en vez de "adivinar".
- `--omit=dev` no instala las `devDependencies` (nodemon, linters, tests): no hacen falta para ejecutar.
- Por eso copiamos `package-lock.json` y **lo subimos a Git**.

ERROR HABITUAL

`"/package-lock.json": not found` al construir: no existe el lockfile o está excluido por `.dockerignore`. Generalo con `npm install` en tu compu y versionalo.

	`npm install`	`npm ci`
Usa el lockfile	Como guía, puede actualizarlo	Al pie de la letra
Resultado	Puede variar con el tiempo	Siempre igual
Uso típico	Desarrollo, agregar paquetes	Builds, CI, Docker

PARA SABER

No copies tu carpeta `node_modules` a la imagen: puede tener binarios compilados para tu sistema operativo (macOS o Windows) que no funcionan en Linux. Las dependencias se instalan **dentro** de la imagen.

EXPLICACIÓN Módulo 5 · Crear una imagen propia con Dockerfile

`EXPOSE` documenta; `-p` publica

dockerfile

```
# 7. Documentamos el puerto en el que escucha la app (NO lo publica)
EXPOSE 3000
```

`EXPOSE 3000` es una **nota** para quien use la imagen: "esta app escucha en el 3000". No abre ni publica ningún puerto.

Para entrar desde tu compu igual hace falta:

Terminal

```
docker run -p 3000:3000 api-express:1.0
```

ANALOGÍA

`EXPOSE` es el cartel "Atención al público: puerta 3000" pegado adentro del edificio. `-p` es abrir esa puerta hacia la calle.

ERROR HABITUAL

"Puse `EXPOSE 3000` y no puedo entrar desde el navegador". Falta `-p 3000:3000` (o `ports:` en Compose).

ATENCIÓN

Tu app tiene que escuchar en **todas las interfaces** del contenedor. `app.listen(PORT)` lo hace por defecto. Si escribís `app.listen(PORT, '127.0.0.1')` o `'localhost'`, solo acepta conexiones desde adentro del contenedor y el puerto publicado no responde.

EXPLICACIÓN Módulo 5 · Crear una imagen propia con Dockerfile

`CMD` y `ENTRYPOINT`

`CMD`: el comando por defecto

dockerfile

```
CMD ["node", "src/index.js"]
```

Se puede **reemplazar** al ejecutar, agregando un comando al final de `docker run`:

Terminal

```
docker run --rm api-express:1.0 node --version
```

SALIDA

```
v24.22.0
```

`--rm` borra el contenedor apenas termina: útil para comandos de una sola vez.

`ENTRYPOINT`: el ejecutable fijo

Define un programa que **siempre** se ejecuta; lo que pongas en `CMD` o al final de `docker run` se le pasa como argumentos. Se usa en imágenes que funcionan como "herramienta" (por ejemplo, una CLI). Para una API, `CMD` alcanza.

IDEA CLAVE

Usá la forma con corchetes (*exec form*): `CMD ["node", "src/index.js"]`. Así `node` recibe directamente la señal `SIGTERM` de `docker stop` y puede cerrar en orden.

PARA SABER

Si el proceso no termina dentro del tiempo de gracia (10 s por defecto), Docker lo fuerza con `SIGKILL` y el estado queda `Exited (137)`.

EXPLICACIÓN Módulo 5 · Crear una imagen propia con Dockerfile

El contexto de construcción y `.dockerignore`



En `docker build -t api-express:1.0 .`, el punto final es el **contexto**: la carpeta cuyos archivos se envían al constructor. `COPY` solo puede copiar archivos de ahí.

ERROR HABITUAL

Ejecutar `docker build` desde otra carpeta: `failed to read dockerfile: open Dockerfile: no such file or directory`. Ubicate en la carpeta del proyecto.

.dockerignore

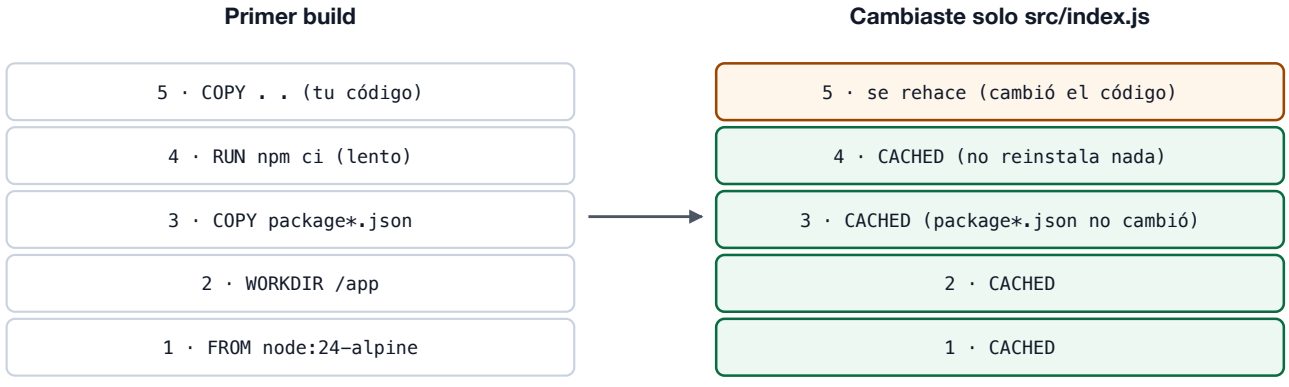
```
# Las dependencias se instalan DENTRO de la imagen con npm ci
node_modules
npm-debug.log*

# Configuración local y secretos: nunca dentro de la imagen
.env

# Archivos que la aplicación no necesita para ejecutarse
.git
.gitignore
Dockerfile
.dockerignore
*.md
```

EXPLICACIÓN Módulo 5 · Crear una imagen propia con Dockerfile

Capas y caché: por qué copiamos en dos pasos



IDEA CLAVE

Docker reutiliza una capa si la instrucción y los archivos que usa no cambiaron. **Cuando una capa cambia, todas las de arriba se rehacen.** Por eso lo que cambia seguido (tu código) va al final, y lo lento (`npm ci`) antes.

ASÍ SE VE AL RECONSTRUIR DESPUÉS DE CAMBIAR EL CÓDIGO

```
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY package.json package-lock.json ./
 => CACHED [4/5] RUN npm ci --omit=dev
 => [5/5] COPY . .
```

PREGUNTA PARA COMPROBAR

¿Qué pasaría con `COPY . .` **antes** de `RUN npm ci`?

Cualquier cambio en el código invalidaría esa capa y todas las siguientes: `npm ci` se ejecutaría en cada build, aunque las dependencias no hayan cambiado. Funciona, pero cada build tarda mucho más.

DEMO DEL DOCENTE · Módulo 5 · Crear una imagen propia con Dockerfile

Construir y ejecutar

DÓNDE SE EJECUTA

En la carpeta `ejemplos/01-api-express` (donde está el Dockerfile).

Terminal

```
docker build -t api-express:1.0 .
```

SALIDA ESPERADA (RESUMIDA)

```
[+] Building 8.1s (11/11) FINISHED
 => [1/5] FROM docker.io/library/node:24-
alpine@sha256:44fce6...
 => [2/5] WORKDIR /app
 => [3/5] COPY package.json package-lock.json
./
 => [4/5] RUN npm ci --omit=dev
 => [5/5] COPY . .
 => exporting to image
 => => naming to docker.io/library/api-
express:1.0
```

`-t api-express:1.0` le pone nombre y etiqueta (*tag*) a la imagen. Es normal ver avisos `npm notice` durante el build.

Terminal

```
docker run -d --name api -p 3000:3000 api-
express:1.0
docker logs api
```

DOCKER LOGS API

```
API escuchando en el puerto 3000
```

Abrí `http://localhost:3000/api/saludo`:

RESPUESTA ESPERADA

```
{"mensaje":"Hola desde un
contenedor","servidor":"58a3fdabd629","hora":"
2026-10-19T18:45:45.526Z"}
```

`servidor` es el ID corto del contenedor: compará con `docker ps`.

DEMO DEL DOCENTE · Módulo 5 · Crear una imagen propia con Dockerfile

Misma imagen, otra configuración

Terminal

```
docker run -d --name api2 -p 3001:3000 -e
MENSAJE="Hola desde otra configuración" api-
express:1.0
```

Abrí `http://localhost:3001/api/saludo`:

RESPUESTA ESPERADA

```
{"mensaje":"Hola desde otra
configuración","servidor":"073efbc38e54",...}
```

IDEA CLAVE

`-e NOMBRE=valor` reemplaza el valor por defecto que pusimos con `ENV` en el Dockerfile. La imagen no cambia: cambia la configuración de **ese** contenedor.

```
Terminal
docker ps
```

Ahora hay **dos contenedores** de la misma imagen, en puertos distintos, con mensajes distintos y nombres de servidor distintos.

PREGUNTA PARA COMPROBAR

¿Qué valor de `PORT` usa la app dentro de `api2`? ¿Por qué publicamos `3001:3000` y no `3001:3001`?

Usa `PORT=3000` (el valor por defecto del Dockerfile): adentro del contenedor la app sigue escuchando en el 3000. Lo que cambia es el puerto de tu compu (3001), que es el de la izquierda.

DEMO DEL DOCENTE · Módulo 5 · Crear una imagen propia con Dockerfile

Cambié el código… ¿y no se ve el cambio?

1. Cambiá el mensaje por defecto en `src/index.js` (por ejemplo, “Hola desde la versión 1.1”).
2. Recargá el navegador: **no cambió nada**. El contenedor usa la imagen 1.0, que tiene el código viejo.
3. Reconstruí con otra etiqueta y recreá el contenedor:

```
Terminal
docker build -t api-express:1.1 .
docker rm -f api
docker run -d --name api -p 3000:3000 api-express:1.1
```

IDEA CLAVE

Las imágenes **no se actualizan solas**. Cambio de código → nueva imagen (`build`) → nuevo contenedor (`rm` + `run`).

PARA SABER

¿Por qué una etiqueta nueva (`1.1`) en lugar de pisar la `1.0`? Porque si algo sale mal, volvés a la versión anterior en segundos. En producción se trabaja así (módulo 10).

EN PROGRAMACIÓN

Fijate en la salida del build: los pasos 2 a 4 dicen `CACHED`. Solo se rehizo la copia del código.

EXPLICACIÓN · Módulo 5 · Crear una imagen propia con Dockerfile

Buenas prácticas (a tu medida)

- ✓ **Imágenes oficiales** como base (`node`, `mysql`, `nginx`).
- ✓ **Versión explícita** en `FROM`; nunca `latest`.
- ✓ **Dependencias reproducibles:** `package-lock.json` + `npm ci`.
- ✓ **Primero dependencias, después código**, para aprovechar la caché.
- ✓ `.dockerignore` con `node_modules`, `.env` y `.git`.
- ✓ **Usuario sin privilegios:** `USER node`.
- ✓ **Configuración por variables**, con valores por defecto razonables.

ATENCIÓN

Nunca pongas contraseñas en el Dockerfile (ni en `ENV` ni en `ARG`): quedan guardadas en la imagen y cualquiera que la tenga las puede leer con `docker history` o `docker inspect`.

PARA SABER

Hay técnicas más avanzadas (multi-stage para backends, imágenes *distroless*, escaneo de vulnerabilidades). Para empezar no hacen falta; las mencionamos en el cierre.

✓ **Etiquetas con versión** para tus imágenes (`1.0`, `1.1`).

EXPLICACIÓN Módulo 5 · Crear una imagen propia con Dockerfile

Errores habituales al dockerizar una API

Síntoma	Causa	Solución
El contenedor arranca, pero el navegador no conecta	Falta `-p`, o la app escucha en `127.0.0.1`	Publicá el puerto; `app.listen(PORT)` sin host
`Exited (1)` y en logs `Error: Cannot find module '/app/src/server.js'`	El `CMD` apunta a un archivo que no existe	Corregí la ruta en `CMD` (y revisá `.dockerignore`)
`Cannot find package 'express'`	No se instalaron dependencias en la imagen	Agregá `RUN npm ci` antes de `CMD`
`"/package-lock.json": not found`	No hay lockfile o está ignorado	`npm install` local, versionar el lockfile
Los cambios de código no aparecen	No reconstruiste ni recreaste	`docker build` + recrear el contenedor
`EACCES: permission denied` al escribir un archivo	Con `USER node`, la app no puede escribir en carpetas de root	Escribí en un volumen o creá la carpeta con permisos (módulo 11)

ACTIVIDAD Módulo 5 · Crear una imagen propia con Dockerfile

Actividad: tu versión 1.1

En `ejemplos/01-api-express`. Tiempo: 15 minutos.

1. Construí la imagen `api-express:1.0` y ejecutala en el puerto 3000 con el nombre `api`. Probá `/api/saludo`.
2. Agregá en `src/index.js` una ruta `GET /api/version` que responda `{ "version": "1.1" }`.
3. Construí `api-express:1.1` y fijate qué pasos salen de la caché.
4. Ejecutá la versión 1.1 en el puerto **3002** con el nombre `api-v11`, **sin** borrar la 1.0, y con el mensaje “Versión nueva”.
5. Comprobá que las dos versiones responden a la vez y que `/api/version` solo existe en la 1.1.
6. Detené ambos contenedores y revisá sus logs: ¿aparece el mensaje de SIGTERM?

SOLUCIÓN Módulo 5 · Crear una imagen propia con Dockerfile

Solución: tu versión 1.1

src/index.js (ruta nueva, antes de app.listen)

```
app.get('/api/version', (req, res) => {
  res.json({ version: '1.1' });
});
```

Terminal

```
docker build -t api-express:1.0 .
docker run -d --name api -p 3000:3000 api-express:1.0

# después de agregar la ruta:
docker build -t api-express:1.1 .
docker run -d --name api-v11 -p 3002:3000 -e MENSAJE="Versión nueva" api-express:1.1

docker stop api api-v11
docker logs api-v11
```

RESULTADO ESPERADO

- En el build 1.1, `WORKDIR`, `COPY package*.json` y `npm ci` salen `CACHED`.
- `localhost:3000/api/version` → 404 (la 1.0 no tiene la ruta).
- `localhost:3002/api/version` → `{"version":"1.1"}`.
- Los logs terminan con `SIGTERM recibido: cerrando el servidor...`

ERROR HABITUAL

"La ruta nueva da 404 en el 3002": construiste antes de guardar el archivo, o ejecutaste la imagen 1.0. Revisá con `docker ps` la columna IMAGE.

REPASO Módulo 5 · Crear una imagen propia con Dockerfile

Recapitulación del módulo 5

IDEA CLAVE

- `FROM` elige la base; `WORKDIR`, `COPY` y `RUN` preparan la imagen; `CMD` dice qué ejecutar al iniciar.
- `RUN` = al construir. `CMD` = al iniciar cada contenedor.
- `EXPOSE` documenta; `-p` publica.
- `.dockerignore` define el contexto; el orden de las instrucciones define cuánto se aprovecha la caché.
- Cambio de código → `build` → recrear el contenedor.

PREGUNTA PARA COMPROBAR

¿Para qué sirve `.dockerignore` si igual usamos `COPY . .`?

Justamente por eso: `COPY . .` copia todo el contexto, y `.dockerignore` evita que entren `node_modules`, `.env` (secretos) o `.git`.

PREGUNTA PARA COMPROBAR

Construiste la imagen, la ejecutaste y en `docker ps` no aparece. ¿Qué hacés?

`docker ps -a` para ver si terminó (`Exited`) y `docker logs nombre` para leer el error.

MÓDULO 6

Docker Compose

API Express + MySQL, con volumen, red, variables de entorno y healthcheck, levantados con un solo comando.

Objetivo

Leer y escribir un `compose.yaml` y manejar el ciclo de vida de una aplicación de varios servicios.

Material

Carpeta `ejemplos/02-api-mysql`.

Tiempo estimado

50 minutos · demo + actividad

EXPLICACIÓN Módulo 6 · Docker Compose

¿Por qué Compose?

Sin Compose, levantar API + MySQL implica crear la red, el volumen, construir la imagen, esperar a MySQL y ejecutar dos `docker run` larguísimos… cada vez y en el orden correcto:

Sin Compose (solo para mirar; no hace falta ejecutarlo)

```
docker network create red-app
docker volume create datos-mysql
docker run -d --name db --network red-app \
  -e MYSQL_ROOT_PASSWORD=cambiar-root-local -e
MYSQL_DATABASE=tareas_db \
  -e MYSQL_USER=tareas_app -e
MYSQL_PASSWORD=cambiar-app-local \
  -v datos-mysql:/var/lib/mysql -v
"$PWD/db/init":/docker-entrypoint-initdb.d:ro \
  -p 127.0.0.1:3307:3306 mysql:8.4
# ...esperar a que MySQL termine de
inicializarse...
docker build -t tareas-backend ./backend
docker run -d --name backend --network red-app \
  -e DB_HOST=db -e DB_USER=tareas_app -e
DB_PASSWORD=cambiar-app-local -e
DB_NAME=tareas_db \
  -p 3000:3000 tareas-backend
```

¿QUÉ PROBLEMA RESUELVE?

Comandos largos, fáciles de equivocar, difíciles de compartir y que no quedan versionados en Git.

RESULTADO ESPERADO

Con Compose, todo eso queda **escrito** en `compose.yaml` y se levanta con:

Terminal

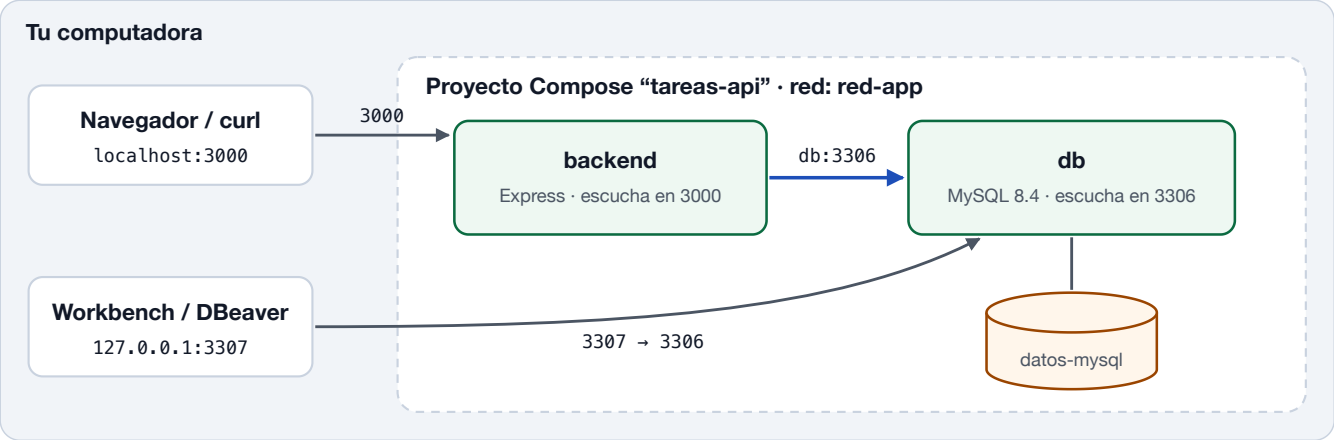
```
docker compose up -d
```

PARA SABER

Compose usa el mismo Docker Engine: crea las mismas redes, volúmenes y contenedores que harías a mano.

EXPLICACIÓN Módulo 6 · Docker Compose

Lo que vamos a construir




```
ejemplos/02-api-mysql/
├── compose.yaml          # los servicios, el volumen y la red
├── .env.example          # plantilla de configuración (se copia como .env)
├── backend/              # API Express: Dockerfile, .dockerignore, package*.json, src/
└── db/init/01-esquema.sql # tabla y datos iniciales
```

EXPLICACIÓN Módulo 6 · Docker Compose

El `compose.yaml` completo

ejemplos/02-api-mysql/compose.yaml

```yaml
# Ejemplo 02: API Express + MySQL con Docker Compose
# Uso:  1) copiar .env.example como .env   2) docker compose up -d --build

name: tareas-api

services:
  db:
    image: mysql:8.4
    environment:
      # ":?" hace que Compose se detenga con un mensaje claro si la variable falta en .env
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Falta MYSQL_ROOT_PASSWORD en .env}
      MYSQL_DATABASE: ${MYSQL_DATABASE:?Falta MYSQL_DATABASE en .env}
      MYSQL_USER: ${MYSQL_USER:?Falta MYSQL_USER en .env}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Falta MYSQL_PASSWORD en .env}
    volumes:
      - datos-mysql:/var/lib/mysql              # volumen con nombre: los datos sobreviven
      - ./db/init:/docker-entrypoint-initdb.d:ro # scripts SQL de la primera inicialización
    ports:
      # Solo desde tu computadora (127.0.0.1) y en 3307 para no chocar con un MySQL local
      - "127.0.0.1:3307:3306"
    networks:
      - red-app
    healthcheck:
      # "Listo" = el servidor definitivo acepta conexiones TCP (no valida usuario ni clave)
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "--silent"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 60s

  backend:
    build: ./backend
    environment:
      PORT: 3000
      DB_HOST: db            # nombre del servicio, NO localhost
      DB_PORT: 3306          # puerto interno de MySQL, NO el publicado
      DB_USER: ${MYSQL_USER}
      DB_PASSWORD: ${MYSQL_PASSWORD}
      DB_NAME: ${MYSQL_DATABASE}
    ports:
      - "3000:3000"
    networks:
      - red-app
```

DÓNDE SE EJECUTA

Va en la raíz del proyecto. Los comandos `docker compose …` se ejecutan en esa carpeta.

PARA SABER

YAML es un formato de texto donde la **indentación con espacios** define la estructura (como las llaves en JSON). Nunca uses tabulaciones.

PARA SABER

No hace falta la línea `version:` que aparece en tutoriales viejos: hoy es obsoleta y Compose la ignora con un aviso.

```yaml
    depends_on:
      db:
        condition: service_healthy  # espera a
que el healthcheck de db pase

volumes:
  datos-mysql:

networks:
  red-app:
```

EXPLICACIÓN Módulo 6 · Docker Compose

`name`, `services`, `image` y `build`

```yaml
name: tareas-api

services:
  db:
    image: mysql:8.4
...
  backend:
    build: ./backend
```

`name`
: Nombre del proyecto (opcional). Es el prefijo de contenedores, red y volúmenes: `tareas-api-db-1`, `tareas-api_datos-mysql`. Si no lo ponés, se usa el nombre de la carpeta.

`services`
: Cada servicio es una pieza de la app. Su nombre (`db`, `backend`) es también su **nombre en la red**.

`image`
: Usa una imagen existente (se descarga si falta).

`build`
: Construye una imagen con el Dockerfile de esa carpeta (es el contexto de construcción).

> **IDEA CLAVE**
> `image` = "usá esta imagen". `build` = "construí la imagen con mi Dockerfile". Para MySQL alcanza con la oficial; para tu API hace falta construirla.

EXPLICACIÓN Módulo 6 · Docker Compose

`ports` y `environment`

```yaml
    environment:
      # ":?" hace que Compose se detenga con
un mensaje claro si la variable falta en .env
      MYSQL_ROOT_PASSWORD:
${MYSQL_ROOT_PASSWORD:?Falta
MYSQL_ROOT_PASSWORD en .env}
      MYSQL_DATABASE: ${MYSQL_DATABASE:?Falta
MYSQL_DATABASE en .env}
...
    ports:
      # Solo desde tu computadora (127.0.0.1)
y en 3307 para no chocar con un MySQL local
      - "127.0.0.1:3307:3306"
```

`environment`
: Variables de entorno que recibe el contenedor (el equivalente a `-e`).

`${VARIABLE}`
: **Interpolación:** Compose reemplaza el valor leyendo el archivo `.env` de la carpeta del proyecto. Con `:?mensaje`, si la variable falta o está vacía, se detiene y muestra ese mensaje.

`ports`
: Publica puertos: `"3000:3000"`. Con `127.0.0.1:` adelante, solo se puede entrar desde tu propia compu.

ejemplos/02-api-mysql/.env.example

```
# Plantilla de configuración para DESARROLLO.
# 1) Copiala como .env      2) Cambiá las
contraseñas por valores propios.
# El archivo .env NO se sube a GitHub (está
en .gitignore).

MYSQL_ROOT_PASSWORD=cambiar-root-local
MYSQL_DATABASE=tareas_db
MYSQL_USER=tareas_app
MYSQL_PASSWORD=cambiar-app-local
```

PARA SABER

¿Y `env_file`? Es otra opción: `env_file: .env` pasa *todas* las variables del archivo al contenedor. Preferimos `environment` explícito para que cada servicio reciba solo lo que necesita (el backend no recibe la clave de root). Lo comparamos en el módulo 9.

ATENCIÓN

Poné los puertos entre comillas. Sin ellas, YAML puede interpretar algunos valores con `:` como números en base 60.

EXPLICACIÓN Módulo 6 · Docker Compose

`volumes` y `networks`

yaml

```
    volumes:
      - datos-mysql:/var/lib/mysql
# volumen con nombre: los datos sobreviven
      - ./db/init:/docker-entrypoint-
initdb.d:ro # scripts SQL de la primera
inicialización
...
    networks:
      - red-app
...
volumes:
  datos-mysql:

networks:
  red-app:
```

`volumes` (en el servicio)
: `nombre:/ruta` monta un **volumen con nombre**. `./carpeta:/ruta` monta una **carpeta de tu compu** (bind mount). `:ro` = solo lectura.

`volumes` (al final, nivel superior)
: Declara los volúmenes con nombre del proyecto. Compose los crea si no existen.

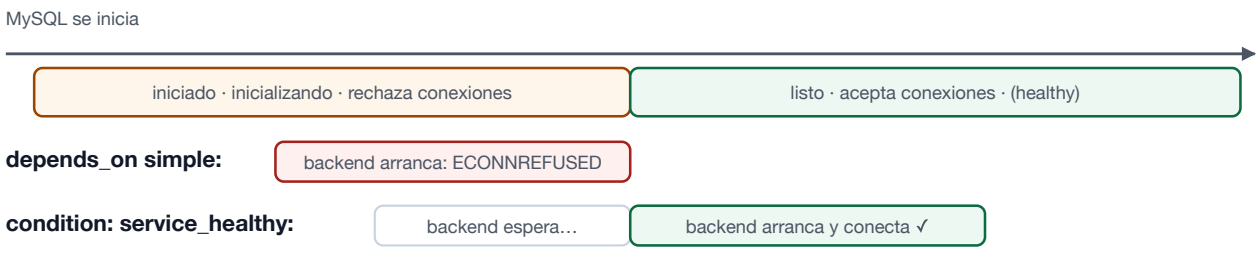
`networks`
: Conecta servicios a una red. Los de la misma red se encuentran por nombre: el backend llega a MySQL con el host `db`.

PARA SABER

Si no declarás redes, Compose crea una llamada `default` y conecta todos los servicios. La declaramos para que se vea y para poder separar redes más adelante.

EXPLICACIÓN Módulo 6 · Docker Compose

`depends_on` y `healthcheck`: iniciado no es lo mismo que listo



yaml

```yaml
    healthcheck:
      # "Listo" = el servidor definitivo
acepta conexiones TCP (no valida usuario ni
clave)
      test: ["CMD", "mysqladmin", "ping", "-
h", "127.0.0.1", "--silent"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 60s
...
    depends_on:
      db:
        condition: service_healthy  # espera
a que el healthcheck de db pase
```

- `depends_on: [db]` solo ordena el **arranque**: inicia `db` antes que `backend`, pero no espera a que MySQL acepte conexiones.
- `healthcheck` le dice a Docker cómo comprobar si el servicio está **sano**: ejecuta `test` cada `interval`; durante `start_period` los fallos no cuentan.
- `condition: service_healthy` hace que el backend espere al estado `healthy`.

ATENCIÓN

`mysqladmin ping` responde bien aunque el usuario o la clave sean incorrectos: comprueba que el servidor acepte conexiones, no tus credenciales. Un error de clave aparece en los logs del backend.

EXPLICACIÓN Módulo 6 · Docker Compose

El backend lee todo de variables de entorno

ejemplos/02-api-mysql/backend/src/config.js

```
// Toda la configuración llega por variables de
entorno.
// Si falta alguna obligatoria, la app termina
con un mensaje claro (mejor que fallar después).
const obligatorias = ['DB_HOST', 'DB_USER',
'DB_PASSWORD', 'DB_NAME'];
const faltantes = obligatorias.filter((nombre) =>
!process.env[nombre]);

if (faltantes.length > 0) {
  console.error(`Faltan variables de entorno:
${faltantes.join(', ')}`);
  process.exit(1);
}

export const config = {
  port: Number(process.env.PORT ?? 3000),
  db: {
    host: process.env.DB_HOST,
    port: Number(process.env.DB_PORT ?? 3306),
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
  },
};
```

ejemplos/02-api-mysql/backend/src/db.js

```
import mysql from 'mysql2/promise';
import { config } from './config.js';

// Pool de conexiones: se conecta a MySQL cuando
hace falta y reutiliza conexiones.
export const pool = mysql.createPool({
  ...config.db,
  waitForConnections: true,
  connectionLimit: 10,
});
```

yaml

```yaml
    environment:
      PORT: 3000
      DB_HOST: db            # nombre del
servicio, NO localhost
      DB_PORT: 3306          # puerto interno
de MySQL, NO el publicado
```

IDEA CLAVE

El código no sabe nada de Docker: lee `process.env`, como en cualquier proyecto bien organizado. Compose es quien le pasa los valores.

EXPLICACIÓN Módulo 6 · Docker Compose

Los comandos de Compose

Comando	Qué hace	Crea	Inicia	Reconstruye	Elimina
`docker compose up -d`	Crea lo que falta e inicia todo en segundo plano	✓	✓	Solo si la imagen no existe	—
`docker compose up -d --build`	Igual, pero reconstruye las imágenes con `build:` y recrea lo que cambió	✓	✓	✓	—
`docker compose ps`	Lista los servicios del proyecto y su estado	—	—	—	—
`docker compose logs -f [servicio]`	Muestra registros en vivo (`Ctrl`+`C` deja de mirar)	—	—	—	—
`docker compose stop` / `start`	Detiene / inicia sin borrar nada	—	✓ (start)	—	—
`docker compose down`	Detiene y elimina contenedores y red	—	—	—	Contenedores y red. **Los volúmenes quedan.**
`docker compose down -v`	Lo mismo y además elimina los volúmenes del proyecto	—	—	—	**+ volúmenes ⚠ datos**

PARA SABER

También existen `docker compose exec servicio comando` (como `docker exec`), `docker compose restart` y `docker compose config` (muestra la configuración final, con las variables ya reemplazadas).

EXPLICACIÓN Módulo 6 · Docker Compose

⚠ `docker compose down -v` borra los datos

ATENCIÓN

`-v` elimina los **volúmenes** administrados por el proyecto: en nuestro caso, `tareas-api_datos-mysql`, que contiene **toda la base de datos**.

No se puede deshacer. No hay papelera.

Querés…	Usá
Apagar todo al terminar el día	`docker compose stop` o `down`
Aplicar cambios de configuración	`docker compose up -d`
Aplicar cambios de código	`docker compose up -d --build`
Empezar con la base vacía (desarrollo)	`docker compose down -v`, **sabiendo** que se pierde todo

IDEA CLAVE

Antes de un `down -v` con datos que importan: copia de seguridad (módulo 7).

DEMO DEL DOCENTE Módulo 6 · Docker Compose

Demo: levantar, probar y bajar

DÓNDE SE EJECUTA

En la carpeta `ejemplos/02-api-mysql`.

Terminal

```
cp .env.example .env
docker compose up -d --build
docker compose ps
```

DOCKER COMPOSE UP -D --BUILD (FINAL)

```
✔ Network tareas-api_red-app        Created
✔ Volume tareas-api_datos-mysql     Created
✔ Container tareas-api-db-1         Healthy
✔ Container tareas-api-backend-1    Started
```

DOCKER COMPOSE PS (COLUMNAS PRINCIPALES)

```
NAME                   SERVICE   STATUS                   PORTS
tareas-api-backend-1   backend   Up 3 seconds             0.0.0.0:3000->3000/tcp
tareas-api-db-1        db        Up 9 seconds (healthy)   127.0.0.1:3307->3306/tcp
```

Probá en el navegador `http://localhost:3000/api/tareas` y creá una tarea:

macOS / Linux / Git Bash

Terminal

```
curl -X POST -H "Content-Type: application/json" -d '{"titulo":"Aprender volúmenes"}'
http://localhost:3000/api/tareas
```

Windows PowerShell

PowerShell

```
Invoke-RestMethod -Method Post -Uri http://localhost:3000/api/tareas -ContentType
'application/json' -Body '{"titulo":"Aprender volúmenes"}'
```

RESPUESTA

```
{"id":4,"titulo":"Aprender volúmenes","completada":false}
```

Terminal

```
docker compose logs backend
docker compose down
```

También podés usar Postman, Thunder Client o la extensión REST Client de VS Code.

EXPLICACIÓN Módulo 6 · Docker Compose

Errores habituales con Compose

Mensaje	Causa	Solución
`required variable MYSQL_ROOT_PASSWORD is missing a value: Falta MYSQL_ROOT_PASSWORD en .env`	No existe `.env` o le falta esa variable	`cp .env.example .env` y completar
`yaml: line 3: found character that cannot start any token`	Una tabulación en el YAML	Reemplazar tabulaciones por espacios
`yaml: … did not find expected key`	Indentación inconsistente	Alinear las claves del mismo nivel
`the attribute `version` is obsolete, it will be ignored`	Línea `version:` de tutoriales viejos	Es solo un aviso: borrá la línea
`Bind for 0.0.0.0:3000 failed: port is already allocated` / `address already in use`	Otro proceso o contenedor usa ese puerto	Liberarlo o cambiar el número de la izquierda
`no such service: backned`	Nombre de servicio mal escrito	Usá los nombres de `services:`
`no configuration file provided: not found`	Estás en otra carpeta	`cd` a la carpeta del `compose.yaml`

ACTIVIDAD Módulo 6 · Docker Compose

Actividad: Compose en tus manos

En `ejemplos/02-api-mysql`. Tiempo: 15 minutos.

1. Levantá el proyecto y esperá a que `db` figure `(healthy)`.
2. Creá dos tareas nuevas y verificá que aparecen en `/api/tareas`.
3. Ejecutá `docker compose down` y luego `docker compose up -d`. ¿Siguen tus tareas?
4. Cambiá el puerto publicado del backend para que la API quede en `http://localhost:3001` (la app sigue escuchando en el 3000 adentro). Aplicá el cambio y probalo.
5. Mirá los logs del backend en vivo mientras recargás la página. ¿Qué imprime?
6. Pregunta: ¿qué comando usarías si cambiaras el código de `backend/src/rutas/tareas.js`?

SOLUCIÓN Módulo 6 · Docker Compose

Solución: Compose en tus manos

compose.yaml (servicio backend)

```
    ports:
      - "3001:3000"
```

Terminal

```
docker compose up -d --build
docker compose ps
docker compose down
docker compose up -d
docker compose up -d          # después de cambiar el puerto: recrea solo el backend
docker compose logs -f backend
```

RESULTADO ESPERADO

- Después de `down` + `up -d`, **las tareas siguen**: el volumen no se borró.
- Al cambiar `ports` y hacer `up -d`, Compose detecta el cambio y recrea solo el contenedor del backend.
- Los logs muestran `API escuchando en el puerto 3000` (el puerto interno no cambió).
- Con cambios de código: `docker compose up -d --build`.

ERROR HABITUAL

Si cambiaste a `"3001:3001"`, la API no responde: adentro sigue escuchando en el 3000.

REPASO Módulo 6 · Docker Compose

Recapitulación del módulo 6

IDEA CLAVE

- `compose.yaml` describe servicios, puertos, variables, volúmenes y redes: el entorno queda versionado.
- Los servicios se encuentran **por nombre** dentro de la red.
- `${VAR:?mensaje}` toma valores de `.env` y frena si faltan.
- `depends_on` ordena; `healthcheck` + `service_healthy` espera a que esté listo.
- `down` conserva los volúmenes; `down -v` los borra.

PREGUNTA PARA COMPROBAR

¿Qué diferencia hay entre `docker compose up -d` y `docker compose up -d --build`?

Sin `--build`, si la imagen del backend ya existe, se reutiliza aunque hayas cambiado el código. Con `--build`, se reconstruye y se recrea el contenedor si la imagen cambió.

PREGUNTA PARA COMPROBAR

¿Por qué el backend usa `DB_PORT: 3306` si publicamos MySQL en el 3307?

Porque el backend habla con MySQL **por la red interna**, donde MySQL escucha en su puerto real, 3306. El 3307 es solo la puerta de entrada desde tu compu.

MÓDULO 7

MySQL: datos, conexión y persistencia

Ya sabés MySQL. Ahora: correrlo en un contenedor, conectarte desde distintos lugares y no perder nunca los datos.

Objetivo

Configurar MySQL en Docker, entender a qué host y puerto conectarse desde cada lugar y garantizar la persistencia.

Vamos a ver

- MySQL con `docker run`
- Variables y scripts de inicialización
- `localhost` vs nombre de servicio
- Ciclo de vida de los datos
- Copias de seguridad y diagnóstico

Tiempo estimado

35 minutos

DEMO DEL DOCENTE · Módulo 7 · MySQL: datos, conexión y persistencia

MySQL en un contenedor, sin Compose

macOS / Linux / Git Bash

Terminal

```
docker run -d --name mysql-prueba \
  -e MYSQL_ROOT_PASSWORD=clave-root-prueba \
  -e MYSQL_DATABASE=escuela \
  -v mysql-prueba-datos:/var/lib/mysql \
  -p 127.0.0.1:3308:3306 \
  mysql:8.4
```

Windows PowerShell

PowerShell

```
docker run -d --name mysql-prueba `
  -e MYSQL_ROOT_PASSWORD=clave-root-prueba `
  -e MYSQL_DATABASE=escuela `
  -v mysql-prueba-datos:/var/lib/mysql `
  -p 127.0.0.1:3308:3306 `
  mysql:8.4
```

Variable de la imagen	Para qué
`MYSQL_ROOT_PASSWORD`	Clave de `root`. Obligatoria (o una de sus alternativas).
`MYSQL_DATABASE`	Crea esa base al inicializar.
`MYSQL_USER` + `MYSQL_PASSWORD`	Crea un usuario con todos los permisos **solo** sobre `MYSQL_DATABASE`.

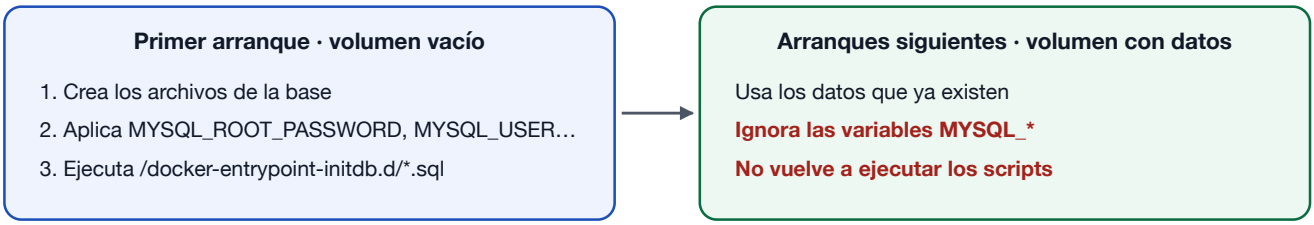
El `\` (bash) o el `` ` `` (PowerShell) al final de línea solo sirven para partir un comando largo. Usamos el puerto 3308 para no chocar con el ejemplo de Compose (3307) ni con un MySQL instalado (3306).

ERROR HABITUAL

Sin `MYSQL_ROOT_PASSWORD`, el contenedor termina enseguida. En `docker logs`: `[ERROR] [Entrypoint]: Database is uninitialized and password option is not specified`.

EXPLICACIÓN Módulo 7 · MySQL: datos, conexión y persistencia

Las variables y los scripts se aplican solo la primera vez



ERROR HABITUAL

"Cambié `MYSQL_PASSWORD` en el `.env` y ahora el backend da `Access denied`". MySQL conserva la clave de la primera inicialización. En desarrollo: volvé a la clave anterior, o cambiala con `ALTER USER`, o empezá de cero con `docker compose down -v` (se pierden los datos).

ERROR HABITUAL

"Agregué una tabla al script de `db/init` y no aparece". El script ya se ejecutó la primera vez. Aplicá el cambio a mano (consola de MySQL) o recreá el volumen en desarrollo.

EXPLICACIÓN Módulo 7 · MySQL: datos, conexión y persistencia

Scripts de inicialización

ejemplos/02-api-mysql/db/init/01-esquema.sql

```sql
-- Se ejecuta SOLO la primera vez que MySQL
arranca con el volumen de datos vacío.
-- La imagen lo aplica sobre la base indicada
en MYSQL_DATABASE.
SET NAMES utf8mb4;

CREATE TABLE IF NOT EXISTS tareas (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  titulo     VARCHAR(200) NOT NULL,
  completada BOOLEAN      NOT NULL DEFAULT
FALSE,
  creada_en  TIMESTAMP    NOT NULL DEFAULT
CURRENT_TIMESTAMP
);

INSERT INTO tareas (titulo, completada)
VALUES
  ('Instalar Docker', TRUE),
  ('Entender la diferencia entre imagen y
contenedor', FALSE),
  ('Levantar MySQL con Compose', FALSE);
```

yaml

```yaml
      - ./db/init:/docker-entrypoint-
initdb.d:ro # scripts SQL de la primera
inicialización
```

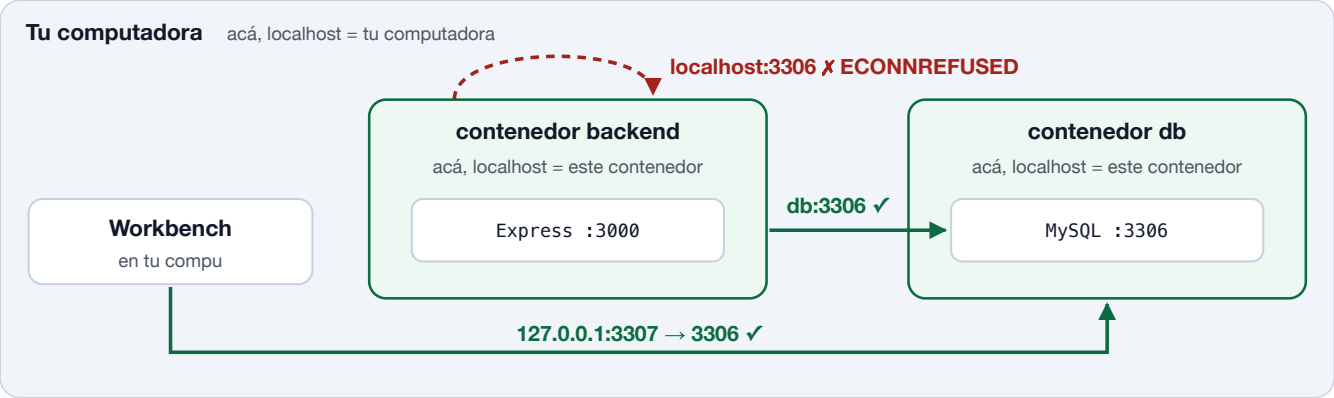
- Todo lo que esté en `/docker-entrypoint-initdb.d` se ejecuta en la primera inicialización: archivos `.sql`, `.sql.gz` y `.sh`.
- En **orden alfabético**: por eso los numeramos (`01-esquema.sql`, `02-datos.sql`…).
- Los `.sql` se aplican sobre la base de `MYSQL_DATABASE`.
- Si un script falla, la inicialización se detiene: mirá `docker compose logs db`.

EN PROGRAMACIÓN

Tu proyecto ya tiene un script para crear tablas (el que usabas en Workbench). Ese es tu `01-esquema.sql`. Si empieza con `CREATE DATABASE …; USE …;`, usá ese mismo nombre en `MYSQL_DATABASE`.

EXPLICACIÓN Módulo 7 · MySQL: datos, conexión y persistencia

¿Quién es `localhost`?



IDEA CLAVE

`localhost` significa "esta misma máquina", y cada contenedor es su propia máquina. **Dentro de Compose, el backend se conecta al nombre del servicio (`db`), nunca a `localhost`.**

EXPLICACIÓN Módulo 7 · MySQL: datos, conexión y persistencia

Host y puerto según desde dónde te conectás

Quién se conecta a MySQL	Desde dónde	Host	Puerto
Backend en Compose	Contenedor en la red `red-app`	`db`	`3306` (interno)
Workbench, DBeaver, extensión de VS Code	Tu compu	`127.0.0.1`	`3307` (publicado)
Tu backend con `npm start` (sin Docker), usando la base del contenedor	Tu compu	`127.0.0.1`	`3307`
Consola `mysql` dentro del contenedor	Adentro de `db`	(no hace falta)	—

IDEA CLAVE

Puerto interno (3306): donde MySQL escucha dentro del contenedor; lo usan otros contenedores. **Puerto publicado** (3307): la entrada desde tu compu; solo existe si lo publicás con `ports`.

ATENCIÓN

Con el cliente `mysql` de línea de comandos en macOS/Linux, `-h localhost` usa el *socket* local e **ignora el puerto**: terminás en el MySQL instalado. Usá `-h 127.0.0.1 -P 3307`.

DEMO DEL DOCENTE Módulo 7 · MySQL: datos, conexión y persistencia

Conectarse para administrar la base

Desde una herramienta gráfica

Tipo de conexión	MySQL · TCP/IP
Host	`127.0.0.1`
Puerto	`3307`
Usuario / clave	`tareas_app` / la de tu `.env`
Base	`tareas_db`

ERROR HABITUAL

`Access denied for user 'tareas_app'@'localhost'` usando el puerto **3306**: te conectaste al MySQL instalado en tu compu, no al del contenedor. Revisá el puerto.

Desde la consola del contenedor

Terminal

```
docker compose exec db mysql -u tareas_app -p tareas_db
```

mysql>

```
SHOW TABLES;
SELECT id, titulo FROM tareas;
exit
```

SALIDA ESPERADA

```
+-----------------------+
| Tables_in_tareas_db   |
+-----------------------+
| tareas                |
+-----------------------+
```

`-p` sin valor hace que pida la clave sin mostrarla. No hace falta host: estás adentro del contenedor.

EXPLICACIÓN Módulo 7 · MySQL: datos, conexión y persistencia

¿Qué pasa con los datos cuando…?

Acción	¿Los datos siguen?	Por qué
`docker compose stop` y `start`	**Sí**	El contenedor no se borra
`docker compose down` y `up -d`	**Sí**	El contenedor se recrea, pero el volumen con nombre queda
`docker compose up -d --build`	**Sí**	Solo cambian contenedores; el volumen se vuelve a montar
Cambiar de `mysql:8.4.10` a `mysql:8.4.11`	**Sí**	Mismo volumen; MySQL actualiza los archivos si hace falta
`docker compose down -v`	**No**	Borra el volumen. Al levantar, se inicializa de nuevo
`docker volume rm tareas-api_datos-mysql`	**No**	Borrado explícito del volumen
Contenedor **sin** volumen con nombre + `docker rm` + nuevo `run`	**No (en el nuevo)**	El contenedor nuevo arranca con una base vacía

PARA SABER

La imagen de MySQL declara `/var/lib/mysql` como volumen. Si no indicás uno, Docker crea un **volumen anónimo** con nombre aleatorio. Al recrear el contenedor se crea otro anónimo: la base aparece vacía y los datos viejos quedan huérfanos ocupando espacio. Por eso siempre usamos un volumen **con nombre**.

ATENCIÓN

Bajar de versión mayor (por ejemplo, de 8.4 a 8.0) con el mismo volumen **no** es seguro. Antes de cambiar versiones: copia de seguridad.

DEMO DEL DOCENTE Módulo 7 · MySQL: datos, conexión y persistencia

Demo: los datos sobreviven al contenedor

Con el contenedor `mysql-prueba` de la diapositiva anterior (volumen `mysql-prueba-datos`).

1. Crear una tabla y cargar datos:

Terminal

```
docker exec -it mysql-prueba mysql -u root -p escuela
```

mysql>

```
CREATE TABLE alumnos (id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100));
INSERT INTO alumnos (nombre) VALUES ('Ada'), ('Alan');
SELECT * FROM alumnos;
exit
```

2. Eliminar el contenedor: `docker rm -f mysql-prueba`
3. Crear uno **nuevo** con el mismo `docker run` (mismo volumen).
4. Consultar de nuevo: `SELECT * FROM alumnos;`

RESULTADO EN EL CONTENEDOR NUEVO

```
+----+--------+
| id | nombre |
+----+--------+
|  1 | Ada    |
|  2 | Alan   |
+----+--------+
```

RESULTADO ESPERADO

El contenedor nuevo encontró el volumen con datos: **no** volvió a inicializar la base y la tabla sigue ahí.

Limpieza al terminar (borra los datos de prueba)

```
docker rm -f mysql-prueba
docker volume rm mysql-prueba-datos
```

DEMO DEL DOCENTE Módulo 7 · MySQL: datos, conexión y persistencia

Copia de seguridad y restauración

Un volumen no es un backup. Un archivo `.sql` guardado en otro lugar, sí.

DÓNDE SE EJECUTA

En la carpeta del proyecto (`ejemplos/02-api-mysql`). Estos comandos funcionan igual en bash, zsh y PowerShell.

Exportar

Terminal

```
docker compose exec db sh -c 'mysqldump -u root -p$MYSQL_ROOT_PASSWORD $MYSQL_DATABASE > /tmp/backup.sql'
docker compose cp db:/tmp/backup.sql ./backup.sql
```

Restaurar

Terminal

```
docker compose cp ./backup.sql db:/tmp/backup.sql
docker compose exec db sh -c 'mysql -u root -p$MYSQL_ROOT_PASSWORD $MYSQL_DATABASE < /tmp/backup.sql'
```

1. `mysqldump` se ejecuta **dentro** del contenedor y usa sus propias variables de entorno (por eso las comillas simples: `$MYSQL_…` lo resuelve el contenedor, no tu terminal).
2. `docker compose cp` copia el archivo a tu compu.

RESULTADO ESPERADO

Probado: después de `docker compose down -v` (base reiniciada con 3 tareas), la restauración recuperó las 4 tareas del backup.

ATENCIÓN

Un backup tiene datos reales: no lo subas a GitHub (el `.gitignore` del repositorio ignora `backup*.sql`). Y un backup que nunca restauraste es una suposición: probalo.

EXPLICACIÓN Módulo 7 · MySQL: datos, conexión y persistencia

Leer los registros de MySQL

Terminal

```
docker compose logs db
```

PRIMERA INICIALIZACIÓN (RESUMIDA)

```
[Note] [Entrypoint]: Initializing database
files
[Note] [Entrypoint]: Starting temporary server
... ready for connections. ... port: 0  MySQL
Community Server - GPL.
[Note] [Entrypoint]: Creating database
tareas_db
[Note] [Entrypoint]: Creating user tareas_app
[Note] [Entrypoint]: /usr/local/bin/docker-
entrypoint.sh: running /docker-entrypoint-
initdb.d/01-esquema.sql
[Note] [Entrypoint]: MySQL init process done.
Ready for start up.
... ready for connections. Version: '8.4.11'
socket: '/var/run/mysqld/mysqld.sock'  port:
3306
```

- Durante la primera inicialización arranca un servidor **temporal** con `port: 0` (no acepta conexiones de red). No está listo todavía.
- MySQL está listo cuando aparece `ready for connections` con `port: 3306`.
- Si no ves `running /docker-entrypoint-initdb.d/…`, tus scripts no se ejecutaron (el volumen ya tenía datos).

PARA SABER

Avisos como `[Warning] … CA certificate ca.pem is self signed` o `root@localhost is created with an empty password` (durante el servidor temporal) son normales en esta imagen.

EXPLICACIÓN Módulo 7 · MySQL: datos, conexión y persistencia

Diagnóstico de errores de conexión

Error (en los logs del backend)	Qué significa	Qué revisar
`connect ECONNREFUSED 127.0.0.1:3306`	Intenta conectarse a `localhost`, adentro de su propio contenedor	`DB_HOST` debe ser `db`
`getaddrinfo ENOTFOUND mysql`	No existe un servicio con ese nombre en su red	El nombre del servicio y que ambos estén en la misma red
`connect ECONNREFUSED 172.24.0.2:3306`	Encontró a `db`, pero MySQL todavía no acepta conexiones	`healthcheck` + `condition: service_healthy`
`Access denied for user 'tareas_app'@'172.24.0.3'`	Llegó a MySQL, pero usuario o clave no coinciden	`.env`; si cambiaste la clave después de inicializar (ver “solo la primera vez”)
`Access denied for user 'tareas_app'@'%' to database 'tareas'`	El usuario es correcto, pero esa base no existe o no tiene permisos sobre ella	`DB_NAME` tiene que ser igual a `MYSQL_DATABASE`
`Table 'tareas_db.tareas' doesn't exist`	Los scripts de inicialización no se ejecutaron	Ruta del bind mount; si el volumen ya existía

Comandos para investigar

```
docker compose ps                      # ¿db está (healthy)?
docker compose logs db                 # ¿inicializó? ¿ejecutó los scripts?
docker compose logs backend            # ¿qué error exacto da?
docker compose exec db mysql -u tareas_app -p tareas_db   # ¿las credenciales andan?
```

EXPLICACIÓN Módulo 7 · MySQL: datos, conexión y persistencia

Puertos y credenciales en desarrollo

yaml

```
    ports:
      # Solo desde tu computadora (127.0.0.1)
y en 3307 para no chocar con un MySQL local
      - "127.0.0.1:3307:3306"
```

- **¿Por qué publicar?** Solo para usar Workbench/DBeaver desde tu compu. El backend no lo necesita.
- **¿Por qué `127.0.0.1`?** Con `"3307:3306"` se publicaría en *todas* las interfaces: cualquiera en tu misma red Wi-Fi podría intentar conectarse.
- **¿Por qué 3307?** Muchos tienen MySQL instalado en el 3306.

IDEA CLAVE

La app usa `MYSQL_USER` (permisos solo sobre su base), **no** `root`.

ATENCIÓN

Las claves de `.env.example` son de ejemplo para desarrollo local. En tu `.env` poné otras, y en producción, claves largas y aleatorias (módulo 9). En producción, la base no se publica.

ACTIVIDAD Módulo 7 · MySQL: datos, conexión y persistencia

Actividad: persistencia y diagnóstico

En `ejemplos/02-api-mysql`, con el proyecto levantado. Tiempo: 15 minutos.

1. Conectate a la base desde Workbench, DBeaver o VS Code (host, puerto, usuario, clave y base correctos). Si no tenés ninguna herramienta, usá la consola del contenedor.
2. Agregá una columna `prioridad TINYINT DEFAULT 1` a `tareas` y cargá un valor.
3. Hacé un backup a `backup.sql`.
4. Ejecutá `docker compose down -v` y `docker compose up -d`. ¿Existe la columna? ¿Por qué?
5. Restaurá el backup y verificá la columna y el dato.
6. Cambiá `DB_HOST` del backend a `localhost`, aplicá con `up -d` y leé el error en los logs. Después volvé a `db`.

SOLUCIÓN Módulo 7 · MySQL: datos, conexión y persistencia

Solución: persistencia y diagnóstico

Paso 2

```
ALTER TABLE tareas ADD COLUMN prioridad
TINYINT DEFAULT 1;
UPDATE tareas SET prioridad = 3 WHERE id = 1;
```

Pasos 3 a 5

```
docker compose exec db sh -c 'mysqldump -u
root -p$MYSQL_ROOT_PASSWORD $MYSQL_DATABASE >
/tmp/backup.sql'
docker compose cp db:/tmp/backup.sql
./backup.sql
docker compose down -v
docker compose up -d
docker compose cp ./backup.sql
db:/tmp/backup.sql
docker compose exec db sh -c 'mysql -u root -
p$MYSQL_ROOT_PASSWORD $MYSQL_DATABASE <
/tmp/backup.sql'
```

RESULTADO ESPERADO

- Después de `down -v` + `up`, la columna **no existe**: se borró el volumen y se volvió a ejecutar `01-esquema.sql`, que no la tiene.
- Después de restaurar, la columna y el valor **vuelven**.
- Con `DB_HOST: localhost`: `/api/salud` responde `"db":"error","detalle":"ECONNREFUSED"` y en los logs aparece `connect ECONNREFUSED 127.0.0.1:3306`.

PARA SABER

Si la columna tiene que existir siempre, agregala también al script `01-esquema.sql` para futuras inicializaciones.

Recapitulación del módulo 7

IDEA CLAVE

- Variables `MYSQL_*` y scripts de `/docker-entrypoint-initdb.d`: **solo con el volumen vacío**.
- Backend → `db:3306`. Tu compu → `127.0.0.1:3307`.
- Dentro de un contenedor, `localhost` es ese contenedor.
- Volumen con nombre = persistencia. Backup = `mysqldump` guardado en otro lugar.
- `ready for connections … port: 3306` = MySQL listo.

PREGUNTA PARA COMPROBAR

Tu backend en Compose da `ECONNREFUSED 127.0.0.1:3306`. ¿Cuál es la causa más probable?

Usa `localhost` (o `127.0.0.1`) como host de la base. Dentro del contenedor eso apunta al propio backend. Tiene que usar el nombre del servicio, `db`.

PREGUNTA PARA COMPROBAR

¿Por qué el backend no necesita que MySQL publique un puerto?

Porque se comunica por la red interna de Compose. Publicar puertos solo sirve para entrar desde fuera de Docker (tu compu u otras máquinas).

MÓDULO 8

Dockerizar Angular + Express + MySQL

La arquitectura de tus proyectos, completa: frontend, backend y base de datos en servicios separados y una red compartida.

Objetivo

Servir Angular con Nginx, conectar con la API sin errores de CORS ni de `localhost`, y saber adaptar el ejemplo a tu proyecto.

Material

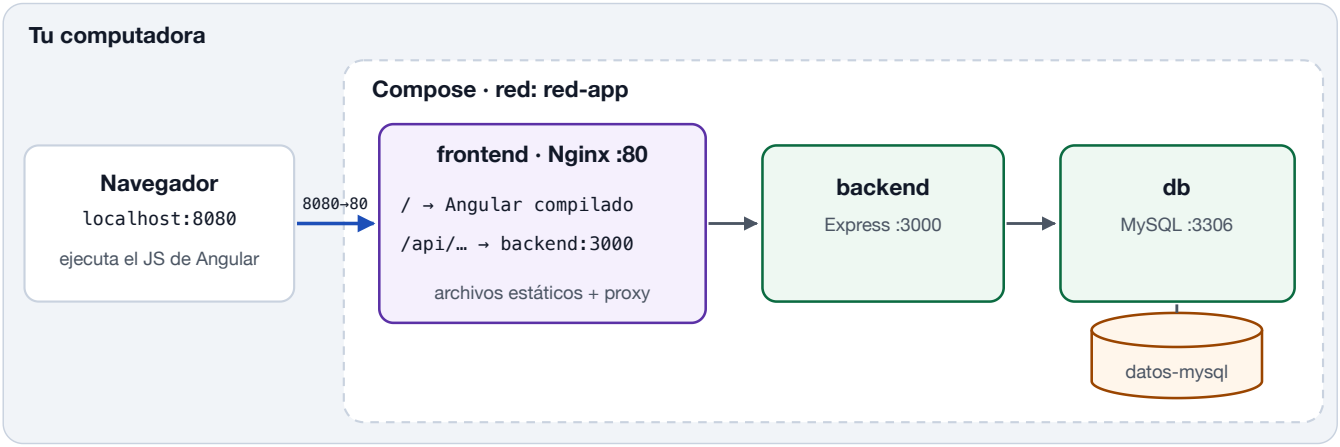
Carpeta `ejemplos/03-angular-express-mysql` (Angular 22, Express 5, MySQL 8.4).

Tiempo estimado

55 minutos

EXPLICACIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

La arquitectura



frontend

Angular **compilado** (HTML, JS, CSS) servido por Nginx, que además reenvía `/api` al backend.

backend

La misma API del ejemplo 02. No cambia nada.

db

MySQL con volumen y scripts de inicialización, igual que antes.

PARA SABER

Nginx es un servidor web muy usado para servir archivos estáticos y como **proxy inverso**: recibe peticiones y las reenvía a otro servidor (acá, al backend).

EXPLICACIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

Dos formas de ejecutar Angular (no las mezcles)

	A · "Como en producción"	B · Desarrollo con `ng serve`
Qué corre	El resultado de `ng build` servido por **Nginx** en un contenedor	El servidor de desarrollo de Angular, en tu compu
Recarga automática	No: hay que reconstruir la imagen	Sí, al guardar
URL	`http://localhost:8080`	`http://localhost:4200`
Quién reenvía `/api`	Nginx (`nginx.conf`)	El proxy de `ng serve` (`proxy.conf.json`)
Backend y base	En Docker	En Docker
Cuándo usarla	Probar la app completa, demos, entregas, deploy	Programar el frontend día a día

ATENCIÓN

`ng serve` es un servidor **de desarrollo**: no se usa en producción. Y la imagen de Nginx no ejecuta `ng serve`: solo sirve archivos ya compilados.

IDEA CLAVE

En ambos modos Angular pide la API a una **URL relativa** (`/api/tareas`). Cambia quién la reenvía, no el código.

Estructura del ejemplo

```
ejemplos/03-angular-express-mysql/
├── compose.yaml          # desarrollo local: construye todo
├── compose.prod.yaml     # producción (módulo 10)
├── .env.example
├── .env.prod.example
├── backend/              # = ejemplo 02
├── db/init/01-esquema.sql
└── frontend/             # proyecto Angular 22
    ├── Dockerfile        # 2 etapas: compilar y servir
    ├── .dockerignore
    ├── nginx.conf        # estáticos + proxy de /api
    ├── proxy.conf.json   # proxy de /api para ng serve
    ├── angular.json
    ├── package.json · package-lock.json
    └── src/
        ├── index.html · main.ts · styles.css
        └── app/
            ├── app.ts              # componente
            ├── app.config.ts       # provideHttpClient()
            └── tareas.service.ts   # llamadas a la API
```

EN PROGRAMACIÓN

Es la estructura que ya usás: un repositorio con carpetas `frontend` y `backend`. Docker suma tres archivos por servicio (`Dockerfile`, `.dockerignore` y, en el frontend, `nginx.conf`) y un `compose.yaml` en la raíz.

PARA SABER

El proyecto Angular se generó con `ng new frontend --minimal` y se le agregaron el servicio y el componente de tareas. Nada específico de Docker vive dentro de `src/`.

Dockerfile del frontend: dos etapas

ejemplos/03-angular-express-mysql/frontend/Dockerfile

```
# ---------- Etapa 1: compilar Angular (esta imagen se descarta al final) ----------
FROM node:24-alpine AS build
WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .
RUN npm run build

# ---------- Etapa 2: servir los archivos estáticos con Nginx ----------
FROM nginx:1.30-alpine

# Nuestra configuración: archivos de Angular + proxy de /api al backend
COPY nginx.conf
/etc/nginx/conf.d/default.conf

# Solo copiamos el resultado compilado (sin Node, sin node_modules, sin código fuente)
COPY --from=build /app/dist/frontend/browser
/usr/share/nginx/html

EXPOSE 80
# El comando de inicio (nginx en primer plano) ya viene definido en la imagen base
```

ANALOGÍA

Una **cocina** y un **mostrador**. En la cocina (etapa `build`, con Node y todas las herramientas) se prepara el plato. Al mostrador (Nginx) solo llega el plato terminado: no la cocina, ni las ollas.

- `FROM … AS build` le pone nombre a la primera etapa.
- `COPY --from=build` copia archivos **desde esa etapa**.
- La imagen final tiene Nginx + los archivos compilados: no tiene Node, ni `node_modules`, ni el código TypeScript.

ATENCIÓN

`dist/frontend/browser` depende de tu proyecto: `frontend` es el nombre del proyecto en `angular.json`. En Angular 17+ con el builder `application` la salida está en `browser/`; en proyectos más viejos suele ser `dist/<nombre>`. Mirá dónde deja los archivos `ng build`.

EXPLICACIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

`nginx.conf`: archivos estáticos y proxy de `/api`

ejemplos/03-angular-express-mysql/frontend/nginx.conf

```
server {
  listen 80;
  server_name _;

  root /usr/share/nginx/html;
  index index.html;

  # 1) Las peticiones a /api/ se reenvían al backend.
  #    "backend" es el nombre del servicio en compose.yaml (DNS interno de la red).
  location /api/ {
    proxy_pass http://backend:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
  }

  # 2) Todo lo demás es la aplicación Angular.
  #    Si la ruta no es un archivo real (por ejemplo /tareas/3), devolvemos index.html
  #    para que el router de Angular la resuelva en el navegador.
  location / {
    try_files $uri $uri/ /index.html;
  }
}
```

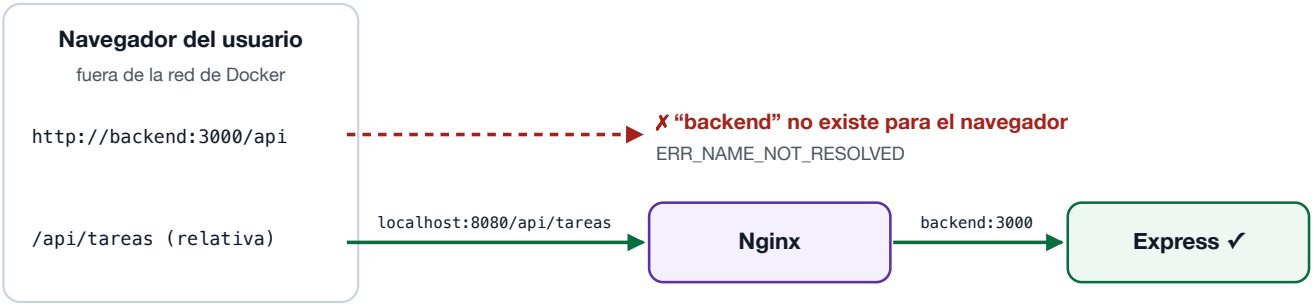
- `location /api/`: todo lo que empiece con `/api/` se reenvía a `http://backend:3000`, conservando la ruta (`/api/tareas`). `backend` es el nombre del servicio: Nginx corre en un contenedor de la red, así que lo resuelve.
- `location /`: sirve los archivos de Angular. `try_files … /index.html` hace que una ruta del router (por ejemplo `/tareas/3`) devuelva `index.html` en vez de un 404.

ERROR HABITUAL

Sin `try_files`, la app anda al entrar por `/`, pero si recargás en `/tareas/3` aparece **404 Not Found** de Nginx: no existe ese archivo en el disco.

EXPLICACIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

El código de Angular corre en el navegador



```ts
  // URL RELATIVA: el navegador la pide al mismo origen que le sirvió Angular.
  // Producción: Nginx reenvía /api al backend. Desarrollo: lo hace el proxy de `ng serve`.
  private readonly apiUrl = '/api/tareas';
```

IDEA CLAVE

Angular se *descarga* desde el contenedor, pero se *ejecuta* en el navegador. Para el navegador, `localhost` es la compu del usuario y los nombres de servicio de Docker no existen.

EXPLICACIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

URL de la API y CORS

CORS (*Cross-Origin Resource Sharing*) es una protección del navegador: una página cargada desde un **origen** (esquema + host + puerto, por ejemplo `http://localhost:8080`) no puede leer respuestas de otro origen (`http://localhost:3000`) salvo que ese servidor lo autorice.

Opción	URL en Angular	¿CORS?
Proxy (recomendada)	`/api/tareas`	**No hace falta: mismo origen**
URL absoluta	`http://localhost:3000/api/tareas`	**Sí: habilitar CORS en Express**
Nombre del servicio	`http://backend:3000/api`	**No funciona nunca desde el navegador**

ERROR HABITUAL

En la consola del navegador: `Access to XMLHttpRequest at 'http://localhost:3000/api/tareas' from origin 'http://localhost:8080' has been blocked by CORS policy`. Estás usando una URL absoluta a otro puerto sin CORS en el backend.

Si tu proyecto ya usa una URL absoluta en `environment.ts` y preferís no cambiarla, habilitá CORS solo para el origen del frontend:

```backend (con el paquete cors: npm install cors)
import cors from 'cors';
app.use(cors({ origin: process.env.CORS_ORIGIN ?? 'http://localhost:4200' }));
```

PARA SABER

Con el proxy, además, la URL no depende del dominio: el mismo build funciona en `localhost:8080` y en `https://tu-dominio.com`.

EXPLICACIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

El `compose.yaml` de la app completa

ejemplos/03-angular-express-mysql/compose.yaml

```yaml
# Ejemplo 03: Angular (Nginx) + Express + MySQL — configuración de DESARROLLO local
# Uso:  1) copiar .env.example como .env   2) docker compose up -d --build
#       3) abrir http://localhost:8080

name: tareas-fullstack

services:
  db:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Falta MYSQL_ROOT_PASSWORD en .env}
      MYSQL_DATABASE: ${MYSQL_DATABASE:?Falta MYSQL_DATABASE en .env}
      MYSQL_USER: ${MYSQL_USER:?Falta MYSQL_USER en .env}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Falta MYSQL_PASSWORD en .env}
    volumes:
      - datos-mysql:/var/lib/mysql
      - ./db/init:/docker-entrypoint-initdb.d:ro
    ports:
      - "127.0.0.1:3307:3306"   # para Workbench/DBeaver desde tu compu
    networks:
      - red-app
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "--silent"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 60s

  backend:
    build: ./backend
    environment:
      PORT: 3000
      DB_HOST: db
      DB_PORT: 3306
      DB_USER: ${MYSQL_USER}
      DB_PASSWORD: ${MYSQL_PASSWORD}
      DB_NAME: ${MYSQL_DATABASE}
    ports:
      - "3000:3000"             # para probar la API directo y para el proxy de `ng serve`
    networks:
      - red-app
    depends_on:
      db:
        condition: service_healthy

  frontend:
    build: ./frontend
    ports:
```

- `db` y `backend`: iguales al ejemplo 02.
- `frontend`: se construye con su Dockerfile y publica `8080:80`.
- `depends_on` con `restart: true`: si Compose recrea el backend, reinicia Nginx para que vuelva a encontrarlo.
- El backend publica el 3000 para probar la API directo y para el modo B (`ng serve`). Si solo usás el modo A, no hace falta.

> **ATENCIÓN**
>
> Comparte puertos con el ejemplo 02: antes de levantar este, ejecutá `docker compose down` en `02-api-mysql`.

```
      - "8080:80"               # la app
completa: http://localhost:8080
    networks:
      - red-app
    depends_on:
      backend:
        condition: service_started
        restart: true           # si se recrea el
backend, se reinicia Nginx

volumes:
  datos-mysql:

networks:
  red-app:
```

DEMO DEL DOCENTE · Módulo 8 · Dockerizar Angular + Express + MySQL

Demo A: todo en Docker

DÓNDE SE EJECUTA

En la carpeta `ejemplos/03-angular-express-mysql`.

Terminal

```
cp .env.example .env
docker compose up -d --build
docker compose ps
```

DURANTE EL BUILD DEL FRONTEND

```
 => [frontend build 4/6] RUN npm ci
 => [frontend build 6/6] RUN npm run build
...
Application bundle generation complete.
Output location: /app/dist/frontend
```

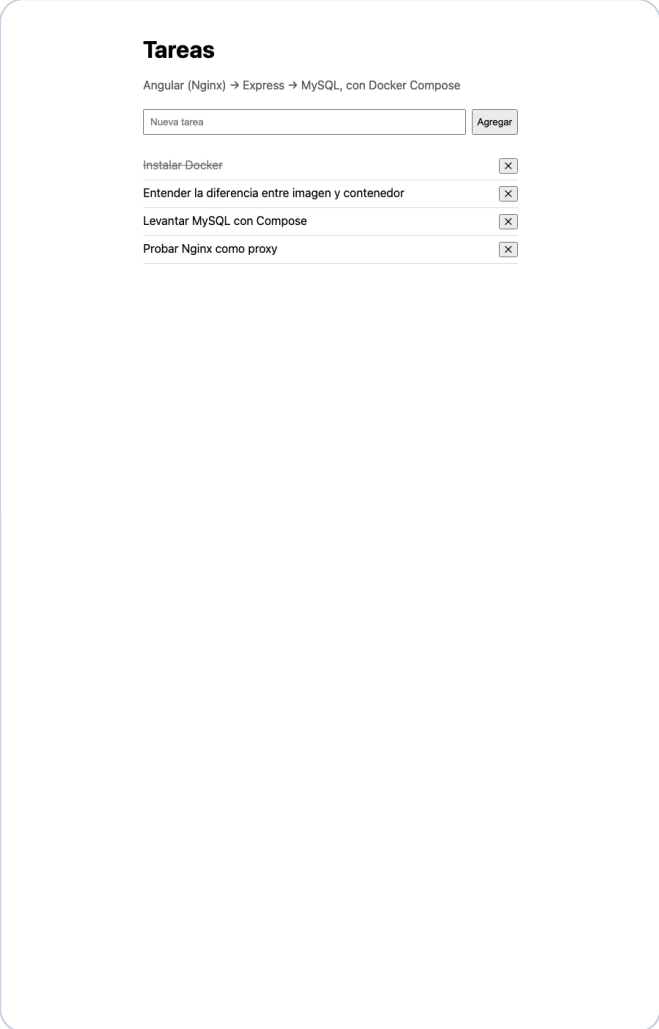
Abrí **http://localhost:8080**, agregá y borrá tareas. Mirá cómo pasan las peticiones por Nginx:

Terminal

```
docker compose logs -f frontend
```

REGISTROS DE NGINX

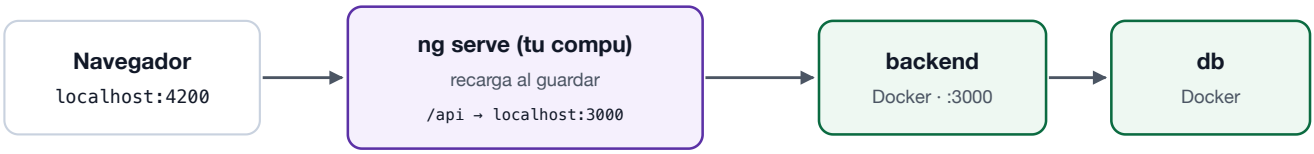
```
"GET / HTTP/1.1" 200 773
"GET /api/tareas HTTP/1.1" 200 326
"POST /api/tareas HTTP/1.1" 201 62
```



Resultado esperado: la lista viene de MySQL, pasando por Nginx y Express.

DEMO DEL DOCENTE Módulo 8 · Dockerizar Angular + Express + MySQL

Demo B: Angular con `ng serve` y la API en Docker



Terminal

```
docker compose up -d db backend   # solo base y API
cd frontend
npm install
npm start                         # = ng serve, con proxy.conf.json
```

Requiere Node en tu compu, compatible con tu versión de Angular (ver Versiones de Node).

ejemplos/03-angular-express-mysql/frontend/proxy.conf.json

```
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false
  }
}
```

angular.json (fragmento)

```
        "serve": {
          "builder": "@angular/build:dev-server",
          "options": {
            "proxyConfig": "proxy.conf.json"
          },
```

PARA SABER

Acá el proxy apunta a `localhost:3000` porque `ng serve` corre **en tu compu**, no en un contenedor: para él, `localhost:3000` es el puerto publicado del backend.

EXPLICACIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

Adaptar tu proyecto existente (sin reescribirlo)

1. **Relevá:** versión de Node y Angular, puertos, variables, script SQL.
2. **Configuración por variables:** buscá `localhost`, usuarios y claves escritos en el código del backend y pasalos a `process.env` (con valores por defecto para desarrollo si querés).
3. **Backend:** copiá el `Dockerfile` y el `.dockerignore` del ejemplo; ajustá el `CMD` a tu archivo de entrada (o `npm start`).
4. **Base:** tu script de creación de tablas va a `db/init/01-esquema.sql`.
5. **Compose:** partí del `compose.yaml` del ejemplo 02 y cambiá nombres, variables y puertos.
6. **Frontend (si hay tiempo):** URL relativa `/api` + `nginx.conf` + Dockerfile de dos etapas con la ruta de `dist` correcta.

IDEA CLAVE

Tus capas (rutas, controladores, servicios, repositorios), tus validaciones y tu estructura de carpetas **no cambian**. Docker solo agrega archivos alrededor.

7 **Probá desde cero:** `docker compose down -v` y `up -d --build`.

Si tu proyecto usa…	Entonces…
`dotenv`	Dejalo: sin archivo `.env` en el contenedor no hace nada y las variables de Compose tienen prioridad.
`nodemon` en `npm start`	Usá `CMD ["node", "src/index.js"]` (nodemon es para desarrollo).
Una conexión única al arrancar	Imprescindible el `healthcheck` + `service_healthy`.
`environment.ts` con URL absoluta	Cambiala a `/api` + proxy, o habilitá CORS.
Otra versión de Angular	Elegí la imagen de Node compatible (siguiente diapositiva).

EXPLICACIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

Versiones de Node para compilar Angular

Angular	Node requerido (por `@angular/core`)	Imagen sugerida
17	`^18.13.0 || >=20.9.0`	`node:20-alpine`
18 y 19	`^18.19.1 || ^20.11.1 || >=22.0.0`	`node:22-alpine`
20 y 21	`^20.19.0 || ^22.12.0 || >=24.0.0`	`node:22-alpine` o `node:24-alpine`
22	`^22.22.3 || ^24.15.0 || >=26.0.0`	`node:24-alpine`

ERROR HABITUAL

`The Angular CLI requires a minimum Node.js version of …` durante `npm run build`: la imagen base de la etapa `build` tiene un Node incompatible. Cambiá la línea `FROM node:…-alpine AS build`.

PARA SABER

Fuente: campo `engines` publicado en npm por cada versión (consultado en octubre de 2026). Para tu caso exacto: `npm view @angular/core@TU_VERSION engines`.

EN PROGRAMACIÓN

Lo más simple: usá la misma versión mayor de Node con la que desarrollás ese proyecto (`node --version`).

EXPLICACIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

Problemas habituales al dockerizar la app completa

Síntoma	Causa probable	Cómo investigar / resolver
`npm ci` falla: `` `npm ci` can only install packages when your package.json and package-lock.json … are in sync ``	Lockfile desactualizado o inexistente	`npm install` en tu compu y versioná el lockfile
`COPY failed` / `"/app/dist/frontend/browser": not found`	La ruta de salida de `ng build` es otra	Mirá la línea `Output location` del build y `outputPath` en `angular.json`
Página en blanco y en la consola 404 de `main-*.js`	Se copió la carpeta equivocada o `<base href>` no es `/`	Revisá que en `/usr/share/nginx/html` esté `index.html`: `docker compose exec frontend ls /usr/share/nginx/html`
404 de Nginx al recargar una ruta de Angular	Falta `try_files … /index.html`	Corregí `nginx.conf` y reconstruí
`502 Bad Gateway` en `/api/…`	El backend está caído o reiniciando	`docker compose ps` y `docker compose logs backend`
Error de CORS en la consola	URL absoluta a otro puerto	Usá `/api` relativa con proxy, o habilitá CORS
`ERR_NAME_NOT_RESOLVED` con `backend`	Usaste el nombre del servicio en Angular	Desde el navegador no existe: `/api`
`port is already allocated` (8080, 3000 o 3307)	Otro proyecto o el ejemplo 02 sigue arriba	`docker ps`; bajá el otro o cambiá el puerto de la izquierda
El backend no conecta / variables vacías	Falta `.env` o el host es `localhost`	`docker compose config` y `docker compose logs backend`

ACTIVIDAD Módulo 8 · Dockerizar Angular + Express + MySQL

Actividad: la app completa

En `ejemplos/03-angular-express-mysql`. Tiempo: 20 minutos.

1. Levantá todo y usá la app en `http://localhost:8080`.
2. En las herramientas de desarrollo del navegador (pestaña Red), ¿a qué URL va la petición de la lista?
3. Cambiá el título `Tareas` por `Tareas del equipo` en `frontend/src/app/app.ts`. ¿Qué comando hace falta para verlo?
4. Detené solo el backend. ¿Qué ves en la app y en `/api/tareas`? Volvé a iniciarlo.
5. Cambiá `apiUrl` a `'http://backend:3000/api/tareas'`, reconstruí y mirá la consola del navegador. Después dejalo como estaba.
6. **Extra:** si tenés Node compatible, levantá el modo B con `ng serve`.

SOLUCIÓN Módulo 8 · Dockerizar Angular + Express + MySQL

Solución: la app completa

Terminal

```
cp .env.example .env
docker compose up -d --build

# después de cambiar el título:
docker compose up -d --build frontend

docker compose stop backend
docker compose start backend
```

RESULTADO ESPERADO

- La petición va a `http://localhost:8080/api/tareas` (mismo origen).
- El título cambia recién después de reconstruir la imagen del frontend: Nginx sirve archivos ya compilados.
- Con el backend detenido, la app muestra “No se pudo conectar con la API…” y `/api/tareas` devuelve **502 Bad Gateway**.
- Con `http://backend:3000`, la consola del navegador muestra `ERR_NAME_NOT_RESOLVED`: el navegador no conoce los nombres de Docker.

REPASO Módulo 8 · Dockerizar Angular + Express + MySQL

Recapitulación del módulo 8

IDEA CLAVE

- Angular se compila en una etapa con Node y se sirve con Nginx en otra (multi-stage).
- El código de Angular corre en el **navegador**: nada de nombres de servicio ni `localhost` de contenedores.
- URL relativa `/api` + proxy (Nginx o `ng serve`) = sin CORS y el mismo build en todos lados.
- `try_files … /index.html` para que funcionen las rutas del router.
- Desarrollo (`ng serve`) y producción (Nginx) se configuran por separado.

PREGUNTA PARA COMPROBAR

¿Por qué Nginx sí puede usar `http://backend:3000` y Angular no?

Porque Nginx se ejecuta dentro de un contenedor conectado a la red de Compose, donde existe el nombre `backend`. El código de Angular se ejecuta en el navegador, fuera de esa red.

MÓDULO 9

Variables de entorno y buenas prácticas

Configurar sin tocar el código y sin regalar contraseñas en GitHub.

Objetivo

Separar configuración de código, usar `.env` y `.env.example` correctamente y evitar exponer secretos.

Vamos a ver

- Qué va en variables
- `.env`, `env_file` e interpolación
- `ENV` vs `environment`
- `.gitignore` vs `.dockerignore`
- Secretos, desarrollo y producción

Tiempo estimado

25 minutos

EXPLICACIÓN Módulo 9 · Variables de entorno y buenas prácticas

¿Qué configuración conviene separar del código?

En variables de entorno

- ✓ Credenciales: usuarios, contraseñas, tokens, claves de API.
- ✓ Direcciones: host y puerto de la base, URL de otros servicios.
- ✓ Lo que cambia entre entornos: `NODE_ENV`, nivel de logs, orígenes de CORS.
- ✓ Puertos en los que escucha la app.

En el código

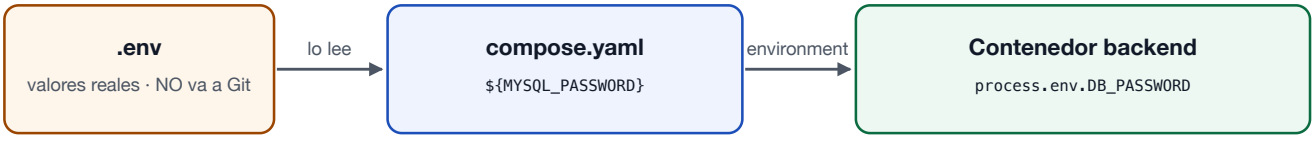
- ✗ La lógica de negocio y las reglas de validación.
- ✗ Las rutas de la API y la estructura del proyecto.
- ✗ Valores que nunca cambian entre entornos.

IDEA CLAVE

Prueba rápida: **¿podrías publicar tu repositorio ahora mismo sin exponer ninguna credencial?** Si la respuesta es no, hay configuración que debería estar en variables.

EXPLICACIÓN Módulo 9 · Variables de entorno y buenas prácticas

`.env` y `.env.example`



ejemplos/02-api-mysql/.env.example

```
# Plantilla de configuración para DESARROLLO.
# 1) Copiala como .env      2) Cambiá las
contraseñas por valores propios.
# El archivo .env NO se sube a GitHub (está
en .gitignore).

MYSQL_ROOT_PASSWORD=cambiar-root-local
MYSQL_DATABASE=tareas_db
MYSQL_USER=tareas_app
MYSQL_PASSWORD=cambiar-app-local
```

	`.env`	`.env.example`
Contiene	Tus valores reales	Nombres de variables, valores de ejemplo y comentarios
¿Va a Git?	**Nunca**	**Sí**
Para qué	Que tu entorno funcione	Que otra persona sepa qué configurar
Se crea con	`cp .env.example .env`	Lo escribís vos y lo mantenés al día

EXPLICACIÓN Módulo 9 · Variables de entorno y buenas prácticas

Interpolación, `environment` y `env_file`

Lo que usamos: interpolación + `environment`

yaml

```
    environment:
      PORT: 3000
      DB_HOST: db            # nombre del
servicio, NO localhost
      DB_PORT: 3306          # puerto interno
de MySQL, NO el publicado
      DB_USER: ${MYSQL_USER}
      DB_PASSWORD: ${MYSQL_PASSWORD}
      DB_NAME: ${MYSQL_DATABASE}
```

Cada servicio recibe **solo** las variables que necesita, con el nombre que espera su código. Valores fijos (`DB_HOST: db`) quedan a la vista en el archivo.

Alternativa: `env_file`

compose.yaml (otra forma)

```
  backend:
    build: ./backend
    env_file:
      - ./backend/.env
```

Pasa **todas** las variables del archivo al contenedor. Es cómodo si tu backend ya tiene su propio `.env` con nombres que funcionan.

Mecanismo	Lo usa	Para
`.env` de la carpeta + `${VAR}`	Compose	Completar valores **dentro** de `compose.yaml`
`environment:`	Contenedor	Variables explícitas del servicio
`env_file:`	Contenedor	Todas las variables de un archivo

> PARA SABER
>
> Si tenés una variable con el mismo nombre definida en tu terminal, para la interpolación gana la de la terminal sobre la del `.env`. Ante la duda: `docker compose config` muestra los valores finales (¡y las claves! no lo compartas).

EXPLICACIÓN Módulo 9 · Variables de entorno y buenas prácticas

`ENV` en el Dockerfile vs `environment` en Compose

	`ENV` (Dockerfile)	`environment` (Compose)
Cuándo se define	Al **construir** la imagen	Al **crear** el contenedor
Dónde queda	Dentro de la imagen, para siempre	Solo en ese contenedor
Uso	Valores por defecto seguros (`PORT=3000`, `NODE_ENV=production`)	Configuración de cada entorno y secretos
Prioridad	Menor	Mayor: reemplaza al `ENV`

Cualquiera que tenga la imagen puede ver sus `ENV`:

Terminal

```
docker history --no-trunc api-express:1.0
```

FRAGMENTO

```
ENV MENSAJE=Hola desde un contenedor
ENV PORT=3000
ENV NODE_ENV=production
```

ATENCIÓN

Por eso **nunca** pongas contraseñas en `ENV` ni en `ARG`: si publicás la imagen en un registro, publicás la contraseña.

EXPLICACIÓN Módulo 9 · Variables de entorno y buenas prácticas

`.gitignore` vs `.dockerignore`

	`.gitignore`	`.dockerignore`
Evita que…	un archivo se suba a **Git/GitHub**	un archivo entre al **contexto de build** (y a la imagen)
Lo lee	Git	`docker build`
Dónde va	Raíz del repositorio (o subcarpetas)	Junto al Dockerfile (raíz del contexto)
Ambos deberían incluir	`.env` · `node_modules` · archivos generados	

ERROR HABITUAL

Tener `.env` en `.gitignore` pero no en `.dockerignore`: no llega a GitHub, pero `COPY . .` lo mete dentro de la imagen. Si la imagen se publica, el secreto también.

.gitignore (raíz del repositorio del curso)

```
# Configuración local y secretos: NUNCA se suben
.env
.env.*
!.env.example
!.env.prod.example

# Dependencias y compilados
node_modules/
dist/
.angular/

# Archivos del sistema y editores
.DS_Store
Thumbs.db
.vscode/
.idea/

# Copias de seguridad generadas en las prácticas
*.sql.bak
backup*.sql
```

EXPLICACIÓN Módulo 9 · Variables de entorno y buenas prácticas

Nunca subas secretos reales a GitHub

- Hay programas que **recorren repositorios públicos** buscando contraseñas y claves de API apenas se publican.
- Borrar el archivo en un commit nuevo **no alcanza**: el secreto sigue en el historial de Git.
- Un repositorio privado hoy puede ser público mañana, o compartirse con más gente.

IDEA CLAVE

Si subiste un secreto: cambiá esa contraseña o revocá esa clave **inmediatamente**. Después limpiá el repositorio. El orden importa: primero invalidar.

Antes de cada commit

Terminal

```
git status          # ¿aparece .env? ¿backup.sql?
git diff --staged   # ¿qué estoy por subir exactamente?
```

PARA SABER

GitHub ofrece *secret scanning* y *push protection*, que detectan algunos formatos de claves conocidos. Ayudan, pero no reconocen una contraseña de MySQL inventada por vos.

EXPLICACIÓN Módulo 9 · Variables de entorno y buenas prácticas

Un `.env` no es un sistema de gestión de secretos

El `.env` es texto plano en un disco: lo puede leer cualquiera con acceso a esa carpeta, a una copia de seguridad o a `docker inspect`. Para aprender y para desarrollo está bien; en producción es lo mínimo, no lo ideal.

EN PROGRAMACIÓN

En un servidor, al menos: archivo fuera del repositorio, permisos restringidos (`chmod 600 .env`), claves distintas a las de desarrollo y que solo conozca quien debe.

Opciones más robustas (para conocer)

- **Secrets de Compose:** montan el secreto como archivo en `/run/secrets/…` en lugar de variable. La imagen de MySQL acepta `MYSQL_PASSWORD_FILE` para leerlo así.
- **Secrets de la plataforma:** GitHub Actions, proveedores de nube y servicios de hosting tienen su propio almacén cifrado de secretos.
- **Gestores dedicados:** herramientas específicas para guardar y rotar secretos en organizaciones.

EXPLICACIÓN Módulo 9 · Variables de entorno y buenas prácticas

Desarrollo vs producción

Aspecto	Desarrollo (tu compu)	Producción (servidor)
Contraseñas	Simples pero propias, en `.env`	Largas, aleatorias, distintas, fuera del repo
Puerto de la base	`127.0.0.1:3307` para tus herramientas	Sin publicar
Imágenes	`build:` desde tu código	`image:` con versión publicada
`NODE_ENV`	`development` o `production`	`production`
Reinicio automático	No hace falta	`restart: unless-stopped`
HTTPS	No	Sí, con proxy inverso
Datos	De prueba, se pueden borrar	Reales: backups periódicos y probados

> PARA SABER
>
> En el ejemplo 03 esto se ve en dos archivos: `compose.yaml` (desarrollo) y `compose.prod.yaml` (producción), cada uno con su plantilla `.env.example` / `.env.prod.example`.

EXPLICACIÓN Módulo 9 · Variables de entorno y buenas prácticas

Evitar credenciales inseguras

> ERROR HABITUAL
>
> `root` / `root`, `123456`, `admin`, o la misma clave de ejemplo del tutorial en producción.

- ✓ La app se conecta con un usuario propio (`MYSQL_USER`), con permisos solo sobre su base.
- ✓ La clave de `root` es distinta y la app no la recibe.
- ✓ Claves de producción generadas al azar y largas.
- ✓ Cada entorno tiene sus propias claves.

Generar una clave aleatoria

macOS / Linux / Git Bash

Terminal

```
openssl rand -hex 24
```

Windows PowerShell

PowerShell

```
[guid]::NewGuid().ToString('N') +
[guid]::NewGuid().ToString('N')
```

EJEMPLO DE RESULTADO (NO USES ESTE)

```
9f2c4e1a7b3d8e6f0a5c2b9d4e7f1a3c6b8d0e2f4a6c8b1d
```

> PARA SABER
>
> Usamos hexadecimal porque solo tiene letras y números: evita problemas con caracteres como `$`, que Compose interpreta como inicio de variable.

REPASO Módulo 9 · Variables de entorno y buenas prácticas

Checklist de buenas prácticas

- ✓ `.env` en `.gitignore` **y** en `.dockerignore`.
- ✓ `.env.example` versionado, completo y sin secretos reales.
- ✓ Ninguna contraseña en el código, el Dockerfile, `compose.yaml` ni el README.
- ✓ `${VAR:?mensaje}` para variables obligatorias.
- ✓ El código valida las variables que necesita al arrancar.
- ✓ Usuario de base propio para la app, no `root`.
- ✓ Base publicada solo en `127.0.0.1` (desarrollo) o sin publicar (producción).
- ✓ Claves distintas por entorno, aleatorias en producción.
- ✓ `git status` antes de cada commit.
- ✓ Si se filtró un secreto: cambiarlo primero, limpiar después.

PREGUNTA PARA COMPROBAR

Alguien del equipo sube el `.env` “solo por un rato” y lo borra en el commit siguiente. ¿Está resuelto?

No. El archivo sigue en el historial de Git y pudo haber sido copiado. Hay que cambiar todas las contraseñas que contenía.

MÓDULO 10

Deploy: llevar la aplicación a un servidor

Del proyecto en tu compu a una aplicación que usa gente real. Sin contratar nada: lo simulamos localmente.

Objetivo

Entender el flujo profesional de despliegue con imágenes, registro y Compose, y qué partes cambian entre desarrollo y producción.

Material

`ejemplos/03-angular-express-mysql/compose.prod.yaml` y `.env.prod.example`.

Tiempo estimado

40 minutos · conceptual + simulación

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

¿Qué significa desplegar?

Desplegar (*deploy*) es poner una versión de tu aplicación a funcionar en un entorno donde la usan otras personas: normalmente un **servidor Linux** accesible por Internet, con un dominio y HTTPS.

ANALOGÍA

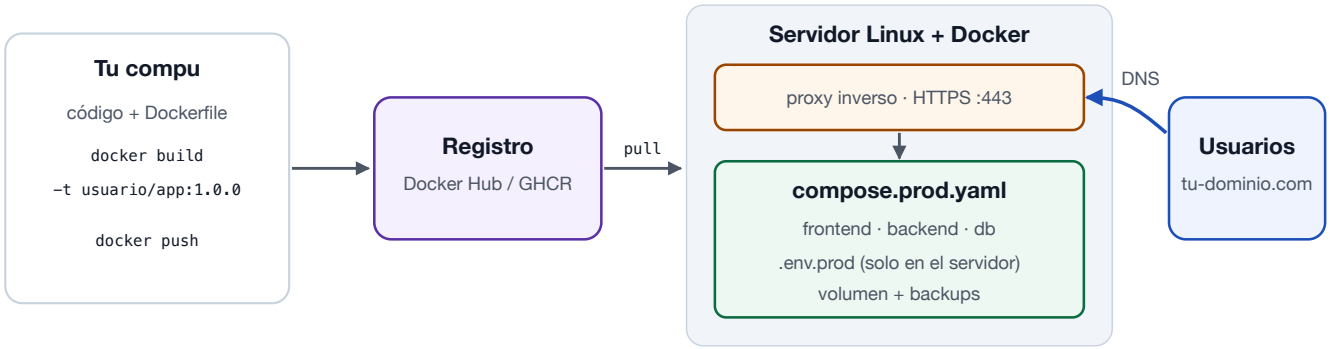
Hasta ahora cocinaste en tu casa para probar. Desplegar es abrir el restaurante: misma receta, pero con clientes reales, horarios, seguridad y la heladera que no se puede vaciar por error.

Por qué Docker lo facilita

- ✓ La imagen que probaste es **exactamente** la que corre en el servidor.
- ✓ El servidor solo necesita Docker: no Node, ni versiones de librerías.
- ✓ Actualizar = cambiar la versión de la imagen. Volver atrás = usar la anterior.
- ✓ La configuración del entorno queda escrita en un archivo (`compose.prod.yaml`).

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

El flujo, de tu compu al servidor



IDEA CLAVE

Se construye **una vez**, se publica con **versión**, y el servidor **descarga** esa versión. La configuración y los secretos de producción viven **solo** en el servidor.

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

¿Construir en el servidor o descargar del registro?

	Construir en el servidor	Construir una vez y publicar en un registro
Cómo	`git clone` en el servidor + `docker compose up -d --build`	`docker push` desde tu compu (o desde CI) + `docker compose pull` en el servidor
Ventaja	Simple, no necesita registro	Lo que probaste es lo que se ejecuta; deploy y rollback rápidos
Desventaja	El servidor necesita el código y recursos para compilar; cada build puede salir distinto	Hay que manejar un registro y credenciales
Para	Proyectos chicos, pruebas	Trabajo profesional ✔

PARA SABER

CI/CD (integración y entrega continuas): herramientas como GitHub Actions pueden construir y publicar las imágenes automáticamente en cada cambio. Docker no lo hace solo; es un paso aparte.

DEMO DEL DOCENTE Módulo 10 · Deploy: llevar la aplicación a un servidor

Publicar imágenes en un registro

Terminal

```
docker login                              # una vez, con tu usuario de Docker Hub

docker build -t tu-usuario/tareas-backend:1.0.0 ./backend
docker build -t tu-usuario/tareas-frontend:1.0.0 ./frontend

docker push tu-usuario/tareas-backend:1.0.0
docker push tu-usuario/tareas-frontend:1.0.0
```

Si ya tenías la imagen con otro nombre, podés agregarle uno: `docker tag tareas-backend:1.0.0 tu-usuario/tareas-backend:1.0.0`.

ATENCIÓN

En Docker Hub, los repositorios nuevos suelen ser **públicos**. Esa es una razón más para que la imagen no tenga secretos adentro.

IDEA CLAVE

Usá **versiones** (`1.0.0`, `1.0.1`), no `latest`: sabés qué está corriendo y podés volver atrás.

ERROR HABITUAL

Arquitectura: si construís en una Mac con Apple silicon (ARM) y el servidor es Intel/AMD, el contenedor puede fallar con `exec format error`. Construí para la plataforma del servidor:

Terminal

```
docker buildx build --platform linux/amd64 -t tu-usuario/tareas-backend:1.0.0 --push ./backend
```

`compose.prod.yaml`

ejemplos/03-angular-express-mysql/compose.prod.yaml

```yaml
# Ejemplo 03: configuración de PRODUCCIÓN (didáctica)
# - No compila nada: usa imágenes ya construidas y publicadas en un registro.
# - La base de datos NO se publica: solo la ve el backend por la red interna.
# - La web queda en 127.0.0.1:8080; en un servidor real, un proxy inverso con HTTPS
#   (Caddy, Nginx, Traefik...) recibe el tráfico público en el 443 y lo reenvía acá.
# Uso:  docker compose -f compose.prod.yaml --env-file .env.prod up -d

name: tareas-prod

services:
  db:
    image: mysql:8.4
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?Falta MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE:?Falta MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER:?Falta MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD:?Falta MYSQL_PASSWORD}
    volumes:
      - datos-mysql:/var/lib/mysql
      - ./db/init:/docker-entrypoint-initdb.d:ro
    networks:
      - red-app
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "--silent"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 60s

  backend:
    image: ${IMAGEN_BACKEND:?Falta IMAGEN_BACKEND}
    restart: unless-stopped
    environment:
      PORT: 3000
      DB_HOST: db
      DB_PORT: 3306
      DB_USER: ${MYSQL_USER}
      DB_PASSWORD: ${MYSQL_PASSWORD}
      DB_NAME: ${MYSQL_DATABASE}
    networks:
      - red-app
    depends_on:
      db:
        condition: service_healthy

  frontend:
```

Diferencias con desarrollo

- `image:` con versión en lugar de `build:`: no compila nada.
- `restart: unless-stopped`: si un proceso se cae o el servidor reinicia, Docker lo vuelve a levantar.
- **MySQL sin `ports`**: no se puede llegar a la base desde fuera.
- **Backend sin `ports`**: solo se accede a través de Nginx.
- Web en `127.0.0.1:8080`: la expone el proxy inverso con HTTPS.
- `name: tareas-prod`: no se mezcla con el proyecto de desarrollo.

```
    image: ${IMAGEN_FRONTEND:?Falta
IMAGEN_FRONTEND}
    restart: unless-stopped
    ports:
      - "127.0.0.1:8080:80"
    networks:
      - red-app
    depends_on:
      backend:
        condition: service_started
        restart: true

volumes:
  datos-mysql:

networks:
  red-app:
```

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

Llevar la configuración sin subir secretos

ejemplos/03-angular-express-mysql/.env.prod.example

```
# Plantilla para PRODUCCIÓN. En el servidor
se copia como .env.prod y se completa ahí.
# Nunca se sube a Git con valores reales.
Generá claves largas y distintas, por
ejemplo:
#   openssl rand -hex 24

# Imágenes publicadas en un registro
(usuario/repositorio:versión)
IMAGEN_BACKEND=tu-usuario/tareas-
backend:1.0.0
IMAGEN_FRONTEND=tu-usuario/tareas-
frontend:1.0.0

# Base de datos (las claves vacías hacen que
Compose se detenga con un aviso)
MYSQL_ROOT_PASSWORD=
MYSQL_DATABASE=tareas_db
MYSQL_USER=tareas_app
MYSQL_PASSWORD=
```

1. Al servidor viajan `compose.prod.yaml` y `db/init/` (por `git clone` o `scp`). **El `.env.prod` no.**
2. En el servidor: `cp .env.prod.example .env.prod` y completás los valores con un editor (`nano .env.prod`).
3. Restringís permisos: `chmod 600 .env.prod` (solo tu usuario puede leerlo).

> **RESULTADO ESPERADO**
>
> Probado: con las claves vacías, Compose se detiene con `required variable MYSQL_ROOT_PASSWORD is missing a value: Falta MYSQL_ROOT_PASSWORD`. Nadie puede levantar producción “sin querer” con claves vacías.

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

Levantar los servicios en un servidor Linux

Desde tu compu

```
ssh usuario@IP-DEL-SERVIDOR
```

En el servidor (cada deploy)

```
docker compose -f compose.prod.yaml --env-
file .env.prod pull
docker compose -f compose.prod.yaml --env-
file .env.prod up -d
docker compose -f compose.prod.yaml --env-
file .env.prod ps
```

En el servidor (una sola vez)

```
# 1. Instalar Docker Engine (módulo 3, Linux) y habilitarlo al inicio
sudo systemctl enable docker.service containerd.service

# 2. Traer los archivos del proyecto
git clone https://github.com/tu-usuario/tu-proyecto.git tareas
cd tareas

# 3. Configuración de producción
cp .env.prod.example .env.prod
nano .env.prod
chmod 600 .env.prod
```

PARA SABER

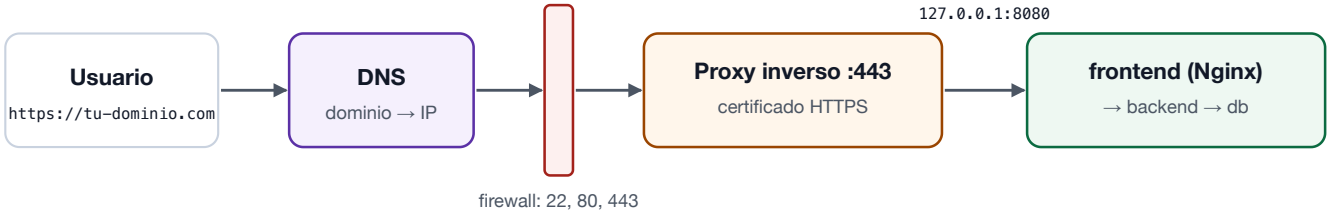
`-f` elige el archivo de Compose y `--env-file` el de variables. Para no repetirlos, podés usar `docker compose --env-file .env.prod -f compose.prod.yaml` en un script `deploy.sh`.

PARA SABER

SSH es la forma estándar de abrir una terminal remota en un servidor. `scp` copia archivos por SSH: `scp .env.prod usuario@IP:~/tareas/`.

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

Puertos, firewall, DNS, HTTPS y proxy inverso (introducción)



- **DNS:** un registro *A* asocia tu dominio con la IP del servidor.
- **Firewall:** solo abiertos SSH (22), HTTP (80) y HTTPS (443).
- **HTTPS:** cifra el tráfico. Los certificados gratuitos de Let's Encrypt se obtienen automáticamente con proxies como Caddy o Traefik.
- **Proxy inverso:** recibe todo el tráfico público y lo reenvía a cada aplicación según el dominio.

Caddyfile (ilustrativo: Caddy instalado en el servidor)

```
tu-dominio.com {
    reverse_proxy 127.0.0.1:8080
}
```

ATENCIÓN

Los puertos que Docker **publica** pueden quedar accesibles aunque el firewall del sistema (ufw, firewalld) diga lo contrario: Docker agrega sus propias reglas. Por eso en producción no se publica la base y la web se publica solo en `127.0.0.1`.

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

Datos y copias de seguridad en producción

- Los datos viven en el volumen `tareas-prod_datos-mysql` **del servidor**.
- Un volumen **no** es un backup: si se rompe el disco o alguien ejecuta `down -v`, se pierde.
- Backups **periódicos** (por ejemplo, un `mysqldump` diario programado con `cron`), guardados **fuera** del servidor.
- **Probá restaurar** cada tanto en otra máquina.

Backup manual en el servidor

```
docker compose -f compose.prod.yaml --env-file .env.prod exec db sh -c 'mysqldump -u root -p$MYSQL_ROOT_PASSWORD $MYSQL_DATABASE > /tmp/backup.sql'
docker compose -f compose.prod.yaml --env-file .env.prod cp db:/tmp/backup.sql ./backup-$(date +%F).sql
```

ATENCIÓN

Nunca ejecutes `down -v` en producción salvo que quieras borrar la base a propósito y tengas un backup probado.

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

Registros, reinicios, actualizaciones y vuelta atrás

Operación diaria (en el servidor)

```
# ¿Está todo arriba?
docker compose -f compose.prod.yaml --env-file .env.prod ps

# ¿Qué pasó? (últimas 100 líneas y en vivo)
docker compose -f compose.prod.yaml --env-file .env.prod logs --tail 100 -f backend

# Reiniciar un servicio
docker compose -f compose.prod.yaml --env-file .env.prod restart backend
```

Actualizar a una versión nueva

1. En `.env.prod`: `IMAGEN_BACKEND=tu-usuario/tareas-backend:1.0.1`
2. `docker compose … pull` y `docker compose … up -d`
3. Compose recrea **solo** el servicio cuya imagen cambió.

Volver atrás

Poné de nuevo `1.0.0` en `.env.prod` y `up -d`.

RESULTADO ESPERADO

Probado localmente: al cambiar la etiqueta del backend, `up -d` recreó solo `tareas-prod-backend-1`; al volver a `1.0.0`, ídem.

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

¿Qué se reutiliza y qué cambia?

Se reutiliza tal cual

- ✓ El código y los Dockerfiles.
- ✓ Las imágenes (la misma versión probada).
- ✓ Los scripts de inicialización de la base.
- ✓ `nginx.conf` y la URL relativa `/api`.
- ✓ La estructura de servicios, red y volumen.

Es específico de producción

- Valores de `.env.prod` (claves fuertes y únicas).
- `image:` con versión en lugar de `build:`.
- Puertos mínimos (base y backend sin publicar).
- `restart: unless-stopped`.
- Proxy inverso, HTTPS, DNS y firewall.
- Backups programados y monitoreo.

EXPLICACIÓN Módulo 10 · Deploy: llevar la aplicación a un servidor

Lo que Docker no garantiza por sí solo

Seguridad

Hay que actualizar imágenes y sistema, usar claves fuertes, cerrar puertos y revisar dependencias.

Escalabilidad

Un `compose.yaml` en un servidor es un servidor. Crecer requiere otras herramientas y diseño.

Disponibilidad

Si el servidor se apaga, la app se apaga. `restart` no protege contra eso.

Deploy automático

Construir, publicar y actualizar en cada cambio es trabajo de CI/CD.

Backups

Los volúmenes no se respaldan solos.

Monitoreo

Saber que algo falló antes que tus usuarios requiere alertas y métricas.

IDEA CLAVE

Docker hace que el entorno sea **reproducible**. Eso facilita todo lo demás, pero no lo reemplaza.

ACTIVIDAD Módulo 10 · Deploy: llevar la aplicación a un servidor

Actividad: simular un deploy en tu compu

DÓNDE SE EJECUTA

En `ejemplos/03-angular-express-mysql`. Tiempo: 15 minutos. Nada sale de tu computadora.

1. Bajá el entorno de desarrollo: `docker compose down`.
2. Construí las imágenes “de producción” con versión: `tareas-backend:1.0.0` y `tareas-frontend:1.0.0`.
3. Creá `.env.prod` desde la plantilla. Completá las imágenes con esos nombres y generá dos claves aleatorias.

PREGUNTA PARA COMPROBAR

En un deploy real, ¿cuál de estos pasos haría el servidor y cuál tu compu (o CI)?

Construir y publicar (2, y el `push`): tu compu o CI. Crear `.env.prod`, `pull`, `up -d`, actualizar y volver atrás: el servidor.

4. Levantá con `compose.prod.yaml` y abrí `http://localhost:8080`.
5. Intentá conectarte a MySQL desde tu compu por el 3307. ¿Qué pasa? ¿Por qué?
6. Creá la versión `1.0.1` del backend (con `docker tag` alcanza), actualizá y volvé a la `1.0.0`.
7. Bajá todo con `down -v` (es una simulación) y borrá `.env.prod`.

SOLUCIÓN **Módulo 10 · Deploy: llevar la aplicación a un servidor**

Solución: simular un deploy

Terminal

```
docker compose down
docker build -t tareas-backend:1.0.0 ./backend
docker build -t tareas-frontend:1.0.0 ./frontend

cp .env.prod.example .env.prod
# en .env.prod:
#   IMAGEN_BACKEND=tareas-backend:1.0.0
#   IMAGEN_FRONTEND=tareas-frontend:1.0.0
#   MYSQL_ROOT_PASSWORD=(openssl rand -hex 24)
#   MYSQL_PASSWORD=(otra distinta)

docker compose -f compose.prod.yaml --env-file .env.prod up -d
docker compose -f compose.prod.yaml --env-file .env.prod ps

docker tag tareas-backend:1.0.0 tareas-backend:1.0.1
# en .env.prod: IMAGEN_BACKEND=tareas-backend:1.0.1
docker compose -f compose.prod.yaml --env-file .env.prod up -d
# volver: IMAGEN_BACKEND=tareas-backend:1.0.0
docker compose -f compose.prod.yaml --env-file .env.prod up -d

docker compose -f compose.prod.yaml --env-file .env.prod down -v
```

DOCKER COMPOSE … PS (PRODUCCIÓN)

```
SERVICE    STATUS                    PORTS
backend    Up 5 seconds              3000/tcp
db         Up 15 seconds (healthy)   3306/tcp, 33060/tcp
frontend   Up 5 seconds              127.0.0.1:8080->80/tcp
```

RESULTADO ESPERADO

- Solo el frontend tiene un puerto publicado.
- MySQL desde tu compu: `ERROR 2003 (HY000): Can't connect to MySQL server on '127.0.0.1:3307'`. No está publicado: solo lo ve el backend.
- Al cambiar la versión, `up -d` recrea solo `tareas-prod-backend-1`.

Recapitulación del módulo 10

IDEA CLAVE

- Build una vez → imagen con versión → registro → el servidor hace `pull` y `up -d`.
- Configuración y secretos de producción: solo en el servidor (`.env.prod`, `chmod 600`).
- En producción: sin `build`, puertos mínimos, `restart`, proxy inverso con HTTPS.
- Actualizar y volver atrás = cambiar la etiqueta de la imagen.
- Backups periódicos fuera del servidor, y probados.

PREGUNTA PARA COMPROBAR

¿Por qué en `compose.prod.yaml` el frontend publica `127.0.0.1:8080` y no `80:80`?

Porque el tráfico público lo recibe el proxy inverso (con HTTPS) y lo reenvía al 8080 local. Publicar directo en el 80 dejaría la app sin HTTPS y sin el control del proxy; y como Docker se saltea el firewall del sistema, cualquier puerto publicado en todas las interfaces queda expuesto.

MÓDULO 11

Errores frecuentes y diagnóstico

Doce problemas reales, con el mensaje que vas a ver, cómo investigarlo y cómo resolverlo. Todos los mensajes fueron reproducidos con Docker 28.

Objetivo
Diagnosticar con método en lugar de probar cosas al azar.

Cómo usar este módulo
Como referencia durante la práctica integradora. Buscá el síntoma que estás viendo.

Tiempo estimado
30 minutos · incluye "rompé y arreglá"

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

Un método de diagnóstico en 6 preguntas

1. **¿Docker funciona?** `docker info`
2. **¿Qué contenedores hay y en qué estado?** `docker compose ps -a` (o `docker ps -a`)
3. **¿Qué dicen los registros?** `docker compose logs servicio` — el error casi siempre está acá.
4. **¿La configuración final es la que creo?** `docker compose config`
5. **¿Qué ve el contenedor por dentro?** `docker compose exec servicio sh` → `ls`, `env`, `cat`
6. **¿Cambió algo y no reconstruí?** `docker compose up -d --build`

IDEA CLAVE

Leé el mensaje de error **completo** y buscá palabras clave: el nombre de un archivo, un puerto, un host, un código (`ECONNREFUSED`, `EACCES`, `137`).

ATENCIÓN

`docker compose config`, `docker inspect` y `env` muestran contraseñas. Antes de pegar su salida en un foro o en un chat, borrá los secretos.

PARA SABER

Códigos de salida comunes: `0` terminó bien · `1` error de la app · `137` fue forzado a terminar (SIGKILL: `docker kill`, tiempo de `stop` agotado o falta de memoria).

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

1 · Docker no está iniciado

SÍNTOMA

MACOS / LINUX

```
Cannot connect to the Docker daemon at unix:///Users/tu-usuario/.docker/run/docker.sock. Is the docker daemon running?
```

WINDOWS

```
error during connect: ... open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
```

CAUSA PROBABLE

La CLI funciona, pero el Engine no: Docker Desktop está cerrado o todavía arrancando; en Linux, el servicio `docker` está detenido.

INVESTIGAR

Terminal

```
docker info
docker context ls
```

`context ls` muestra a qué Engine apunta la CLI (marcado con `*`).

SOLUCIÓN

Abrí Docker Desktop y esperá *Engine running*. En Linux: `sudo systemctl start docker` (y `enable` para que arranque solo).

2 · Puerto ocupado

SÍNTOMA

LO USA OTRO PROGRAMA

```
ports are not available: exposing port TCP 0.0.0.0:3000 -> 127.0.0.1:0: listen tcp
0.0.0.0:3000: bind: address already in use
```

LO USA OTRO CONTENEDOR

```
Bind for 0.0.0.0:3000 failed: port is already allocated
```

CAUSA PROBABLE

Otro proceso (tu proyecto con `npm start`, un MySQL instalado, otro contenedor) ya escucha en ese puerto de tu compu.

INVESTIGAR

macOS / Linux / Git Bash

Terminal

```
docker ps
lsof -nP -iTCP:3000 -sTCP:LISTEN
```

Windows PowerShell

PowerShell

```
docker ps
netstat -ano | findstr :3000
```

SOLUCIÓN

Detené lo que ocupa el puerto, o cambiá el número de la **izquierda**: `"3001:3000"`.

Ojo: un `docker run` que falló por el puerto deja el contenedor creado. Al reintentar con el mismo nombre: `Conflict. The container name "/api" is already in use`. Borralo con `docker rm api`.

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

3 · El contenedor está detenido

SÍNTOMA

La app no responde y el contenedor no aparece en `docker ps`.

DOCKER PS -A

```
NAMES   STATUS
api     Exited (1) 2 minutes ago
```

CAUSA PROBABLE

El proceso principal terminó (por un error o porque alguien lo detuvo). Un contenedor vive mientras vive su proceso principal.

INVESTIGAR

Terminal

```
docker ps -a
docker logs api
docker inspect --format '{{.State.ExitCode}}' api
```

SOLUCIÓN

Corregí la causa que muestran los logs y volvé a iniciarlo: `docker start api` o `docker compose up -d`. Si cambiaste código o configuración, recrealo.

4 · Imagen no encontrada

SÍNTOMA

NOMBRE MAL ESCRITO

```
pull access denied for ngnix, repository does not exist or may require 'docker login'
```

ETIQUETA INEXISTENTE

```
failed to resolve reference "docker.io/library/node:24-alpne": docker.io/library/node:24-alpne: not found
```

CAUSA PROBABLE

Error de tipeo en el nombre o la etiqueta; o es una imagen propia que nunca construiste (Docker la busca en Docker Hub y no existe).

INVESTIGAR

Terminal

```
docker images
docker search nginx
```

O buscá las etiquetas válidas en hub.docker.com (https://hub.docker.com).

SOLUCIÓN

Corregí el nombre o la etiqueta. Si es tu imagen, construila primero con `docker build -t nombre:tag .` (o `build:` en Compose).

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

5 · La aplicación termina inmediatamente

SÍNTOMA

El contenedor pasa a `Exited` a los pocos segundos.

DOCKER LOGS

```
Error: Cannot find module '/app/src/server.js'
  code: 'MODULE_NOT_FOUND'
```

CAUSAS PROBABLES

- `CMD` apunta a un archivo inexistente.
- Faltan variables y la app sale a propósito (`Faltan variables de entorno: DB_NAME`).
- La imagen no tiene un proceso que quede corriendo (por ejemplo, `ubuntu` ejecuta una shell y termina: `Exited (0)`).

INVESTIGAR

Terminal

```
docker logs api
docker run -it --rm --entrypoint sh api-express:1.0
# adentro: ls -R /app/src
```

Así entrás a la imagen sin ejecutar el `CMD` y revisás qué archivos tiene.

SOLUCIÓN

Corregí el `CMD` o la ruta, completá las variables, o asegurate de que el proceso principal quede en primer plano (un servidor escuchando).

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

6 · Error de conexión con MySQL

SÍNTOMA

DOCKER COMPOSE LOGS BACKEND

```
Error: getaddrinfo ENOTFOUND mysql
Error: connect ECONNREFUSED 172.24.0.2:3306
Error: Access denied for user 'tareas_app'@'172.24.0.3' (using password: YES)
```

CAUSA PROBABLE

- `ENOTFOUND`: el host no es un servicio de la red (nombre mal escrito u otra red).
- `ECONNREFUSED` con IP de Docker: MySQL todavía no está listo.
- `Access denied`: credenciales distintas a las de la primera inicialización.

INVESTIGAR

Terminal

```
docker compose ps
docker compose logs db
docker compose exec db mysql -u tareas_app -p tareas_db
```

SOLUCIÓN

`DB_HOST` = nombre del servicio; `healthcheck` + `condition: service_healthy`; credenciales iguales a `.env` (si cambiaron después de inicializar: módulo 7).

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

7 · El backend usa `localhost`

SÍNTOMA

Funciona con `npm start` en tu compu, pero no en Compose.

DOCKER COMPOSE LOGS BACKEND

```
Error: connect ECONNREFUSED ::1:3306
Error: connect ECONNREFUSED 127.0.0.1:3306
```

CAUSA PROBABLE

Dentro del contenedor, `localhost` (`127.0.0.1` o `::1` en IPv6) es el propio backend, donde no hay MySQL. Pasa cuando el host está escrito en el código o falta `DB_HOST` y se usa el valor por defecto.

INVESTIGAR

Terminal

```
docker compose config
docker compose exec backend env
```

Buscá `DB_HOST`. Recordá que `env` también muestra claves.

SOLUCIÓN

En `compose.yaml`: `DB_HOST: db` (el nombre del servicio). En el código: leer `process.env.DB_HOST`.

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

8 · Variables de entorno faltantes

SÍNTOMA

AL EJECUTAR DOCKER COMPOSE UP

```
required variable MYSQL_ROOT_PASSWORD is missing a value: Falta MYSQL_ROOT_PASSWORD en .env
```

O EN LOS LOGS DE LA APP

```
Faltan variables de entorno: DB_NAME
```

CAUSA PROBABLE

No existe `.env` (por ejemplo, al clonar el repo), está en otra carpeta, o le falta una variable que agregaste a `.env.example`.

INVESTIGAR

Terminal

```
ls -a            # ¿existe .env en esta carpeta?
docker compose config
```

SOLUCIÓN

`cp .env.example .env` y completar. Prevenirlo: `${VAR:?mensaje}` en Compose y validación al arrancar en el código, para que el error sea claro e inmediato.

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

9 · Se perdieron los datos

SÍNTOMA

Después de recrear los contenedores, la base está vacía o con los datos iniciales.

CAUSA PROBABLE

- MySQL sin volumen con nombre en `/var/lib/mysql`.
- Se ejecutó `docker compose down -v` o `docker volume rm`.
- Cambió el nombre del proyecto (otra carpeta o `name:`) y Compose creó un volumen nuevo.

INVESTIGAR

Terminal

```
docker volume ls
docker inspect --format '{{json .Mounts}}' tareas-api-db-1
```

SOLUCIÓN

Volumen con nombre en `compose.yaml`; no usar `-v` sin pensar; restaurar desde un backup. Si el volumen viejo existe (otro nombre de proyecto), los datos siguen ahí.

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

10 · Cambié el código y no se ve el cambio

SÍNTOMA

Editaste una ruta, guardaste, recargaste… y la API responde lo de antes.

CAUSA PROBABLE

El contenedor ejecuta el código copiado **dentro de la imagen** al construirla. Tus archivos nuevos no están ahí hasta que reconstruís.

INVESTIGAR

Terminal

```
docker compose images
docker compose exec backend cat src/rutas/tareas.js
```

Compará la fecha de la imagen y el contenido del archivo adentro.

SOLUCIÓN

`docker compose up -d --build`. En un `docker run` suelto: `docker build` + recrear el contenedor.

Para desarrollo con recarga automática existen técnicas (montar el código con un bind mount o `docker compose watch`); quedan para más adelante.

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

11 · Archivos excluidos por `.dockerignore`

SÍNTOMA

AL CONSTRUIR

```
failed to compute cache key: failed to calculate checksum of ref ...: "/package-lock.json":
not found
```

AL EJECUTAR

```
Error: Cannot find module '/app/src/rutas/tareas.js'
```

CAUSA PROBABLE

Un patrón de `.dockerignore` deja afuera algo necesario (el lockfile, una carpeta de código, un archivo de configuración).

INVESTIGAR

Terminal

```
cat .dockerignore
docker run --rm --entrypoint sh api-express:1.0 -c "ls -R /app/src"
```

SOLUCIÓN

Ajustá los patrones. Para excluir todo menos un archivo se usa `!`: `.env*` seguido de `!.env.example`.

EXPLICACIÓN Módulo 11 · Errores frecuentes y diagnóstico

12 · Permisos o arquitectura incompatible

SÍNTOMAS

PERMISOS (CON USER NODE)

```
Error: EACCES: permission denied, open '/app/uploads.txt'
```

ARQUITECTURA (POR EJEMPLO MYSQL:5.7 EN APPLE SILICON)

```
no matching manifest for linux/arm64/v8 in the manifest list entries
```

IMAGEN DE OTRA ARQUITECTURA

```
exec format error
```

CAUSA PROBABLE

- La app corre como `node` (sin privilegios) y quiere escribir en una carpeta de `root`.
- La imagen no existe para tu procesador (ARM vs Intel/AMD), o se construyó para otro.

INVESTIGAR

Terminal

```
docker compose exec backend ls -la /app
docker image inspect --format '{{.Architecture}}' mysql:8.4
uname -m
```

SOLUCIÓN

Dockerfile: carpeta escribible (antes de USER node)

```
RUN mkdir -p /app/uploads && chown node:node /app/uploads
```

Mejor aún, guardá archivos subidos en un volumen. Arquitectura: usá una versión con soporte para tu procesador (`mysql:8.4`), o `platform: linux/amd64` en Compose (emulado, más lento); para servidores, `docker buildx build --platform …`.

ACTIVIDAD Módulo 11 · Errores frecuentes y diagnóstico

Actividad: rompé y arreglá

En parejas, con `ejemplos/02-api-mysql`. Una persona introduce **un** error (sin decir cuál); la otra lo diagnostica con el método. Después cambian. Tiempo: 20 minutos.

1. En `.env`, borrá la línea de `MYSQL_DATABASE`.
2. En `compose.yaml`, cambiá `DB_HOST: db` por `DB_HOST: localhost`.
3. En `backend/Dockerfile`, cambiá el `CMD` a `src/server.js` y levantá con `--build`.
4. En `compose.yaml`, publicá el backend como `"3000:3001"`.

IDEA CLAVE

Regla: quien diagnostica solo puede usar comandos de diagnóstico (`ps`, `logs`, `config`, `exec`, `inspect`) hasta decir la causa en voz alta. Recién ahí corrige.

ATENCIÓN

Al terminar, dejá todo como estaba: `git checkout -- .` en el repositorio de ejemplos (y revisá tu `.env`, que no está versionado).

SOLUCIÓN Módulo 11 · Errores frecuentes y diagnóstico

Solución: rompé y arreglá

Error introducido	Qué se observa	Dónde se ve
1. Falta `MYSQL_DATABASE`	`required variable MYSQL_DATABASE is missing a value: Falta MYSQL_DATABASE en .env`; no se levanta nada	La salida de `docker compose up`
2. `DB_HOST: localhost`	El backend arranca, `/api/salud` responde `"db":"error","detalle":"ECONNREFUSED"`	`docker compose logs backend`: `connect ECONNREFUSED 127.0.0.1:3306`
3. `CMD` a `src/server.js`	El backend queda `Exited (1)`	`docker compose ps -a` y logs: `Cannot find module '/app/src/server.js'`
4. `"3000:3001"`	El backend está `Up` y sus logs dicen “escuchando en el puerto 3000”, pero `curl` falla: `Recv failure: Connection reset by peer`	`docker compose ps` (columna PORTS: `3000->3001`) vs. el puerto que muestra el log

IDEA CLAVE

En los cuatro casos, la pista estaba en un solo comando: la salida de `up`, `logs` o `ps`. Leer antes de cambiar.

REPASO Módulo 11 · Errores frecuentes y diagnóstico

Recapitulación del módulo 11

IDEA CLAVE

- Primero `ps` y `logs`; después `config` y `exec`.
- `ECONNREFUSED 127.0.0.1` → `localhost` mal usado. `ENOTFOUND` → nombre de servicio. `Access denied` → credenciales.
- `Exited (1)` → error de la app, mirá los logs. `137` → terminado a la fuerza.
- Si no ves tus cambios: `--build`.
- No compartas salidas con secretos.

PREGUNTA PARA COMPROBAR

Alguien del grupo dice: “Docker no anda, ya reinstalé dos veces”. ¿Qué le pedís que te muestre primero?

La salida de `docker info` (¿el Engine responde?), `docker compose ps -a` (¿qué estado tienen los contenedores?) y `docker compose logs` del servicio que falla. Casi nunca hace falta reinstalar.

MÓDULO 12

Repaso y cierre

Todo junto: conceptos, comandos, preguntas, ejercicios y una guía para decidir qué usar en cada caso.

Objetivo

Consolidar lo aprendido y saber reconocer un proyecto bien dockerizado.

Incluye

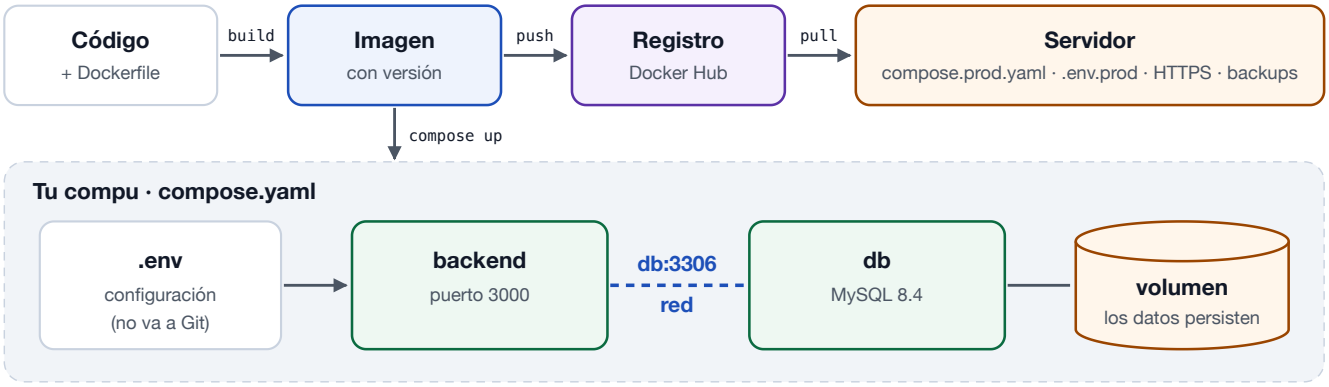
- Resumen visual
- Comandos esenciales
- Preguntas y ejercicios
- Cuestionario con respuestas explicadas
- Guía de decisión y checklist

Tiempo estimado

30 minutos

REPASO Módulo 12 · Repaso y cierre

Resumen visual



Imagen

Paquete de solo lectura. Se construye o se descarga.

Contenedor

Imagen en ejecución. Se recrea sin miedo.

Volumen

Donde viven los datos que importan.

Variables

Misma imagen, distinta configuración.

REPASO Módulo 12 · Repaso y cierre

Comandos esenciales

Docker

Terminal

```
docker run -d --name web -p 8080:80 nginx:1.30-alpine
docker ps -a                        # contenedores (todos)
docker images                       # imágenes
docker logs -f web                  # registros en vivo
docker exec -it web sh              # entrar
docker inspect web                  # configuración completa
docker stop web && docker start web
docker rm -f web                    # borrar contenedor
docker rmi nginx:1.30-alpine        # borrar imagen
docker build -t api:1.0 .           # construir (en la carpeta del Dockerfile)
docker volume ls                    # volúmenes
docker system df                    # espacio usado
```

Docker Compose

Terminal

```
docker compose up -d                # crear e iniciar
docker compose up -d --build        # reconstruir y aplicar cambios
docker compose ps                   # estado
docker compose logs -f backend      # registros
docker compose exec db mysql -u tareas_app -p tareas_db
docker compose config               # configuración final (¡muestra secretos!)
docker compose stop                 # detener sin borrar
docker compose down                 # borrar contenedores y red (datos quedan)
docker compose down -v              # ⚠ también borra los volúmenes
```

PARA SABER

Todos los `docker compose …` se ejecutan en la carpeta del `compose.yaml`.

REPASO Módulo 12 · Repaso y cierre

Preguntas de comprensión

PREGUNTA PARA COMPROBAR

¿Por qué un contenedor arranca en segundos y una máquina virtual tarda minutos?

Porque el contenedor no arranca un sistema operativo completo: es un proceso aislado que comparte el kernel del anfitrión. La VM tiene que iniciar su propio kernel y sistema.

PREGUNTA PARA COMPROBAR

Si ya tengo `depends_on`, ¿para qué el `healthcheck`?

`depends_on` solo ordena el arranque. MySQL tarda en estar listo después de iniciado; con `healthcheck` + `condition: service_healthy`, el backend espera a que acepte conexiones.

PREGUNTA PARA COMPROBAR

¿Qué pasa con un archivo creado dentro de un contenedor si hacés `stop` y `start`? ¿Y si hacés `rm` y `run`?

Con `stop`/`start` sigue (es el mismo contenedor). Con `rm` + `run` se pierde: el contenedor nuevo empieza desde la imagen. Salvo que el archivo esté en un volumen.

PREGUNTA PARA COMPROBAR

¿Por qué Angular no puede llamar a `http://backend:3000`?

Porque su código corre en el navegador del usuario, fuera de la red de Docker, donde el nombre `backend` no existe. Se usa `/api` relativa y un proxy (Nginx) que sí está en la red.

PREGUNTA PARA COMPROBAR

¿Por qué en el Dockerfile copiamos `package*.json` antes que el resto del código?

Para aprovechar la caché: si solo cambia el código, la capa de `npm ci` se reutiliza y el build es mucho más rápido.

PREGUNTA PARA COMPROBAR

Nombrá tres diferencias entre `compose.yaml` y `compose.prod.yaml` del ejemplo 03.

`image:` con versión en lugar de `build:`; base y backend sin puertos publicados; `restart: unless-stopped`; web en `127.0.0.1:8080`; otro nombre de proyecto y otro archivo de variables.

ACTIVIDAD Módulo 12 · Repaso y cierre

Ejercicios cortos

1. Levantá MySQL 8.4 con `docker run`: base `biblioteca`, accesible solo desde tu compu en el puerto 3309, con un volumen llamado `biblio-datos`.

Terminal

```
docker run -d --name biblio-db -e
MYSQL_ROOT_PASSWORD=una-clave-local -e
MYSQL_DATABASE=biblioteca -v biblio-
datos:/var/lib/mysql -p
127.0.0.1:3309:3306 mysql:8.4
```

2. Escribí el Dockerfile de una API Express cuyo archivo de entrada es `server.js` (en la raíz) y que escucha en el puerto 4000.

dockerfile

```
FROM node:24-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY . .
ENV PORT=4000
EXPOSE 4000
USER node
CMD ["node", "server.js"]
```

4. Ordená estas líneas para aprovechar la caché: `COPY . .` · `FROM node:24-alpine` · `RUN npm ci --omit=dev` · `CMD ["node","src/index.js"]` · `WORKDIR /app` · `COPY package.json package-lock.json ./`

`FROM` → `WORKDIR` → `COPY package*.json` → `RUN npm ci` → `COPY . .` → `CMD`.

5. Escribí un `.dockerignore` mínimo para un backend Node.

.dockerignore

```
node_modules
.env
.git
npm-debug.log*
```

6. Alguien de tu equipo clonó tu repo y `docker compose up` dice `required variable MYSQL_PASSWORD is missing a value`. ¿Qué le faltó hacer y qué deberías revisar vos?

Le faltó crear el `.env` desde `.env.example`. Vos revisá que el README lo indique y que `.env.example` tenga todas las variables.

3. ¿Qué dos problemas tiene este fragmento?

compose.yaml

```yaml
db:
  image: mysql
  ports:
    - "3306:3306"
backend:
  environment:
    DB_HOST: localhost
```

(a) `mysql` sin etiqueta usa `latest` y publica la base en todas las interfaces, en el 3306 que probablemente ya ocupa un MySQL local; mejor `mysql:8.4` y `"127.0.0.1:3307:3306"`. (b) `DB_HOST: localhost` apunta al propio backend: debe ser `db`.

REPASO **Módulo 12 · Repaso y cierre**

Cuestionario (1 de 2)

1. ¿Qué es una imagen de Docker?

- Un contenedor detenido
- **Un paquete de solo lectura con lo necesario para ejecutar una aplicación ✓ correcta**
- Una máquina virtual comprimida

Un contenedor detenido sigue siendo un contenedor (con su capa de escritura). La imagen es la plantilla inmutable a partir de la cual se crean contenedores, y no incluye un kernel como una VM.

2. En `-p 8080:80`, ¿qué es el 8080?

- El puerto donde escucha la app dentro del contenedor
- **El puerto de tu compu por el que entrás ✓ correcta**
- El puerto de MySQL

El formato es `host:contenedor`. A la izquierda, tu compu; a la derecha, donde escucha la app adentro.

4. En Compose, ¿cómo se conecta el backend a MySQL?

- `localhost:3306`
- `127.0.0.1:3307`
- **`db:3306` ✓ correcta**

Nombre del servicio y puerto interno. `localhost` es el propio backend; el 3307 es el puerto publicado para tu compu.

5. ¿Qué borra `docker compose down -v`?

- Solo los contenedores
- **Contenedores, red y volúmenes del proyecto ✓ correcta**
- Las imágenes

Sin `-v`, los volúmenes quedan. Con `-v` se borran y con ellos los datos de la base. Las imágenes no se tocan.

3. ¿Qué hace `EXPOSE 3000`?

- Publica el puerto 3000 en tu compu
- Abre el firewall
- **Documenta que la app escucha en el 3000 ✓ correcta**

Es informativo. Para entrar desde fuera hace falta publicar con `-p` o `ports:`.

6. ¿Qué hace `depends_on: [db]` sin condición?

- Espera a que MySQL acepte conexiones
- **Solo inicia `db` antes que el servicio que depende de él ✓ correcta**
- Reinicia `db` si falla

Para esperar a que esté listo hace falta un `healthcheck` en `db` y `condition: service_healthy`.

REPASO Módulo 12 · Repaso y cierre

Cuestionario (2 de 2)

7. Cambiaste `MYSQL_PASSWORD` en `.env` después de que la base ya se había inicializado. ¿Qué pasa?

- MySQL toma la clave nueva
- **El backend recibe la clave nueva, MySQL conserva la vieja: `Access denied` ✓ correcta**
- MySQL se reinicializa solo

Las variables `MYSQL_*` se aplican solo con el volumen vacío. Hay que cambiar la clave con `ALTER USER` o recrear el volumen (perdiendo datos).

8. Angular servido por Nginx, ¿qué URL debería usar para la API?

- `http://backend:3000/api`
- **`/api` (relativa), con Nginx como proxy ✓ correcta**
- `http://db:3306`

El código de Angular corre en el navegador, que no conoce los nombres de Docker. La URL relativa va al mismo origen y Nginx la reenvía al backend: sin CORS.

9. Cambiaste código del backend. ¿Qué comando aplica el cambio?

- `docker compose restart backend`
- **`docker compose up -d --build` ✓ correcta**
- `docker compose logs backend`

`restart` reinicia el mismo contenedor con la misma imagen (código viejo). Hay que reconstruir la imagen.

10. ¿Dónde va la contraseña de la base de producción?

- En un `ENV` del Dockerfile
- En `.env.example`
- **En `.env.prod`, creado solo en el servidor ✓ correcta**

En el Dockerfile queda dentro de la imagen; `.env.example` se sube a Git. El archivo de producción vive solo en el servidor, con permisos restringidos.

11. ¿En qué se diferencia un contenedor de una máquina virtual?

- **El contenedor comparte el kernel del anfitrión ✓ correcta**
- El contenedor siempre es más seguro
- La VM no puede ejecutar Linux

Por compartir el kernel es más liviano y rápido. El aislamiento de una VM es, en general, más fuerte.

12. Un contenedor figura `Exited (1)`. ¿Qué hacés primero?

- Reinstalar Docker
- **Leer `docker logs` del contenedor ✓ correcta**
- Borrar todas las imágenes

El código 1 indica un error de la aplicación, y la explicación casi siempre está en sus registros.

REPASO Módulo 12 · Repaso y cierre

Guía de decisión: ¿qué uso para…?

Quiero…	Uso…
Probar una imagen existente (nginx, mysql)	`docker run`
Empaquetar mi aplicación	Un **Dockerfile** + `docker build`
Levantar mi app con su base de datos (y frontend)	`compose.yaml` + `docker compose up -d`
Que los datos sobrevivan a los contenedores	Un **volumen con nombre**
Que dos servicios se comuniquen	La misma **red** + el **nombre del servicio**
Entrar desde mi navegador o mi cliente MySQL	Un **puerto publicado** (`ports`), en `127.0.0.1` si es solo para mí
Cambiar configuración sin reconstruir	**Variables de entorno** (`.env` + `environment`)
Aplicar un cambio de código	`docker compose up -d --build`
Saber por qué algo falla	`docker compose ps` → `logs` → `config` → `exec`
Respaldar la base	`mysqldump` + `docker compose cp`, guardado fuera
Compartir mi imagen o desplegarla	Un **registro**: `docker push` / `docker compose pull`

REPASO Módulo 12 · Repaso y cierre

¿Mi proyecto está bien dockerizado?

- ✓ En una compu nueva se levanta con: `git clone` → `cp .env.example .env` → `docker compose up -d --build`.
- ✓ Imágenes base **oficiales** y con **versión**.
- ✓ Dockerfile: dependencias antes del código, `npm ci`, `USER node`, `CMD` con corchetes.
- ✓ `.dockerignore` excluye `node_modules`, `.env` y `.git`.
- ✓ No hay secretos en el repositorio, en el historial ni en las imágenes.
- ✓ `.env.example` completo y comentado.
- ✓ La base usa un **volumen con nombre** y los datos sobreviven a `down` + `up`.
- ✓ El backend se conecta por **nombre de servicio** y espera al `healthcheck`.
- ✓ Solo se publican los puertos necesarios; la base en `127.0.0.1` o sin publicar.
- ✓ La app falla rápido y claro si falta configuración.
- ✓ El README explica requisitos, cómo levantar, puertos, variables, cómo detener y problemas frecuentes.
- ✓ Alguien más lo probó siguiendo solo el README.

EXPLICACIÓN Módulo 12 · Repaso y cierre

Para seguir aprendiendo

Documentación oficial

- Docker: Get started (https://docs.docker.com/get-started/)
- Referencia de Dockerfile (https://docs.docker.com/reference/dockerfile/)
- Referencia de compose.yaml (https://docs.docker.com/reference/compose-file/)

Próximos temas

- **Desarrollo con recarga automática** en contenedores (bind mounts, `docker compose watch`).
- **Multi-stage para backends** (por ejemplo, compilar TypeScript y ejecutar solo el JavaScript).
- **CI/CD:** construir y publicar imágenes con GitHub Actions.

- Buenas prácticas para construir imágenes (https://docs.docker.com/build/building/best-practices/)
- Imágenes oficiales: node (https://hub.docker.com/_/node) · mysql (https://hub.docker.com/_/mysql) · nginx (https://hub.docker.com/_/nginx)

- **Seguridad de imágenes:** escaneo de vulnerabilidades y actualizaciones.
- **Orquestación:** cuando un servidor no alcanza (Kubernetes y similares).

REPASO **Módulo 12 · Repaso y cierre**

Cierre

Docker no cambia cómo programás. Cambia cuánto tarda otra persona —o un servidor— en tener tu aplicación funcionando igual que en tu compu.

Hoy aprendiste

Imágenes, contenedores, Dockerfile, Compose, volúmenes, redes, variables, deploy y diagnóstico.

Ahora toca

Dockerizar tu propio proyecto en la práctica integradora.

Tenés a mano

Esta presentación, la guía en PDF y los ejemplos probados del repositorio.

PRÁCTICA INTEGRADORA

Dockerizar tu propio proyecto

Que tu aplicación se levante en cualquier computadora con un solo comando, sin instalar Node ni MySQL a mano, sin exponer secretos y sin perder datos.

Camino A (recomendado)

Tu proyecto Express + MySQL (con o sin Angular), en una rama nueva: `git switch -c docker`.

Camino B

Si no tenés un proyecto adecuado: la API de biblioteca de `practica/proyecto-base`, que funciona sin Docker.

Duración y modalidad

2 a 4 horas. Individual o en parejas: cada integrante tiene que poder explicar todo.

ACTIVIDAD **Práctica integradora**

Objetivos mínimos

1. Identificar los componentes y dependencias de tu aplicación.
2. Crear un `Dockerfile` para el backend.
3. Crear la configuración de Compose.
4. Ejecutar MySQL en un servicio independiente.
5. Configurar variables de entorno sin exponer secretos.
6. Garantizar la persistencia de los datos.
7. Levantar el sistema con un comando.
8. Comprobar que la aplicación funciona.
9. Detener y volver a iniciar los servicios.
10. Documentar cómo reproducir el entorno en otra computadora.

IDEA CLAVE

El frontend Angular se dockeriza solo si forma parte del proyecto **y** el tiempo alcanza. Primero, backend + MySQL funcionando bien.

ACTIVIDAD Práctica integradora

Paso a paso, con puntos de control

#	Paso	✅ Punto de control
1	**Relevamiento:** tabla con componentes, versiones, puertos y variables (en tu README).	Sabés qué versión de Node y MySQL usa tu proyecto.
2	**Configuración por variables:** reemplazá `localhost`, usuarios y claves escritos en el código por `process.env.*`.	La app sigue funcionando sin Docker.
3	**Dockerfile + .dockerignore** del backend (base oficial con versión, `npm ci`, `USER node`).	`docker build -t mi-backend:1.0 .` termina sin errores.
4	**compose.yaml:** `db` + `backend`, volumen, red, `healthcheck`, `depends_on` con `service_healthy`, scripts en `db/init`.	`docker compose ps`: `db (healthy)` y backend `Up`.
5	**Variables:** `.env` (ignorado) y `.env.example` (versionado).	`git status` no muestra `.env`.
6	**Prueba funcional** de tus endpoints principales.	Respuestas correctas desde navegador, Postman o `curl`.
7	**Persistencia:** crear un dato → `down` → `up -d`.	El dato sigue.
8	**README** con la plantilla.	Otra persona lo levanta siguiendo solo el README, en una copia limpia.

Consigna completa: practica/README.md (https://github.com/pedroibanez97/docker/tree/main/practica) · Plantilla: PLANTILLA-README.md (https://github.com/pedroibanez97/docker/blob/main/practica/PLANTILLA-README.md)

ACTIVIDAD Práctica integradora

Camino B: el proyecto base

```
practica/proyecto-base/
├── package.json · package-lock.json
├── .env.example     # para correr SIN Docker
├── db/esquema.sql   # base "biblioteca" +
datos
└── src/
    ├── index.js     # Express + /api/salud
    ├── db.js        # pool mysql2 (lee
process.env)
    └── rutas/libros.js
```

Endpoints: `GET /api/salud`, `GET /api/libros`, `GET /api/libros/:id`, `POST /api/libros`.

Tenés que agregar

- ✓ `Dockerfile` y `.dockerignore`
- ✓ `compose.yaml` (API + MySQL, volumen, red, healthcheck)
- ✓ `.env` (ignorado) y `.env.example` para Docker
- ✓ Tu README

PREGUNTA PARA COMPROBAR

Pistas: ¿qué valor necesita `DB_HOST` dentro de Compose? ¿En qué carpeta del contenedor de MySQL va `esquema.sql`? ¿Qué nombre de base tiene que tener `MYSQL_DATABASE`?

`DB_HOST: db` (o el nombre que le des al servicio); `./db:/docker-entrypoint-initdb.d:ro`; `MYSQL_DATABASE: biblioteca`, porque el script crea y usa esa base y el usuario de la app necesita permisos sobre ella.

RESULTADO ESPERADO

Probado: el proyecto base se dockeriza con el mismo patrón del ejemplo 02 y responde `GET /api/libros` con los cuatro libros de ejemplo.

ACTIVIDAD Práctica integradora

Entregables

- ✓ Enlace al repositorio con el código actualizado.
- ✓ `Dockerfile` del backend (y del frontend, si corresponde) y `compose.yaml`.
- ✓ `.dockerignore` y `.env.example`.
- ✓ `README.md` con instrucciones de ejecución.
- ✓ **Evidencia de funcionamiento:** salida de `docker compose ps` y una respuesta de la API (o la app en el navegador).
- ✓ **Explicación breve de la arquitectura:** servicios, cómo se comunican, qué puertos se publican y por qué.
- ✓ **Demostración de persistencia:** un dato creado antes de `down` que sigue después de `up -d`.

ATENCIÓN

Revisá las capturas antes de entregarlas: que no muestren contraseñas (por ejemplo, de `docker compose config`).

ACTIVIDAD Práctica integradora

Rúbrica de evaluación (100 puntos)

Criterio	Pts	Insuficiente	Suficiente	Muy bueno
Comprensión y arquitectura	20	No explica qué hace cada servicio ni imagen vs contenedor.	Explica servicios, red y puertos con ayuda.	Explica con precisión por qué el backend usa el nombre del servicio y qué pasaría con `down -v`.
Funcionamiento	25	No levanta o la API no conecta con la base.	Levanta con un comando y los endpoints principales responden.	Además espera a la base (healthcheck), errores claros y frontend funcionando si existe.
Configuración Docker	20	Copia `node_modules`, usa `latest` o no hay `.dockerignore`.	Imagen oficial con versión, `npm ci`, `.dockerignore`, red y volumen.	Además aprovecha la caché, usuario no root y sin configuración innecesaria.
Persistencia	10	Los datos se pierden al recrear.	Volumen con nombre y demostración.	Además scripts de inicialización y backup probado.
Seguridad básica	15	Contraseñas reales o `.env` en el repositorio.	`.env` ignorado, `.env.example` sin secretos, la app no usa `root`.	Además la base no se publica o solo en `127.0.0.1`, y el README explica cómo generar claves.
Documentación	10	Sin README o no permite reproducir.	Requisitos, pasos, puertos y variables.	Además evidencia, arquitectura y problemas frecuentes.

ACTIVIDAD Práctica integradora

Criterios de aprobación

Para aprobar

- ✓ **60 puntos o más**, y
- ✓ al menos **Suficiente** en Funcionamiento, Persistencia y Seguridad básica, y
- ✓ ningún error bloqueante.

Errores que impiden considerar terminada la actividad

- ✗ No levanta siguiendo el README en una copia limpia.
- ✗ Secretos reales o `.env` en el repositorio, en cualquier commit.
- ✗ El backend no se conecta a MySQL (por ejemplo, usa `localhost`).
- ✗ Los datos se pierden con `down` + `up -d`.
- ✗ Falta el README o la evidencia.
- ✗ Quien entrega no puede explicar su `Dockerfile` ni su `compose.yaml`.

ACTIVIDAD Práctica integradora

Lista de verificación final (antes de entregar)

- ✓ Cloné mi repositorio en una carpeta nueva y seguí **solo** mi README.
- ✓ `docker compose up -d --build` funciona desde cero (probé antes con `down -v`).
- ✓ `docker compose ps` muestra la base `(healthy)` y el backend `Up`.
- ✓ Los endpoints principales responden correctamente.
- ✓ Un dato creado sobrevive a `docker compose down` + `up -d`.
- ✓ `git log --all -- .env` no muestra nada: nunca subí el `.env`.
- ✓ `.env.example` tiene todas las variables, sin valores reales.
- ✓ `.dockerignore` excluye `node_modules`, `.env` y `.git`.
- ✓ Las imágenes base tienen versión explícita.
- ✓ El backend se conecta por nombre de servicio.
- ✓ Solo publico los puertos necesarios (base en `127.0.0.1`).
- ✓ El README tiene arquitectura, puertos, variables, cómo detener, problemas frecuentes y la evidencia.